Système domotisé ouvert et déployable pour les espaces de travail au sein de petites et moyennes entreprises

Patrick Pagnoulle
Bachelier en Automatisation
2020-2025

IRAM
Enseignement pour Adultes

Tout d'abord, je tiens à remercier toute mon équipe pour m'avoir encouragé toutes ces années dans l'accomplissement de mon changement de métier, dans mes études et pour avoir toujours été présente pour moi. C'est une équipe en or qu'on n'a pas toujours la chance d'avoir dans le milieu du travail.

Je remercie également Gilles Bertrand pour m'avoir fait confiance en m'intégrant dans celle-ci, pour m'avoir soutenu dans le projet, pour avoir été présent à chaque étape et pour m'avoir encouragé dans les moments les plus difficiles. Il a toujours su être à et prodiguer des conseils afin de parvenir aux solutions adéquates tout au long du processus.

Je remercie spécialement Monsieur Emmanuel Godissard d'avoir accepté d'être mon promoteur. Du début du projet jusqu'à la présentation de celui-ci , il a toujours répondu présent à chaque sollicitation de ma part en me fournissant des conseils pertinents avec énormément de bienveillances.

Je remercie toute l'équipe pédagogique pour ces quatre années d'études passées à leurs côtés. Ils ont toujours été présents dans toutes les difficultés que peuvent comporter des études, surtout avec une classe qui a démarré en visioconférence en pleine période de pandémie.

Pour finir, un grand merci à mes relecteurs et relectrices, Stéphanie Formaglio, Leila Nunez Derenoncourt, Joëlle Hoquart, Rémi Marcelle, Vincent Deulin Campion, Amaury Deflorenne qui ont pris le temps de compulser ce dossier et de m'avoir donné leurs conseils et corrections.

Table des matières

1 Présentation personnelle 7

2 Présentation de l'entreprise 8

2.1 Historique et activités principales 8

2.2 Organisation et équipe 8

2.2.1 Organigramme 8

2.2.2 Présentation de l'équipe 9

2.3 Rôle et missions au sein de l'entreprise 9

2.4 Lien entre l'entreprise et le projet de domotique 10

3 Introduction du projet 10

3.1 Contexte du projet 10

3.2 Problématique 10

3.3 SWOT 11

3.4 Objectif du projet présenté 12

3.5 Méthodologie de travail 12

3.6 Vers un système domotique ouvert et déployable pour les espaces de travail 13

3.6.1 Exemple de plateformes existantes 13

3.6.2 Critères Clés 13

3.6.3 Axes d'amélioration 14

3.7 Exemples de principes 14

3.7.1 Gestion de l'environnement d'un bureau 14

3.7.2 Porte à ouverture par RFID 15

3.7.3 Supervision de l'Énergie Électrique avec un Compteur 16

4 Cahier des charges 17

4.1 Contexte 17

4.2 Périmètre fonctionnel de présentation 18

4.2.1 Technique : 18

4.2.2 Visuel 18

4.2.3 Exigences fonctionnelles 19

4.2.4 Exigences non-fonctionnelles 19

4.3 Schémas de principe 20

4.4 Matériel utilisé : 23

4.5 MoSCoW 23

4.5.5 Must Have (Indispensable) 23

4.5.6 Should Have (Important mais pas essentiel au démonstrateur) 24

4.5.7 Could Have (Si le temps permet de le développer) 24

4.5.8 Won't Have (Hors du périmètre actuel) 24

4.6 Pour Finir … 25

5 Mise en œuvre 26

5.1 Le matériel 26

5.1.1 Platine de démonstration 26

5.1.2 ESP32 pour émuler des températures 29

5.1.3 Enregistreur de badge 29

5.1.4 Lecteur de badges (accès porte) 30

5.1.5 Badges Mifare Classic 1K 31

5.2 Schéma de communication 31
5.3 Installation d'un linux sur le raspberry Pi et mise en place du serveur Mosquitto 32
5.4 Création du backend node.js 33
5.5 Simulateur de température 35
5.6 Mise en place du compteur d'énergie 35
5.6.1 Prise en main du convertisseur 35
5.6.2 Résolution des problèmes de communication 37
5.6.3 Mises en place avec le LOGO 37
5.7 L'enregistreur et le lecteur de badges Mifare. 40
5.7.1 Le backend 40
5.7.2 L'enregistreur de badge 40
5.7.3 Processus d'enregistrement de badge 41
..... 41
5.7.4 Le cryptage d'un badge 42
5.7.5 Le Lecteur de badges 42
5.7.6 User story 43
5.7.7 Processus de lecture de badge 44
5.8 Configuration du broker Mosquitto avec un port sécurisé (SSL/TLS) 45
5.9 Le refactor du backend 45
5.9.1 Acceptation des deux ports MQTT (1883 et 8883) 45
5.9.2 La petite erreur qui bloque le système 46
5.10 Rendre le code plus propre 46
5.11 Mise en place de la première sécurité 46
5.12 L'interface utilisateur 48
5.12.1 La connexion 48
5.12.2 L'onglet Utilisateurs. 49
5.12.3 L'onglet appareils. 50
5.12.4 L'onglet tableau de bord 51
5.12.5 L'onglet Journal d'accès 52
6 Le Déploiement de l'application 52
6.1 La préparation 53
6.1.1 Création une clé SSH 53
6.1.2 Création du fichier config 53
6.1.3 Authorized_keys 53
6.2 Préparation de docker sur le Rapsberry Pi 53
6.2.4 Ajouter la clé Docker officielle 53
6.2.5 Ajouter le dépôt aux sources Apt 54
6.3 Préparation de l'application 55
6.3.6 L'environnement 55
6.3.7 Les DockerFile (en Annexe) 55
6.3.8 Le docker compose 55
6.3.9 Les scripts 55
6.4 Procédure de mise en production 56
6.5 In Fine 57
7 Sécurité du système 57
7.1 Le matériel physique 57
7.2 Le réseau 58
7.3 Le code de l'application 58

8 Résultats 59
9 Conclusion et perspectives 60
ANNEXES 61
Annexe A. Retro planning 61
Annexe B. Protocoles de communication. 63
- MQTT : 63
- Modbus : 64
- Http : 65
- Websocket : 65
- UART : 66
- TCP/IP : 67
Annexe C. Les langages de programmation 68
- FBD : 68
- JavaScript / TypeScript 69
- C++ : 70
Annexe D. Frameworks et librairies importantes 71
- Frontend : 71
- Backend : 71
Annexe E. Logiciels utilisés lors de la conception 73
Annexe F. Étape de création du backend 74
Annexe G. Schémas Backend/Frontend 76
Annexe H. Littérature 81
- MQTT : 81
- http : 81
- uart : https://fr.wikipedia.org/wiki/UART 81
- tcp/ip 81
- FSB 81
- Javascript 81
- Frontend 82
- Backend 82
- DB 82
- Git et github 82
Annexe I. Coût total du matériel après développement 83
Annexe J. Sketch arduino 84

A – Simulation température. *84*
B – Lecteur de badges *89*
C – Exemple de libraire pour le led *96*

1 Présentation personnelle

Dès mes débuts professionnels, j'étais passionné par l'informatique et le hardware, ce qui m'a conduit à travailler comme conseiller dans le domaine. Par la suite, je suis devenu technicien dans le théâtre alimentant toujours cette passion pour les machines et pour la technique. Cela m'a amené à entreprendre des études en Automatique à l'Iram (IRAM Enseignement pour Adultes). En parallèle, j'ai suivi une formation en développement frontend pour des applications web, puis j'ai effectué mon stage au sein de l'entreprise Triptyk, où je travaille depuis plus de deux ans.

Avec l'expérience, j'ai acquis de « solides » compétences en frontend grâce au rigoureux framework EmberJs. Ce qui demande une grande précision et méthodologie. Avec le temps, j'ai développé des compétences en backend, particulièrement en Node.js, ce qui fait que je suis devenu développeur fullstack et que je suis capable de maîtriser plusieurs technologies JavaScript.

Grâce aux cours à l'Iram, j'ai pu apprendre beaucoup de choses, mais il était nécessaire de consolider mes compétences en automatique. La suite de mes objectifs a donc été de combiner ces compétences en développement avec mes nouvelles connaissances gagnées avec mes études. Partant de cette idée et en discutant domotique avec mon patron, j'ai élaboré un projet de système domotisé simple et scalable, me permettant de plonger dans les protocoles tels que Modbus, MQTT, etc. J'ai pu ainsi coder avec un automate programmable Siemens Logo, des microcontrôleurs, du JavaScript et créer des communications bidirectionnelles entre les outils utilisés.

2 Présentation de l'entreprise

2.1 Historique et activités principales

Existante depuis 2008, TRIPTYK[1] est une entreprise spécialisée dans la transformation numérique. Elle accompagne les entreprises en retard dans la numérisation de leurs documents et les aide à automatiser et informatiser leurs processus pour leur faciliter le travail au quotidien. Elle intervient aussi bien sur des sites comme Crehopa et Technocité que sur des applications ambitieuses comme Reaktiv et Mediacio.

En plus du développement web et applicatif, TRIPTYK propose aussi du conseil, du coaching digital et des formations pour aider les entreprises à améliorer leurs compétences numériques et à optimiser leurs outils.

Un des aspects importants de l'entreprise est sa conscience écologique, notamment en ce qui concerne l'énergie consommée dans la gestion des données. TRIPTYK s'engage donc dans l'intégration de technologies économes en ressources et cherche à atteindre une autonomie énergétique en mettant en place des solutions éco-énergétiques dans ses infrastructures.

2.2 Organisation et équipe

2.2.1 Organigramme

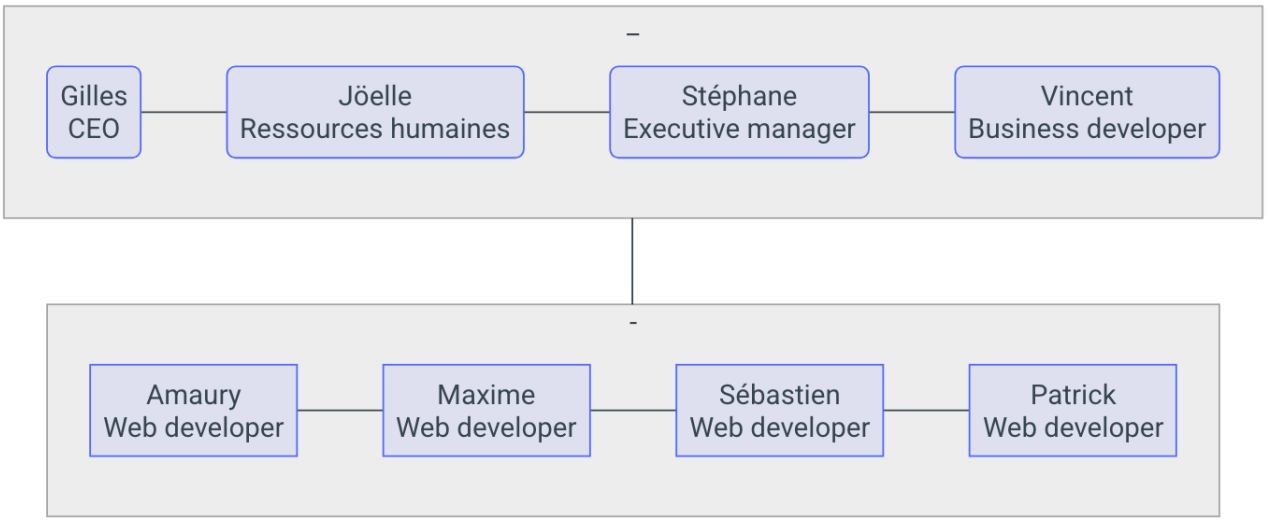


Figure n°1: organigramme

[1] www.triptyk.eu

2.2.2 Présentation de l'équipe

Gilles Bertrand : fondateur de l'entreprise Triptyk, il gère l'ensemble des projets et fait le lien avec les clients. Il fait aussi de la consultance externe et transmet son savoir au travers de formation dans le domaine de l'informatique et de la programmation.

Joëlle Hoquart : Elle est chargée du bon fonctionnement administratif interne et externe de l'entreprise.

Vincent Deulin-Campion : Vincent Deulin-Campion : Consultant externe engagé dans Triptyk. Il est une personne centrale de l'équipe. Il s'occupe de la bonne gestion des projets en utilisant la méthode agile. Il fait le lien entre le client et les développeurs. Il est aussi formateur en méthode scrum et agile

Stéphane Meermans : Consultant externe engagé au sein de Triptyk aussi. Il est l'œil UX/UI de la boite. Il prépare les projets afin que ceux-ci soient le plus ergonomique et user friendly possible. C'est aussi un formateur UX/UI et en logiciels son, car il est aussi musicien.

Amaury Deflorenne : lead developer. Amaury est le ciment des projets. Il choisit les technologies et il fait les déploiements pour les clients. C'est un véritable couteau suisse. Il a une grande connaissance dans la plupart des technologies web mais il est aussi capable de travailler dans d'autres langages tels que le C/C++, le Rust, le Java, le PHP… il est un support pour nous tous. Il est aussi formateur dans des centres de formation surtout sur la partie backend.

Sébastien Cardon : Lead frontend. Sébastien est un lien avec le client concernant les sites, il s'occupe aussi du backend mais sa force réside dans sa résilience en frontend. Il peut switcher entre les technologies ou en utiliser de nouvelles avec une grande agilité, une souplesse intellectuelle et une extrême efficacité. Lui aussi est formateur, plutôt côté frontend.

Maxime Godard Hupets : UX/UI. Maxime est un chef en design au service de l'utilisateur, capable de faire de la logique frontend. Il allie le développement au designer. C'est lui qui fait en sorte que tout soit harmonieux sur les projets en développement. Maxime est aussi un formateur efficace et plein d'énergie en frontend et en UX/UI.

2.3 Rôle et missions au sein de l'entreprise

Je suis donc rentré comme développeur frontend janvier 2023 et avec le temps, je suis devenu fullstack, c'est-à-dire frontend et backend en même temps. Je suis encore la jeune recrue de la boite. Je commence doucement à m'occuper de certains déploiements en production surtout des mises à jour de projet en place. J'ai aussi une place en tant que formateur en centre. Plutôt axé algorithme javascript et aussi en svelte.

2.4 *Lien entre l'entreprise et le projet de domotique*

Les premiers aspects qui relient le projet et l'entreprise sont la durabilité et l'optimisation énergétique, qui sont au cœur des valeurs de Triptyk. Dans cette logique, TRIPTYK souhaite se développer dans des projets s'inscrivant dans une démarche plus large visant à réduire l'empreinte énergétique de ses infrastructures et à expérimenter des solutions innovantes en automatisation et gestion des consommations. Le second aspect concerne la traçabilité des accès des zones sensibles telles que les archives. À travers cette approche, l'entreprise prend à cœur de vouloir optimiser ses bureaux sans pour autant que les coûts ne deviennent un poids trop important et, si possible, à mettre en place une solution qui pourra dans le futur aider d'autre PME à suivre une démarche identique pour leurs propres locaux.

3 Introduction du projet

3.1 *Contexte du projet*

C'est dans cette perspective que s'inscrit le projet. Celui-ci a pour objectif de créer une solution qui ne soit pas trop coûteuse pour l'entreprise ni pour de futures entreprises clientes, tout en restant flexible grâce à son adaptabilité et à une mise en place rapide et efficace lors du déploiement. La solution offrirait également la possibilité d'intégrer simplement des éléments de domotique (ex. : éclairage Ikea, enceinte connectée Sonoff, etc.). Et, qui sait, à plus long terme, proposer une maintenance ainsi qu'un service client permettant un suivi régulier, avec des mises à jour transparentes de l'application et une expertise technique évolutive.

3.2 *Problématique*

Dans le milieu professionnel, les solutions offertes sont souvent des solutions propriétaires fermées ce qui tend à obliger le client à s'enfermer dans un écosystème qui ne permet pas de choisir des solutions qui leur conviennent. Par exemple, avec la solution UniFi d'Ubiquiti[2], il est impossible d'intégrer des éléments externes à leur écosystème sans passer par une autre plateforme telle que Home Assistant[3], Jeedom[4], etc

Comme les solutions propriétaires sont souvent coûteuses et qu'avec l'arrivée du DIY[5] et des microcontrôleurs bon marché, de nouvelles possibilités apparaissent, l'idée a été de voir s'il était possible de créer une solution intermédiaire et flexible, tout en gardant la robustesse et la pertinence des solutions plus onéreuses.

[2] https://ui.com/
[3] https://www.home-assistant.io/
[4] https://jeedom.com/
[5] Do It Yourself: est une philosophie visant à créer, à réparer ou à modifier des objets de la vie courante par soit même

3.3 SWOT

Forces (Strengths)	Faiblesses (Weaknesses)
• **Modularité et compatibilité avec objets connectés ouverts** Support natif de modules gratuits comme via Home Assistant (Sonoff, IKEA Tradfri, ESP32…), permettant aux clients d'ajouter et gérer des équipements tiers sans coût supplémentaire dans une interface beaucoup plus simple que les solutions gratuites. • **Tout comme les solutions propriétaires tout-en-un** Acteurs comme UniFi ou Samsung SmartThings offrent une gestion réseau et IoT intégrée, avec une interface unique et un support global, attirant les clients recherchant une expérience homogène. • **Solution clé en main et simple à déployer** Conçue pour être installée rapidement avec un serveur pré-configuré et une interface épurée pour la gestion d'éléments à risque comme la traçabilité et d'autres fonctionnalités propres aux clients.	• **Le client est dépendant de notre propre évolution** Les nouvelles fonctionnalités et protocoles sont planifiés selon notre roadmap. • **Surcoûts potentiels pour développements spécifiques** Intégrations ou adaptations particulières peuvent nécessiter des développements facturés en sus. • **Absence de garantie couvrant les tiers** La garantie et le support portent sur notre solution, mais nous ne serons pas en mesure de garantir le bon fonctionnement des modules IoT tiers. Les dysfonctionnements liés à ces équipements peuvent générer des tickets de support hors périmètre.
Opportunités (Opportunities)	**Menaces (Threats)**
• **Expansion vers les PME** Les petites entreprises et les particuliers avertis recherchent des solutions professionnelles abordables, souples et adaptables à leur croissance. • **Valeur ajoutée via services de sécurité** Proposition d'audits de sécurité, de hardening réseau et de conformité RGPD en option, générant des revenus additionnels. • **Intégration de nouveaux standards IoT** Adoption rapide de protocoles émergents (Matter[6], Thread[7]) pour offrir une compatibilité étendue et future-proof.	• **Évolution rapide des technologies** Les nouveaux standards et matériels nécessitent une veille constante et des mises à jour régulières. • **Contraintes réglementaires** Exigences RGPD et sécurité fonctionnelle imposent des délais et coûts de certification.

[6] Standard de connectivité permettant aux appareils de différentes marques (Amazon, Apple, Google, etc.) de fonctionner ensemble.

[7] Basé sur des technologies sans fil conçu pour l'Internet des objets, il renforce la fiabilité et la puissance de Matter.

3.4 Objectif du projet présenté

La suite de ce document présente le développement d'un démonstrateur de supervision de la consommation d'énergie au sein des locaux de l'entreprise Triptyk. Cette approche combine la gestion énergétique du bâtiment avec un volet de traçabilité des accès pour les zones sensibles, allant de lieux de stockage de biens précieux jusqu'à de petites salles serveurs. Si les résultats s'avèrent concluants, l'objectif serait d'explorer l'opportunité de proposer cette solution à d'autres PME, qu'il s'agisse de bureaux, de centres d'affaires, d'espaces de coworking, etc…

Son objectif est d'évaluer la pertinence de ce projet sur un chantier plus important, que ce soit en termes de temps homme ou de coût de développement. Il s'agit également de déterminer si l'entreprise pourra entrer dans une phase plus conséquente et, à plus long terme, de voir si ce projet apportera une réelle plus-value, aussi bien en interne que comme solution pouvant être proposée à d'autres petites et moyennes entreprises.

3.5 Méthodologie de travail

L'approche choisie est itérative et organisée autour d'objectifs. Au début du projet, un cahier des charges a été établi en fonction des besoins identifiés, tout en conservant une logique agile. C'est-à-dire que celui-ci n'avait pas pour vocation de définir un objectif final figé, il est ici rédigé afin de servir de support à la conceptualisation du démonstrateur. Grâce à lui, des objectifs clés ont pu être définis et, selon les problématiques rencontrées ou les opportunités apparues en cours de développement, ils ont pu être adaptés aux nouveaux besoins. Par exemple, l'idée d'intégrer un lecteur de badges a été envisagée, mais la nécessité de gérer l'enregistrement préalable de badges sécurisés a été omise — un point sur lequel nous reviendrons plus loin dans ce dossier.

Pour résumer, par étape on va:

1 – Élaborer un cahier des charges simple.
2 – Créer un MoSCow[8] pour pouvoir établir une liste des objectifs principaux au secondaire.
3 - Mettre en place un retro-planning basé sur les objectifs.
4 - Produire et tester tout en s'adaptant aux contraintes.
5 - Faire des présentations du livrable au PO[9] afin d'avoir un feedback.

[8] MoSCow [Must, Should, Could, Won't]: est une méthode visant à établir les objectifs au cours du développement d'un projet.
[9] PO [Product Owner] : celui-ci a pour tâche de représenté les utilisateurs et les clients. Il est responsable de la vision du produit et il s'assure que celui-ci répond au besoin utilisateurs.

3.6 Vers un système domotique ouvert et déployable pour les espaces de travail

L'objectif d'un système domotique pour des espaces partagés ou des bureaux est d'avoir un système modulable et sécurisé tout en garantissant une interopérabilité, une simplicité de déploiement et d'utilisation finale.

3.6.1 Exemple de plateformes existantes

- Open source (Home Assistant, OpenHAB[10]) :

Celles-ci sont basées sur des communautés très actives ce qui leur permet de bénéficier d'une vaste bibliothèque de plugins. Cependant, cela donne des applications parfois complexes à utiliser et souffrant d'un certain manque d'ergonomie. Leur déploiement peut également prendre du temps. La gestion quotidienne de ces applications est aussi énergivore car étant communautaire, il peut y avoir beaucoup de mises à jour.

- Propriétaire (KNX[11], Somfy[12]) :

Ces systèmes domotiques sont fermés et nécessitent l'utilisation de hubs propriétaires pour pouvoir être interconnectés. Généralement, les interfaces sont plus simples et plus faciles à mettre en place. En revanche, l'abondance de matériel nécessaire afin de les rendre interopérables et le fait que ça soit propriétaire rend les coûts de mise en place et de maintenance assez exorbitants.

3.6.2 Critères Clés

Critère	**Exigence**
Interopérabilité	Support natif de MQTT, Modbus
Modularité	Ajout/retrait de capteurs sans refonte
Sécurité	Chiffrement, authentification

[10] https://www.openhab.org/
[11] https://knx-automation.com/
[12] https://www.somfy.be/fr-be/

3.6.3 Axes d'amélioration

L'objectif du projet est d'ouvrir la voie vers une architecture hybride légère, embarquant différents protocoles avec une intégration simplifiée. Il s'agit de proposer une interface de déploiement basée sur des conteneurs Docker[13], préconfigurés et prêts à l'utilisation, avec la possibilité de connecter des éléments de toute marque, à condition qu'ils soient compatibles avec une gestion centralisée via une interface web.

En résumé, le projet se veut ambitieux en proposant une solution clé en main, capable de combiner plusieurs protocoles industriels et des objets connectés grand public.

3.7 Exemples de principes

3.7.1 Gestion de l'environnement d'un bureau

3.7.1.1 Capteur de température

Principe : Le capteur de température mesure la température ambiante et transmet ces données au système domotique. Les données sont ensuite affichées sur un tableau de bord visuel et utilisées pour contrôler des équipements comme le chauffage ou la climatisation.

Matériel nécessaire :

1. **Capteur de température** : DHT22.
2. **Microcontrôleur** : Arduino et Raspberry Pi.
3. **Backends** : Serveur Node.js avec MQTT.
4. **Base de données** : PostgreSQL.[14]
5. **Frontend application** : Framework EmberJs[15].
6. **Communication** : WiFi Module (ESP32).

Exemple de fonctionnement :

1. Le capteur de température mesure la température ambiante.
2. Les données sont envoyées au microcontrôleur.
3. Le microcontrôleur envoie au Raspberry Pi.
4. Le backend récupère les données et les enregistre dans la base de données.
5. L'application frontend récupère les données de la base de données et les affiche sur un tableau de bord visuel.

[13] https://www.docker.com/
[14] https://emberjs.com/
[15] https://emberjs.com/

3.7.1.2 Capteur de luminosité

Principe : Le capteur de luminosité mesure l'intensité lumineuse et transmet ces données au système domotique. Les données sont affichées sur un tableau de bord visuel et utilisées pour contrôler des équipements comme les volets ou l'éclairage.

Matériel nécessaire :

1. **Capteur de luminosité** : BH1750 ou LDR (Light Dependent Resistor).
2. **Microcontrôleur** : Arduino et Raspberry Pi.
3. **Backend** : Serveur Node.js avec MQTT.
4. **Base de données** : PostgreSQL.
5. **Frontend application** : Framework EmberJs.
6. **Communication** : WiFi Module (ESP32).

Exemple de fonctionnement :

1. Le capteur de luminosité mesure l'intensité lumineuse.
2. Les données sont envoyées au microcontrôleur.
3. Le microcontrôleur envoie au raspberry.
4. Le backend récupère les données et les enregistre dans la base de données.
5. L'application frontend récupère les données de la base de données et les affiche sur un tableau de bord visuel.

3.7.2 Porte à ouverture par RFID

Principe : Le système RFID permet de contrôler l'accès à la porte en scannant un badge RFID. Les informations du badge sont vérifiées et, si valides, la porte est déverrouillée. Il pourrait être nécessaire de sécuriser une zone particulière du bâtiment.

Matériel Nécessaire :

1. **Lecteur RFID** : RC522.
2. **Microcontrôleur** : Arduino et Raspberry Pi.
3. **Relais** : Pour contrôler la serrure électrique.
4. **Serrure Électrique** : Serrure magnétique ou motorisée.
5. **Backend** : Serveur Node.js avec MTTQ.
6. **Base de Données** : PostgreSQL.
7. **Frontend Application** : Framework React.
8. **Communication** : WiFi Module (ESP32).

Exemple de Fonctionnement :

1. Le badge RFID est scanné par le lecteur.
2. Le lecteur envoie les données au microcontrôleur.
3. Le microcontrôleur vérifie les données via l'API.
4. Le backend valide les informations et renvoie la réponse.
5. Si le badge est valide, le microcontrôleur active le relais pour déverrouiller la porte.
6. Les informations d'accès sont enregistrées dans la base de données pour des raisons de sécurité et de suivi.

3.7.3 Supervision de l'Énergie Électrique avec un Compteur

Principe *:* Le système de supervision de l'énergie électrique permet de surveiller la consommation d'électricité en temps réel à l'aide d'un compteur modulaire. Les données collectées sont envoyées à un backend qui les enregistre dans une base de données et les affiche sur un tableau de bord visuel. Cela permet de suivre la consommation énergétique, d'identifier les pics de consommation et de prendre des mesures pour optimiser l'utilisation de l'énergie.

Matériel Nécessaire

1. **Compteur Modulaire d'Énergie Électrique** :
    - Exemples : Schneider Electric iEM2000,
    - Fonctionnalités : Mesure de la consommation électrique (kWh), courant, tension, puissance, etc.
2. **Microcontrôleur** :
    - Exemples : Arduino, ESP8266, ESP32, Raspberry Pi.
    - Fonctionnalités : Collecte des données du compteur et transmission au backend.
    - **Module de Communication** :
    - Exemple : ESP32
    - Fonctionnalité : Envoi des données collectées au backend via Internet.
3. **Backend** :
    - Exemples : Serveur Node.js ou Rust.
    - Fonctionnalités : Traitement des données, stockage dans la base de données, fourniture des données à l'application frontend.
4. **Base de Données** :
    - Exemple : PostgreSQL
    - Fonctionnalité : Stockage des données de consommation énergétique.
5. **Frontend Application** :
    - Framework : React
    - Fonctionnalités : Affichage des données de consommation sur un tableau de bord visuel, graphiques, alertes.

Exemples de fonctionnement

1. **Collecte des Données** :
   - Le compteur modulaire mesure la consommation électrique en temps réel et transmet les données au microcontrôleur.
2. **Transmission des Données** :
   - Le microcontrôleur, équipé d'un module de communication WiFi envoie les données au backend via une API.
3. **Traitement et Stockage** :
   - Le backend reçoit les données, les traite et les enregistre dans la base de données.
4. **Affichage des Données** :
   - L'application frontend récupère les données de la base de données et les affiche sur un tableau de bord visuel. Le tableau de bord permet de visualiser les données en temps réel, de consulter l'historique de la consommation, et de configurer des alertes en cas de dépassement de seuils définis.

4 Cahier des charges

4.1 Contexte

Ce projet s'inscrit dans le cadre de la modification des espaces de travail en espace domotisé afin de les rendre plus confortables, sécurisés et avec une meilleure gestion énergétique de ceux-ci. Le projet vise à concevoir un démonstrateur destiné à un petit environnement de travail (open-space ou bureau, ou PME, petits bâtiments). Le but est de voir la pertinence et la viabilité d'un projet de cette envergure. Ce projet s'inscrit dans un intérêt aussi bien pour une clientèle que pour sa propre infrastructure.

- **Environnement cible**

  - **Espaces collaboratifs** (salle de réunion, open-space) équipés de capteurs de température, luminosité et présence et d'actionneurs pour l'éclairage, les stores et …
  - **Zones sensibles** (salle serveur, archives, bureau d'études, fiduciaire) intégrant capteurs (fumée, inondation, intrusions) et contrôles d'accès (RFID, verrous automatisés, caméras).

- **Contraintes initiales**

  - **Budget limité** : matériel IoT standard, prototypage rapide.

- **Réseau d'entreprise** : coexistence sécurisée des VLAN[16] IoTs[17] et IT, chiffrement TLS[18], authentification JWT[19].
- **Normes et conformité** : RGPD[20] pour les données personnelles (caméras, accès), bonnes pratiques IT (logging, backups).

4.2 Périmètre fonctionnel de présentation

4.2.1 Technique :

- **Pilotage d'un chauffage**: régulation automatique selon présence et consignes de température et lecture de capteurs.
- **Contrôle d'accès** : serrures connectées, historique des ouvertures/fermetures.
- **Surveillance et alertes** : notifications temps réel (e-mail) en cas d'anomalie.
- **Ajout d'objets connectés personnalisés :** prise en charge directement à l'ajout sans besoin de redémarrer le système.

4.2.2 Visuel

- **Tableau de bord**: avec visualisation dans le temps de la consommation d'énergie.
- **Gestion d'utilisateur** : Création et gestion d'utilisateur avec rôle.
- **Gestion de badge d'accès** : Création et révocation du badge par utilisateur.
- **Gestion des objets connectés** : Création d'IoTs personnalisés.

[16] VLAN [Virtual Local Area Network] : technologie permettant de créer plusieurs réseaux logiques distincts au sein du même réseau physique

[17] IoT [Internet of Things] : maillage d'objets connectés entre eux et à internet comme les capteurs, cameras, détecteurs, alarmes… capables de communiquer et recevoir des données et pouvant être supervisé à distance

[18] TLS [Transport Layer Security] : protocole de sécurité assurant la confidentialité, l'intégrité et l'authentification des échanges de données sur un réseau reposant sur et le chiffrement de celles-ci et sur des certificats numériques.

[19] JWT [JSON web token]: standard ouvert défini dans la RFC 7519 permettant l'échange sécurisé de jetons (tokens) entre plusieurs parties.

[20] RGPD [*Règlement Général sur la Protection des Données*] : règlementation européenne entrée en vigueur milieu 2018 encadrant le traitement et la circulation des données personnelles. Elle a pour objectif de garantir la protection de la vie privée des citoyens

4.2.3 Exigences fonctionnelles

- **Interface web** intuitive et multilingue.
- **Gestion des utilisateurs :** création et mise à jour utilisateur avec rôle.
- **Création et gestion du badge d'accès.**
- **Gestion des objets connectés :** création et mise à jour.
- **Accès multi-utilisateur** : niveaux de permissions (admin, utilisateur, invité).
- **Access logs** : pour la surveillance.
- **Data logs :** de toutes les données de capteurs et de compteurs d'énergie.

4.2.4 Exigences non-fonctionnelles

- **Sécurité des données** : chiffrement TLS, authentification forte (2FA).
- **Scalabilité** : prise en charge de dizaines à centaines de capteurs/objets.
- **Interopérabilité** : protocoles standards (MQTT, Zigbee, Modbus, TCP/IP).

4.3 Schémas de principe

Figure n°2: schéma d'enregistrement et utilisation de cartes sécurisées

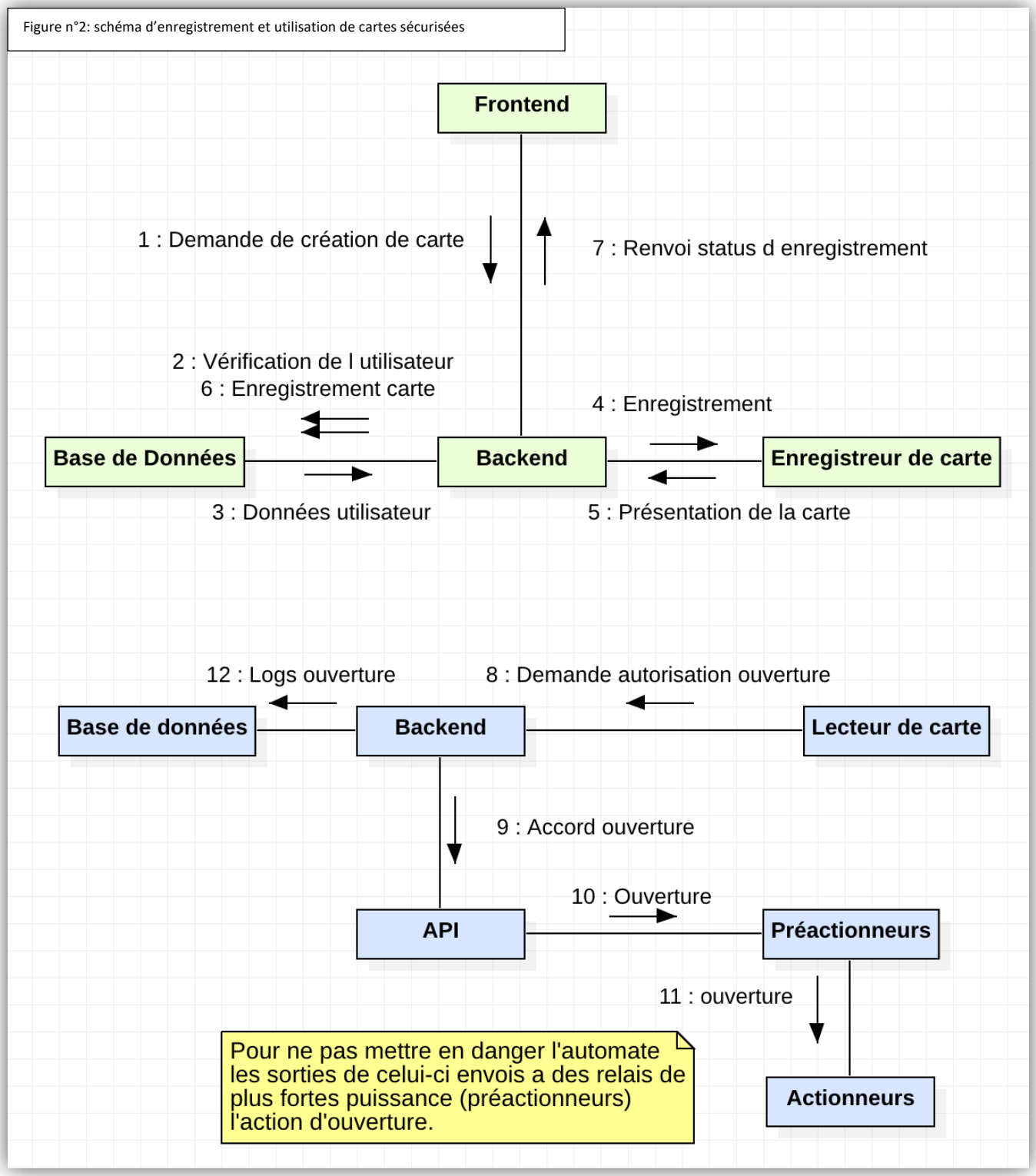


Figure n°3 : schéma de communication des statistiques

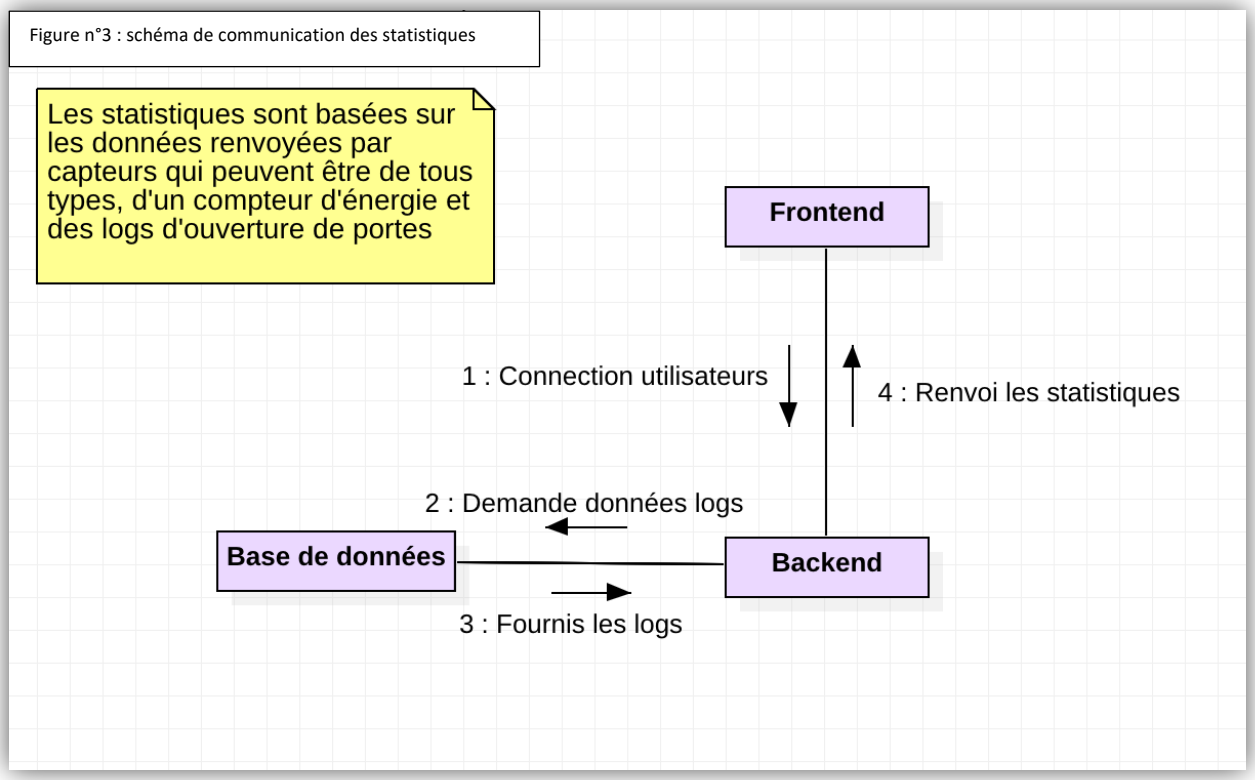


Figure n°4: schéma d'enregistrement de données des capteurs

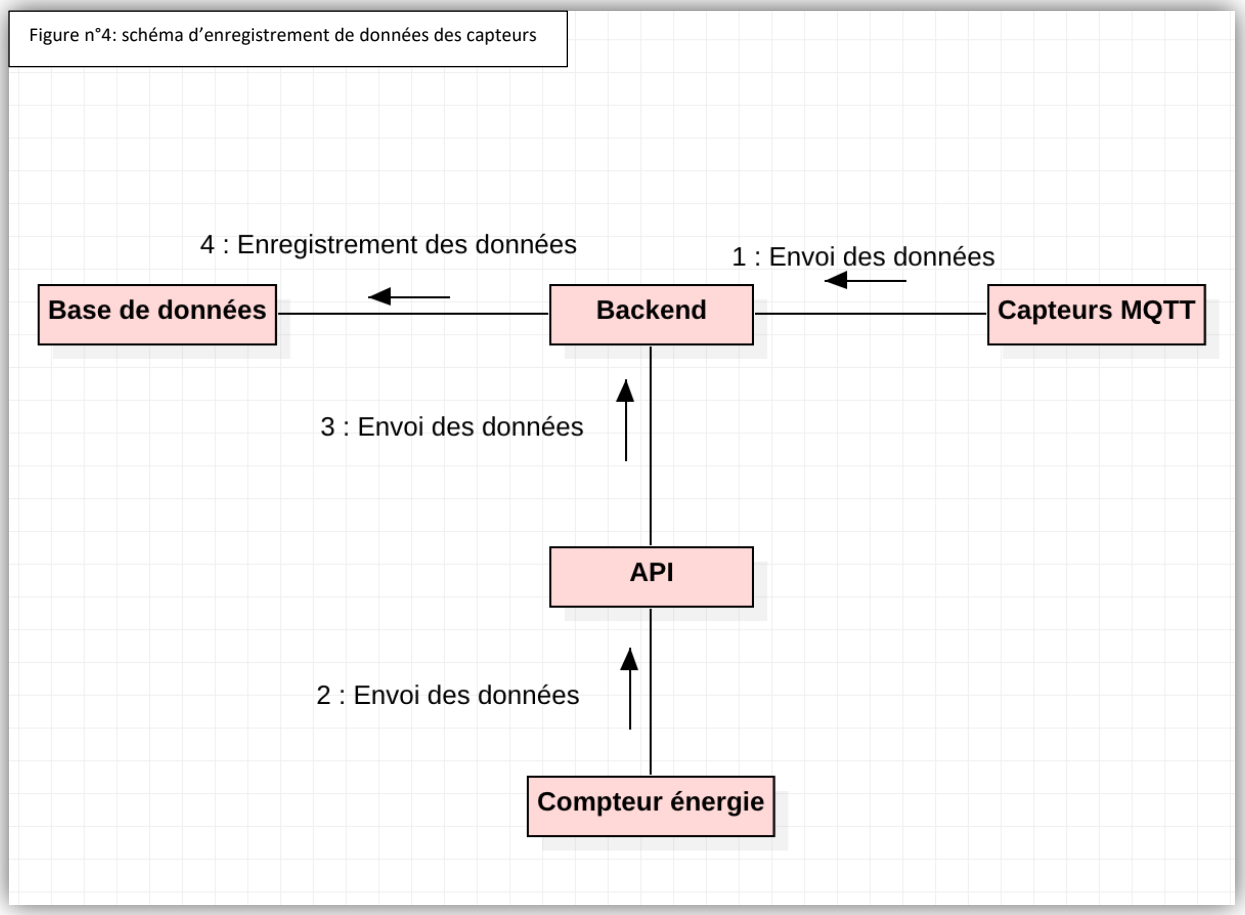
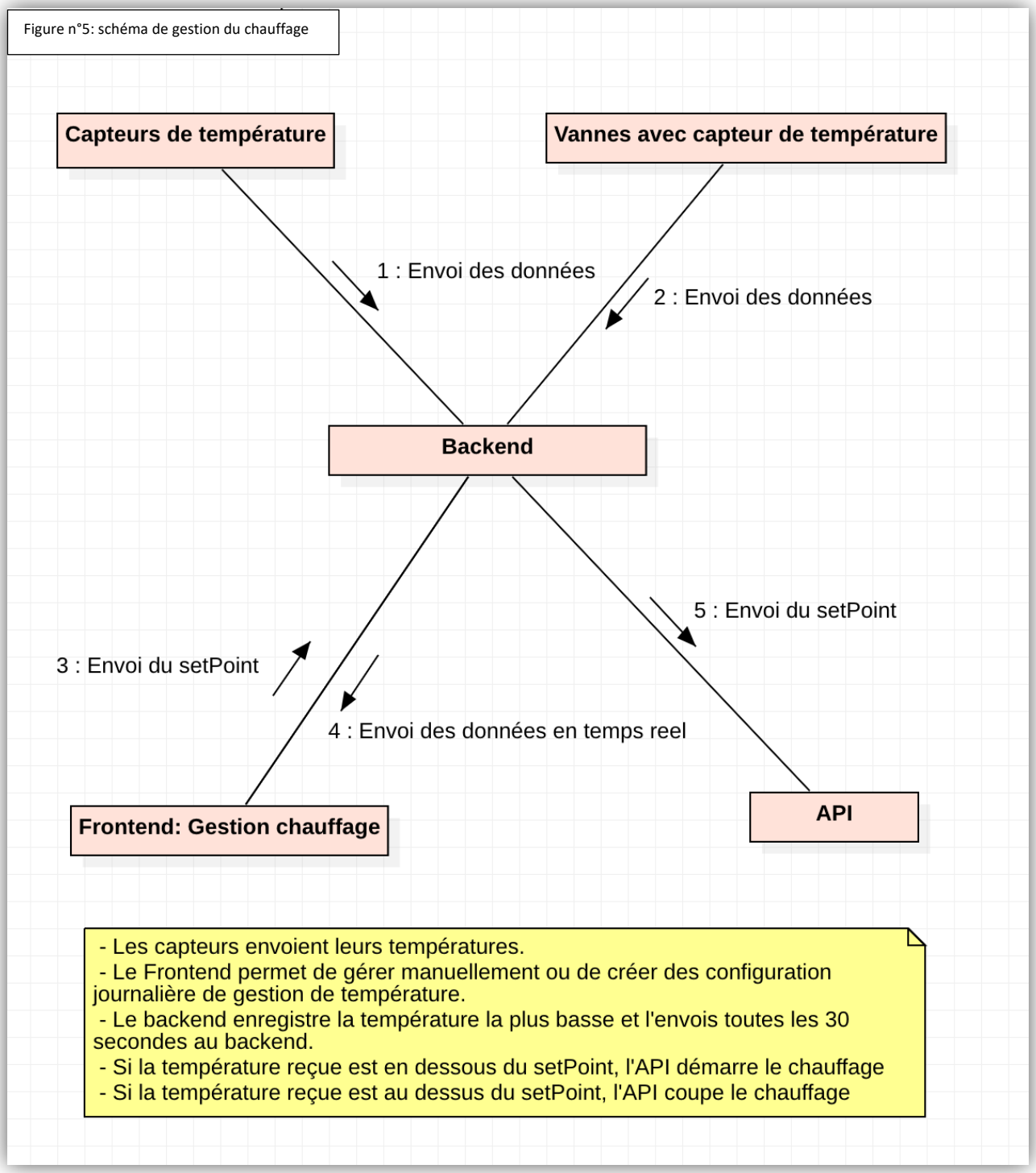


Figure n°5: schéma de gestion du chauffage



4.4 Matériel utilisé :

- Ordinateur de type Mac Book Pro pour le développement
- Raspberry Pi de type PI4 4Go pour la communication avec l'API et l'hébergement de l'application.
- Automate programmable de type Siemens Logo pour la gestion des préactionneurs et alertes
- Microcontrôleurs de type ESP32 et arduino mini pour servir de lecteur du badge et de simulateur de température
- Badges RFID de type mifare classic 1K
- Compteur d'énergie de type Schneider iEM 2055 pour contrôler l'énergie consommée.
- Autres : transformateurs, convertisseurs,…

4.5 MoSCoW

4.5.5 Must Have (Indispensable)

API

- Recevoir et enregistrer les données du compteur d'énergie.
- Recevoir des instructions du backend afin d'actionner ses sorties (allumage du chauffage / ouverture de la porte sécurisée).
- Gestion des pannes IoTs : pertes de contact avec le capteur de température / la porte / le compteur

Backend

- Communication avec l'API pour recevoir les données d'énergie et envoyer des commandes aux préactionneurs.
- Enregistrement des données en temps réel dans la base de données.
- Interface entre la base de données et l'application frontend.
- Réception MQTT d'un capteur de température déporté.

Frontend

- Tableau de bord affichant les mesures de température et les statistiques énergétiques.
- Interface pour envoyer des commandes aux actionneurs (chauffage).

Hardware

- Lecteur de badges permettant d'ouvrir une porte sécurisée.
- Capteur WiFi envoyant en temps réel la température d'une pièce.

4.5.6 Should Have (Important mais pas essentiel au démonstrateur)

Backend

- Communication IoT avec le protocole Zigbee.
- Enregistrement des actionneurs et préactionneurs tiers.

Frontend

- Gestion avancée des actions différées.
- Programmation de scénarios domotiques (ex : activation automatique des lampes sur luminosité faible et extinction automatique à une heure données).
- Supervision énergétique plus détaillée, avec alertes (ex : consommation anormale détectée).
- Authentification et gestion des utilisateurs, avec une interface permettant de gérer les droits d'accès aux commandes domotiques.

4.5.7 Could Have (Si le temps permet de le développer)

API

- Récupération automatique des tarifs énergétiques pour calculer le coût total.
- Historique avancé des mesures et actions
- Graphiques interactifs sur plusieurs semaines/mois.
- Enregistrement de scénarios domotiques (ex : activation du chauffage à des horaires programmés quotidiennement).

4.5.8 Won't Have (Hors du périmètre actuel)

- Interface mobile optimisée avec PWA[21] (Progressive Web App).
- Sécurité avancée avec chiffrement bout en bout pour les communications API.
- Rapport

[21] https://fr.wikipedia.org/wiki/Progressive_web_app

4.6 Pour Finir …

Ce projet vise à évaluer la faisabilité de l'implantation d'un système domotique au sein de l'entreprise en termes de temps de travail et de coût. Bien que celui-ci ne soit pas la réalité, il en est un reflet. Pour mettre en place un système domotique professionnel, il pourrait être nécessaire d'utiliser du matériel plus spécialisé et de respecter les normes belges et européennes si elles existent afin que ce démonstrateur puisse devenir une solution professionnelle viable. Étant donné que le projet est un démonstrateur, certains des éléments de celui-ci ne seront pas viables pour une phase de développement de plus grosse envergure. Certaines parties devront êtes revues comme le fait de ne passer qu'en filaire pour la sécurité de la porte et non par du wifi comme présenté sur ce démonstrateur. Il est préférable de faire transiter les données via câblage pour ne pas donner la possibilité de capter celle-ci mais comme le but est de pouvoir le montrer il était préférable de faire un système simplifié et viable afin de pouvoir le transporter, le monter et le démarrer facilement.

Exemple de bénéfices potentiels de ce projet :

Réduction des coûts énergétiques : Un système domotique optimisé peut réduire considérablement la consommation d'énergie de l'entreprise, entraînant ainsi des économies substantielles sur les factures d'électricité. Diminution de la consommation du chauffage entraine une diminution des coûts en gaz par exemple.

Amélioration du confort et du bien-être des employés : En intégrant des capteurs de température et d'autres dispositifs domotiques, l'entreprise peut créer un environnement de travail optimal, adapté aux besoins individuels de ceux-ci. Cela peut contribuer à une meilleure productivité et à une diminution des interruptions liées à des conditions inconfortables

5 Mise en œuvre

5.1 Le matériel

5.1.1 Platine de démonstration

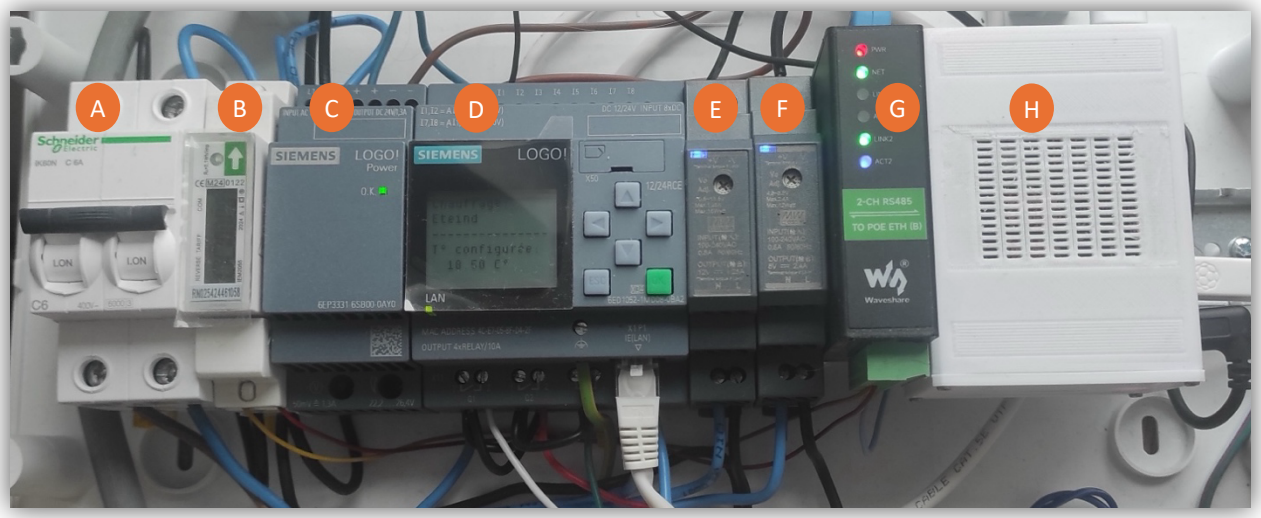


Figure n°6: image de la platine de démonstration

A – Disjoncteur 6A Schneider :

Sécurité de la platine. Normalement, un disjoncteur de 2A est suffisant mais comme la platine sert de démonstrateur, nous avons le compteur d'énergie dessus et donc pour la simulation, il sera possible d'ajouter sur la ligne du matériel qui consomme du courant dépassant les 2A.

B - Compteur d'énergie Schneider Acti9 IEM2055 :

Permet de faire une lecture précise de la quantité d'énergie consommée. Celui-ci se place juste après le différentiel principal. Il a la particularité de pouvoir communiquer soit par impulsion soit via le modbusRTU. Celui-ci lit le monophasé jusqu'à 45A maximum. Il est très complet et très facile d'utilisation. Il est également possible de lire négativement si des panneaux photovoltaïques sont installés.

Ses registres permettent la lecture de :

- La tension
- Le courant
- La puissance active
- La puissance réactive
- La puissance apparente
- La fréquence

- Les énergies :
  - Active directe, inverse, totale
  - Réactive directe, inverse, totale
  - Partielle directe, inverse
  - Par tarif active, réactive pour deux tarifs différents selon le/les fournisseurs

C – Transformateur Siemens LOGO ! 230V / 24V :

Permet d'alimenter l'automate programmable Siemens Logo !

D – Siemens LOGO 12/24RCE V8.4 :

C'est plutôt un module logique qu'un automate avancé tel qu'un Siemens Simatic S7[22]. Celui-ci est composé 8 entrées numériques et de 4 sorties numériques 10A max. Une entrée RJ45 Ethernet. Il peut communiquer en modbusTCP/IP et MQTT. Il est également possible de créer des alertes par mail avec celui-ci. La version utilisée est la version avec un écran. Il est possible de le programmer en Ladder ou en bloc de fonction (FBD). Nous le programmerons en bloc de fonction.

Caractéristiques	**Logo!8**
Complexité	Simple à programmer
Public cible	Débutants / PME / Domotique
Logiciel de programmation	Logo!8 Soft Comfort
Capacités logiques	Limitées mais suffisantes
Utilisation type	Domotique et Automatisation industrielle

Au niveau matériel, pour débuter un projet comme celui-ci, le Logo!8 est parfait et montre sa capacité à être didactique, fiable et il entre tout à fait dans la vision des besoins pour ce qui était demandé de faire.

Seul bémol, le logiciel Soft Comfort qui est daté du point de vue interface. Siemens a plutôt tendance à rester sur ses acquis. Au fur et à mesure des améliorations du Logo!8 , ils ajoutent des fonctionnalités au logiciel sans aucune refonte de ce dernier. Ce qui donne un logiciel lent avec une architecture vintage sur des machines récentes. D'où les gros soucis de compatibilité avec le dernier OS d'Apple.

[22] https://www.siemens.com/fr/fr/produits/automatisation-entrainements/systemes-automatisation/industrial/plc.html

Deux aspects de la politique de Siemens m'ont mis mal à l'aise. Le premier concerne l'achat du logiciel, fourni uniquement sur DVD et sans code d'activation, alors qu'il est pourtant facilement téléchargeable gratuitement sur leur propre site. Le second point est lié à la documentation, qui n'est pas toujours claire : certains exemples sont rédigés dans un français approximatif et manquent de précision, en particulier pour certains blocs fonctionnels. Cela m'a parfois causé des difficultés pour réaliser des fonctions pourtant simples.

Dans l'ensemble, après plusieurs heures de prise en main, le Logo ! en lui-même effectue le travail et il le fait extrêmement bien !

E - Transformateur 230V/12V MeanWell HDR-15-12 rail DIN :

Celui-ci alimente le convertisseur ModbusRTU / TCP/IP.

F – Transformateur 230V/5V Mean Well HDR-15-5 5 V/DC 2.4 A 12 W rail DIN :

Celui-ci alimente le Raspberry Pi.

G – Convertisseur ModbusRTU TCP/IP Waveshare 2-CH RS485 TO POE ETH (B):

L'appareil est composé de deux entrées RJ45 et de deux entrées Modbus RTU, et peut être alimenté soit en PoE, soit via une alimentation externe comprise entre 12 et 48V. Son utilisation s'est révélée particulière, notamment en raison d'un manuel traduit automatiquement du chinois vers le français, rendant certaines explications difficiles à comprendre. Cependant, avec un peu de pratique, l'équipement devient assez simple à utiliser. Néanmoins, pour un usage professionnel, il est préférable d'opter pour un modèle plus sécurisé : bien que des garanties existent, la transparence de Waveshare en matière de sécurité reste limitée.

H – Raspberry Pi 4 4go :

Ce micro-ordinateur sert pour le serveur MQTT et il hébergera l'application. Le fait qu'il puisse avoir une carte mémoire et que l'on peut aussi y placer des disques durs pour enregistrer les données lui donne toutes les facultés à pouvoir être facilement déployable et à maintenir dans des coûts hors concurrence avec ses 75€.

5.1.2 ESP32 pour émuler des températures

Les ESP sont des microcontrôleurs qui embarquent une puce intégrant le wifi. Ce qui permet de pouvoir l'utiliser facilement sur le système sans avoir une multitude de câblage. Ceux utilisés sont des AZDelivery ESP32 CP2102 qui ne sont que des rebrandings de la marque Espressif. Celui utilisé ici n'a qu'un simple code qui fait une boucle de température et qui émule la perte de communication. Il enverra des températures au backend qui les retransmettra au Logo!8 .

5.1.3 Enregistreur de badge

Pour simuler l'ouverture d'une porte magnétique avec un lecteur de badges. Il nous faut un enregistreur qui a pour but d'écrire sur les badges Mifare RFID lors de l'ajout d'un nouvel utilisateur. Il se compose d'un ESP32, d'une led RGB et d'un module RFID PN-532 V3.

Lors de la construction de celui-ci, j'ai remarqué que la LED que ça soit en 5v ou en 3v3 ne fonctionnait pas bien alors que tout se passait correctement. Après quelques recherches, j'ai vu que cela venait de l'ESP. Afin de ne pas bloquer le travail, puisque l'enregistrement se fait, j'ai pris la décision de continuer et d'y revenir si j'ai plus de temps.

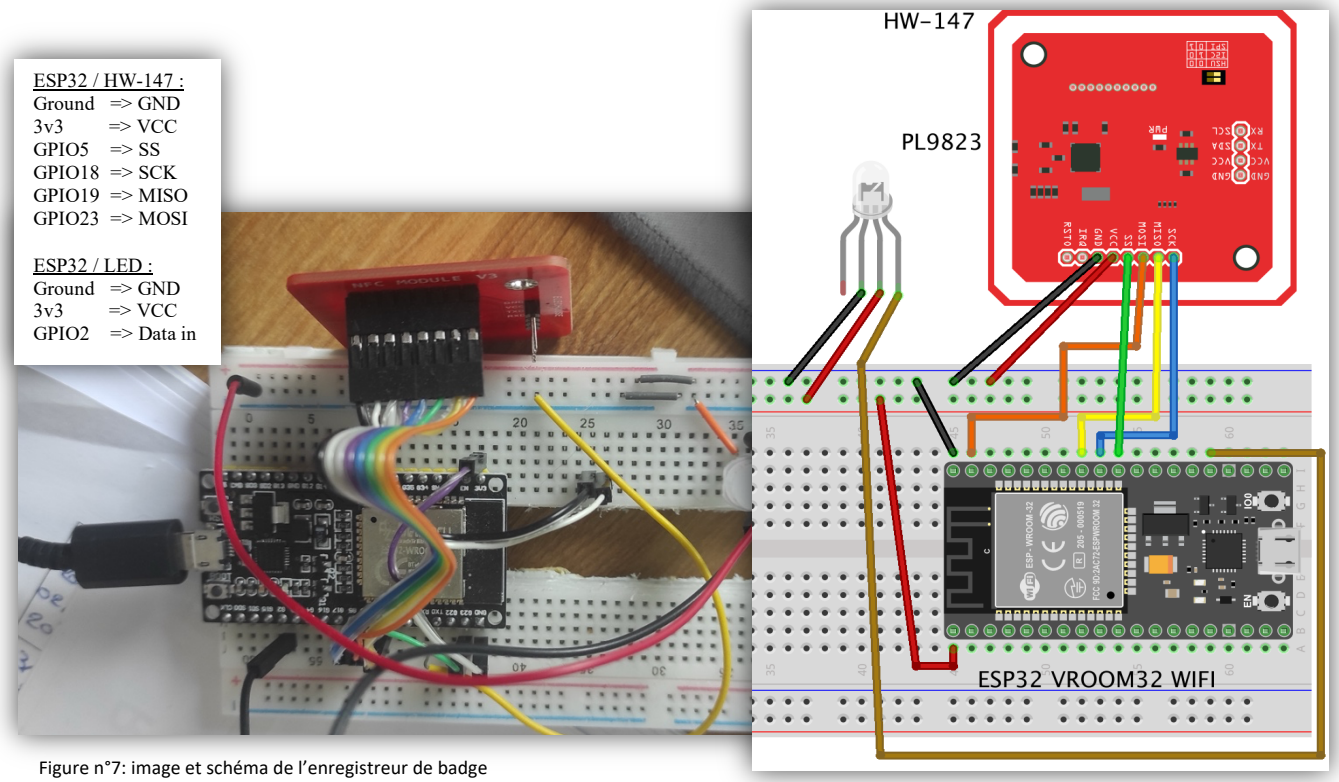


Figure n°7: image et schéma de l'enregistreur de badge

5.1.4 Lecteur de badges (accès porte)

Étant dans une configuration de simulation, acheter un lecteur professionnel aurait été prohibitif. J'ai pris la décision de rester sur une configuration presque identique au lecteur. Il se composera d'un ESP32 avec un module RFID PN-532 V3 et d'un arduino nano avec une led RGB et un écran Nokia 5110.

Le problème de la LED est vite revenu sur ce système, mais cette fois c'était plus gênant car il touchait directement l'utilisateur. En cherchant, j'ai constaté que sur l'ESP32 les GPIO ne sont pas stables avec du 5V. C'est sans doute pour ça que la LED se comporte bizarrement, et je soupçonne que ce soit aussi lié aux capteurs de température.

J'ai donc fini par couper le système en deux : une partie gérée par l'ESP et l'autre par un Arduino Uno. Cette solution a permis de régler le problème, de simplifier le code de l'ESP et de le rendre plus facile à modifier. Ça a pris un peu de temps, mais le résultat est un système stable et facilement ajustable.

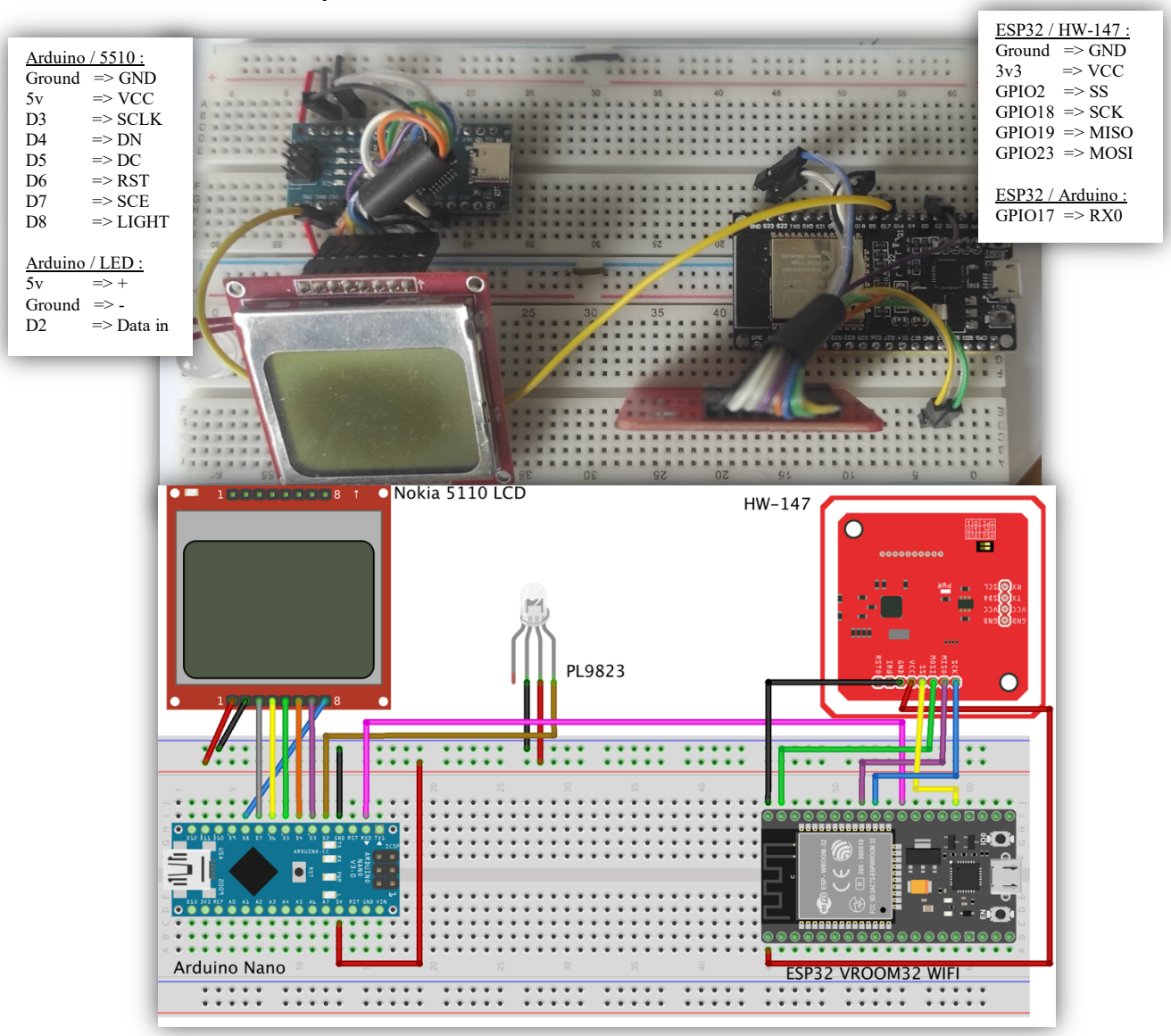


Figure n°8: image et schéma du lecteur de badges

5.1.5 Badges Mifare Classic 1K

Il aurait été plus sécurisé d'utiliser des badges MIFARE Desfire qui ont la possibilité de gérer le cryptage AES-256 en interne et d'être sécurisés contre le clonage. Vu le coût de celle-ci et étant donné que les badges MIFARE Classic 1K sont courants. Nous partirons de ces badges qui ont un ID, une clé A et B et des bits d'accès.

Structure Mémoire d'un badge Mifare classic 1k

Secteur	Blocs	Taille (octets)	Remarques
0	0, 1, 2, 3	16 × 4 = 64 octets	Contient le **NUID** (dans bloc 0) et les données de fabrication (non modifiables)
1–15	4 blocs chacun	64 octets / secteur	Blocs 0–2 = données, bloc 3 = secteur trailer (clés + bits d'accès)
	Exemple secteur 1	Blocs 4, 5, 6, **7**	Bloc 7 = secteur trailer

Figure n°9 : https://kb.supremainc.com/knowledge/doku.php?id=en:1xfaq_how_to_configure_mifare_card_memory_layout

5.2 Schéma de communication

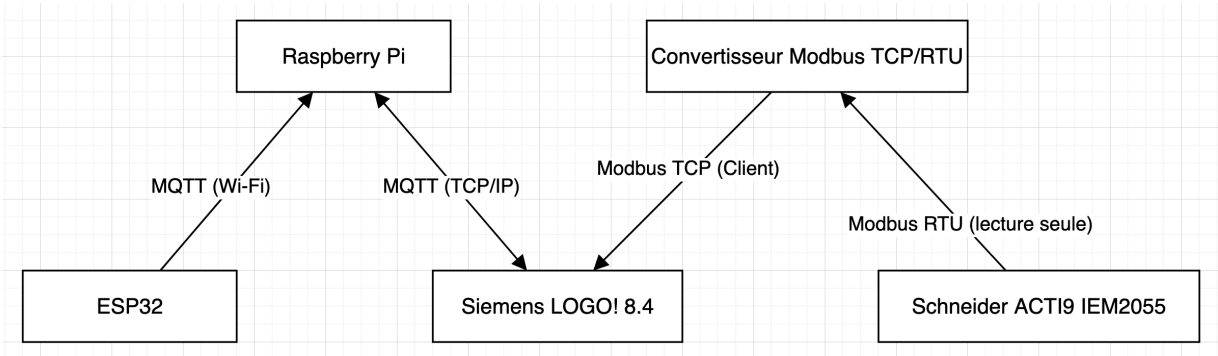


Figure n°10 : Schéma de communication matériel

5.3 *Installation d'un linux sur le raspberry Pi et mise en place du serveur Mosquitto*

Pour commencer le projet, j'ai installé le système d'exploitation RaspberryPi OS[23] 64bits sur le Raspberry pi 4 et monté un serveur Mosquitto port 1883 avec seulement la sécurité par mot de passe, pour me familiariser avec le protocole et pouvoir communiquer entre les éléments.

Mosquitto est le serveur principal permettant la communication sur le protocole MQTT. Il servira de passerelle entre les éléments du projet en recevant et transmettant les messages. La particularité de Mosquitto est qu'il est très simple d'utilisation.

Avec quelques lignes dans le terminal de linux, on installe rapidement le serveur mosquitto.

- sudo apt update & sudo upgrade -y (mise à jour du système d'exploitation avant l'installation)
- sudo apt install -y mosquitto mosquitto-clients (pour l'installation)
- sudo systemctl enable mosquitto (pour activer le service du serveur)
- sudo systemctl start mosquito (pour démarrer le serveur)

Ensuite, on peut configurer le serveur.

- sudo nano /etc/mosquitto/mosquitto.conf

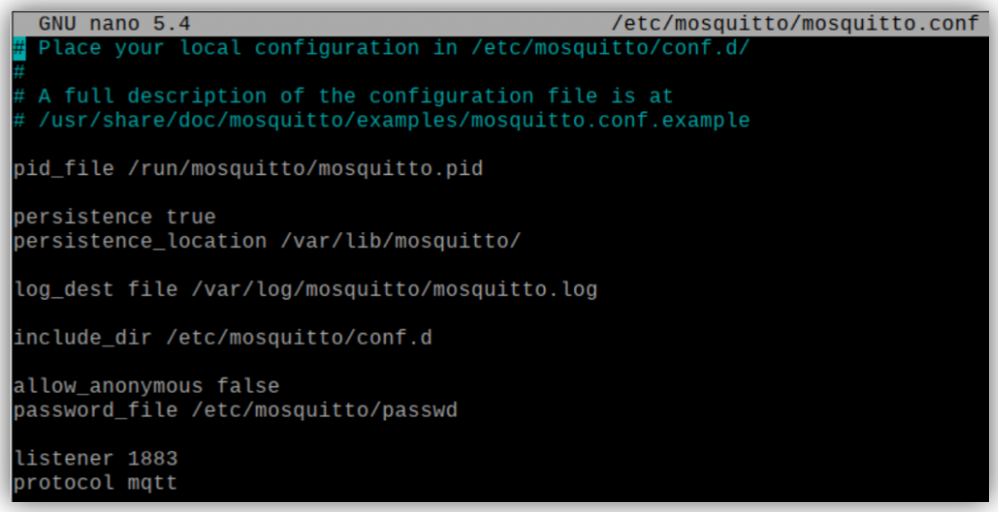


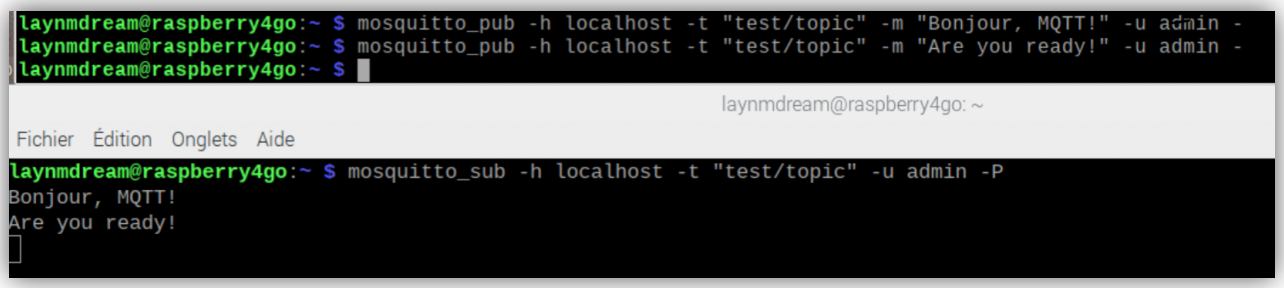
Figure n°11 : fichier de configuration serveur mqtt

Le principal à ce stade est de ne pas rendre le serveur transparent avec la clé allow_anonymous passée en false. Il ne reste plus qu'à créer un utilisateur avec lequel tous les éléments pourront souscrire au serveur (pc, arduino, LOGO…)

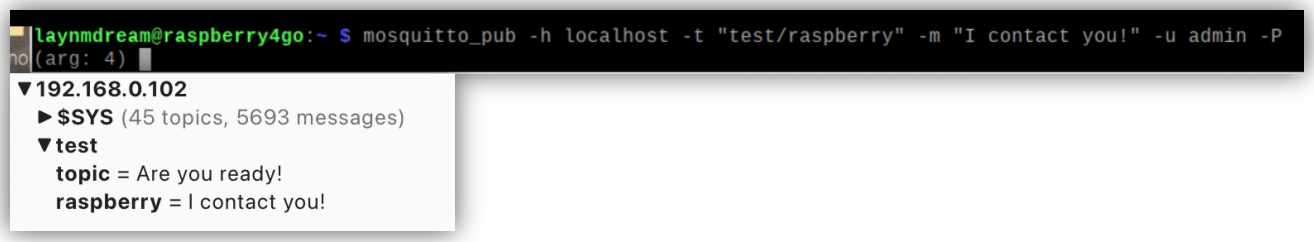
[23] https://www.raspberrypi.com/software/operating-systems/

Il ne reste qu'à tester en installant le logiciel client MQTTexplorer. On se connecte au serveur via celui-ci, ce qui permet de tester si un abonnement est bien publié à partir du Raspberry Pi en recevant le message transmis et inversement.

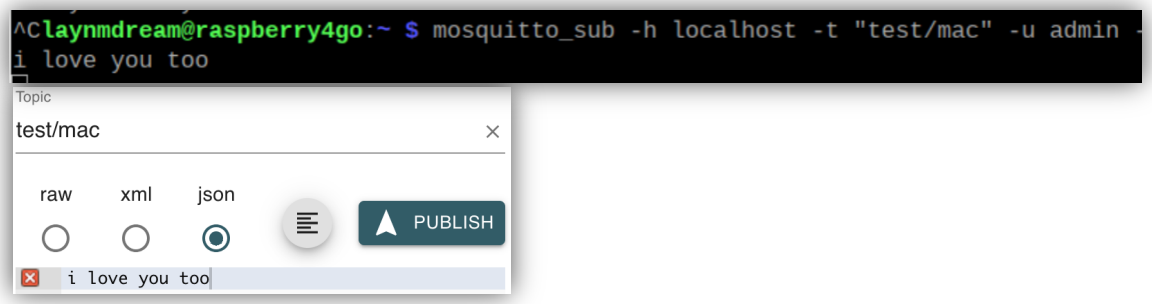
Topic à partir du Raspberry Pi



Topic avec MQTT explorer subscrition sur le mac



Topic avec MQTT explorer d'une publication à partir du mac



5.4 Création du backend node.js

L'application proposera un tableau de bord de supervision à la connexion d'un utilisateur, il lui faut donc un cœur central qui communique avec tous les éléments du système et qui enregistre dans une base de données les états et les valeurs importantes de ces communications. J'ai donc mis en place un backend en JavaScript / Node.js. Le package mosquitto.js y est installé afin de pouvoir communiquer avec le serveur dédié. D'autres packages utiles au fonctionnement de celui-ci sont aussi installés. L'installation de base n'est pas très compliquée mais nécessite un certain temps pour pouvoir le faire. (Voir annexe)

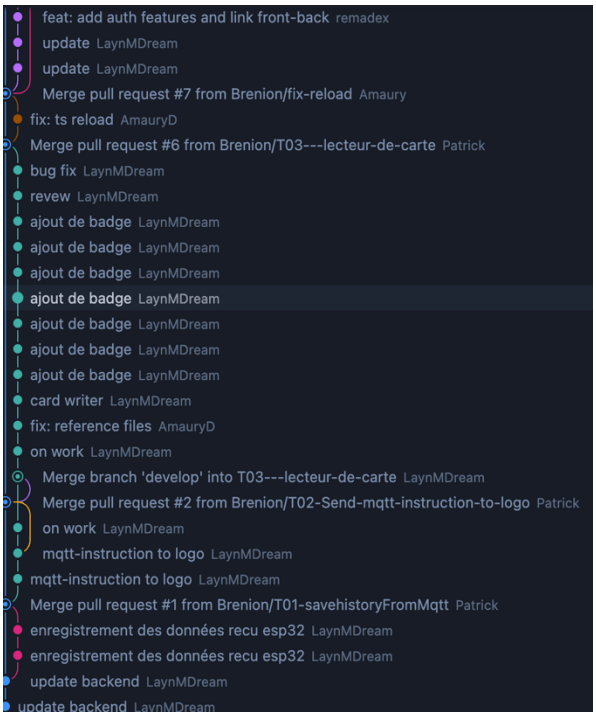


Figure n°12 : vue des branches Git à partir de VisualStudio Code

Nous pouvons travailler au fur et à mesure sur l'ordinateur, mais il faut sauvegarder tout le projet à chaque fois, ce qui devient vite contraignant et risque de poser des problèmes si on oublie. Il serait dommage, si le projet ne fonctionne plus, de devoir tout recommencer. Pour éviter cela, j'ai créé un dépôt Git. Celui-ci est un dossier virtuel qui conserve l'historique du projet. Il permet aussi de travailler par tâche en créant des branches. En développement, on utilise généralement deux branches principales : main, la branche de production, qui récupère les mises à jour de develop ; et develop, qui reçoit les mises à jour des branches de tâches une fois qu'elles sont terminées.

J'ai un compte GitHub qui me permet d'avoir un dépôt distant garantissant plus de sécurité.

Concrètement, je crée une tâche, je travaille dessus, je la pousse régulièrement (push) sur mon dépôt, puis, une fois terminée, elle est vérifiée par un collègue. Il peut faire des commentaires (peer review). Quand la branche est validée, je mets à jour develop avec ma tâche (merge). Ensuite, sur ma machine, je récupère la nouvelle version de develop (pull) et je recommence.

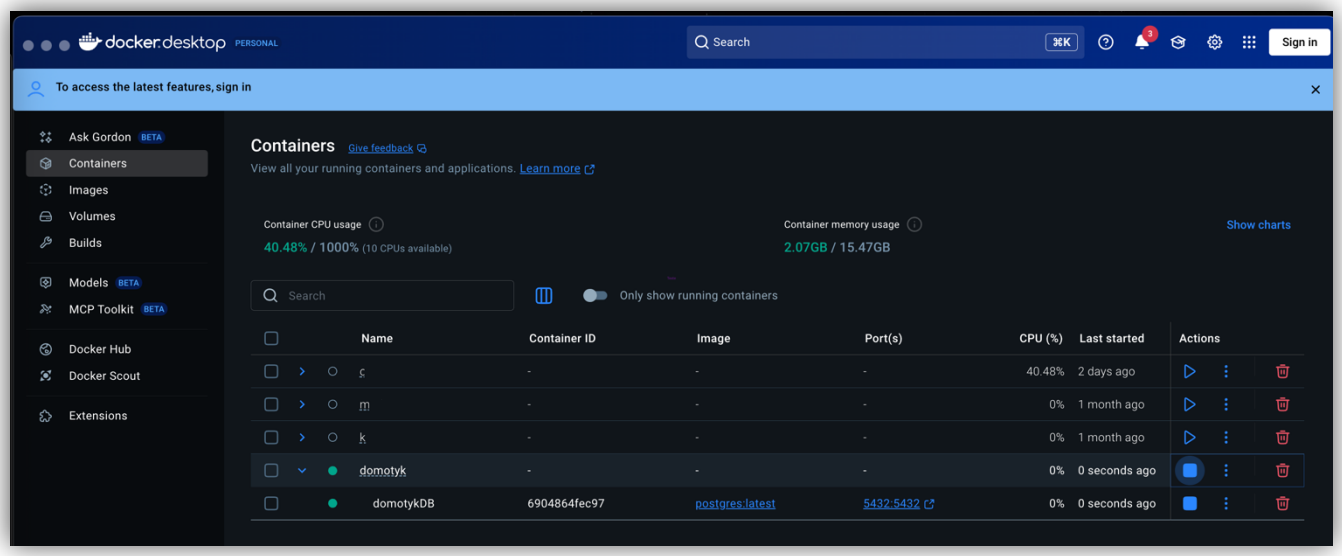


Figure n°13 : docker desktop : permet de visualiser et gérer les containers, images et volumes pendant le développement

Pour le backend, la base de données est dans un conteneur Docker. Ce logiciel est un système de conteneurs. Il permet l'isolation d'environnements d'exécution légers et rapides. Déployable facilement sur n'importe quel machine en sécurisant l'espace utilisateur. Il fonctionne par virtualisation de systèmes d'exploitations sans interface graphique. Pour

l'instant, seule la base de données tourne dans Docker. On y reviendra lors de la mise en production.

5.5 Simulateur de température

Un code en C++ a été créé pour simuler une boucle de température allant de 15 °C à 23 °C, avec en plus un passage de perte du capteur. L'objectif était d'avoir des données cohérentes pour pouvoir tester les interactions avec le backend et l'automate. Les données servent à ce dernier pour mettre en route le chauffage. Chaque réception des données par le backend est enregistrée dans la base de données. Un seeder a été écrit afin de créer le capteur dans la base de données pour qu'il soit reconnu au démarrage du serveur nodeJs.

Après cela, j'ai mis en place la communication entre le backend et le Logo!8 en MQTT. J'ai écrit un service qui permet d'envoyer une instruction au Logo!8 chaque fois qu'un seuil de température haute ou basse est atteinte. Un ruban led RGBW comme témoin d'activation les sorties du Logo!8. Les états s'affichent également sur l'écran de celui-ci.

ID	Name	Range	Type	Address	Length	At Frequency	Period(d:h:m:s)	On Change	Writ
1	HEATINGSTATUS	M	Bit	M1	1	☐	10s	☑	☑
2	SENSORSTATUS	M	Bit	M2	1	☐	5s	☑	☑

Figure n°14 : variables inscrites dans le LOGO

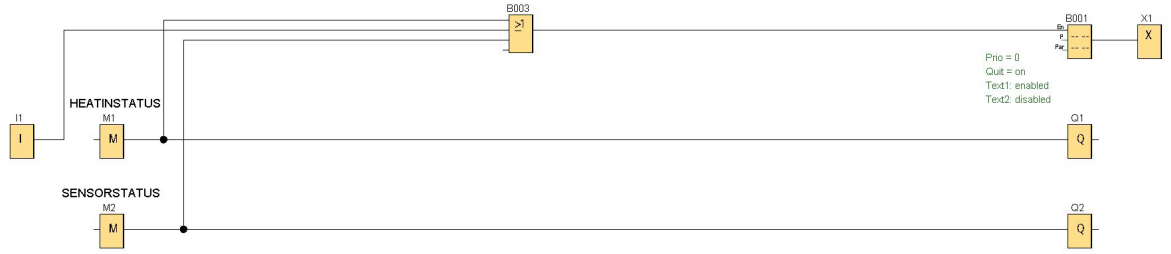


Figure n°15 : exemple de codage du LOGO

5.6 Mise en place du compteur d'énergie

5.6.1 Prise en main du convertisseur

La première étape consiste à configurer le convertisseur Waveshare qui sert de passerelle pour établir la communication Modbus entre le compteur d'énergie Schneider IEM2055 et le siemens Logo!8. Après un certain temps de prise en main, on découvre que le premier canal Modbus visible dans le logiciel est en réalité le second sur le boîtier, et vice-versa. Au début, je ne savais pas vraiment lequel je manipulais. Ensuite, avec un logiciel MBpoll, j'ai tenté de lire le registre mémoire 51 du compteur pour recevoir le modèle comme réponse. Après plusieurs tentatives infructueuses avec le logiciel Modbus poll, j'ai tenté une autre approche.

```
21:14:24.015 -> Erreur Modbus : E2
21:14:28.188 -> Erreur Modbus : E2
21:14:31.171 -> Erreur Modbus : E2
21:14:34.167 -> Erreur Modbus : E2
21:14:37.178 -> Erreur Modbus : E2
21:14:40.194 -> Erreur Modbus : E2
21:14:43.186 -> Erreur Modbus : E2
21:14:46.192 -> Erreur Modbus : E2
21:14:49.240 -> Erreur Modbus : E2
21:14:52.211 -> Erreur Modbus : E2
21:14:55.215 -> Erreur Modbus : E2
21:14:58.239 -> Erreur Modbus : E2
21:15:01.250 -> Erreur Modbus : E2
21:15:04.234 -> Erreur Modbus : E2
21:15:07.240 -> Erreur Modbus : E2
21:15:10.280 -> Erreur Modbus : E2
21:15:13.279 -> Erreur Modbus : E2
21:15:16.277 -> Erreur Modbus : E2
21:15:19.297 -> Erreur Modbus : E2
```

Figure n°16 : Message reçu sur

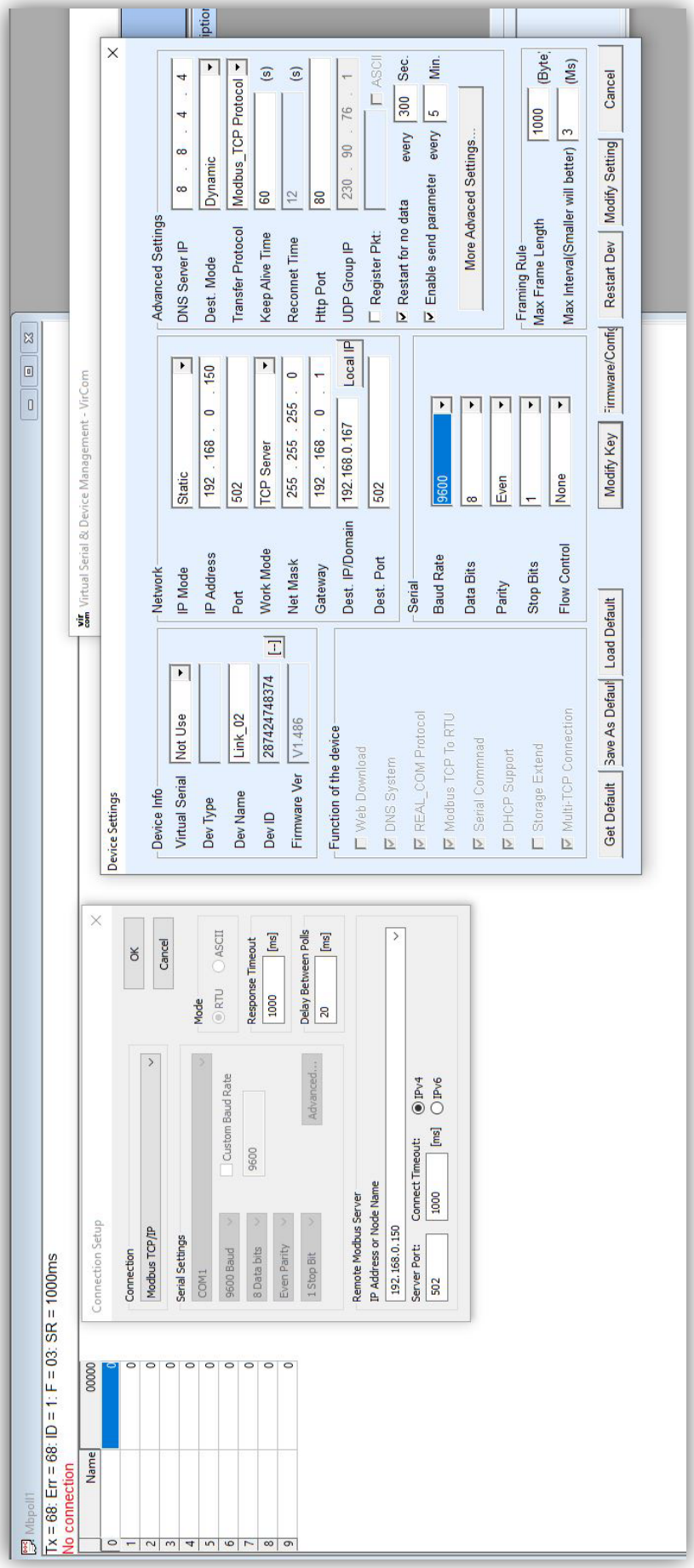


Figure n°17 : configuration MBpoll et VirCom

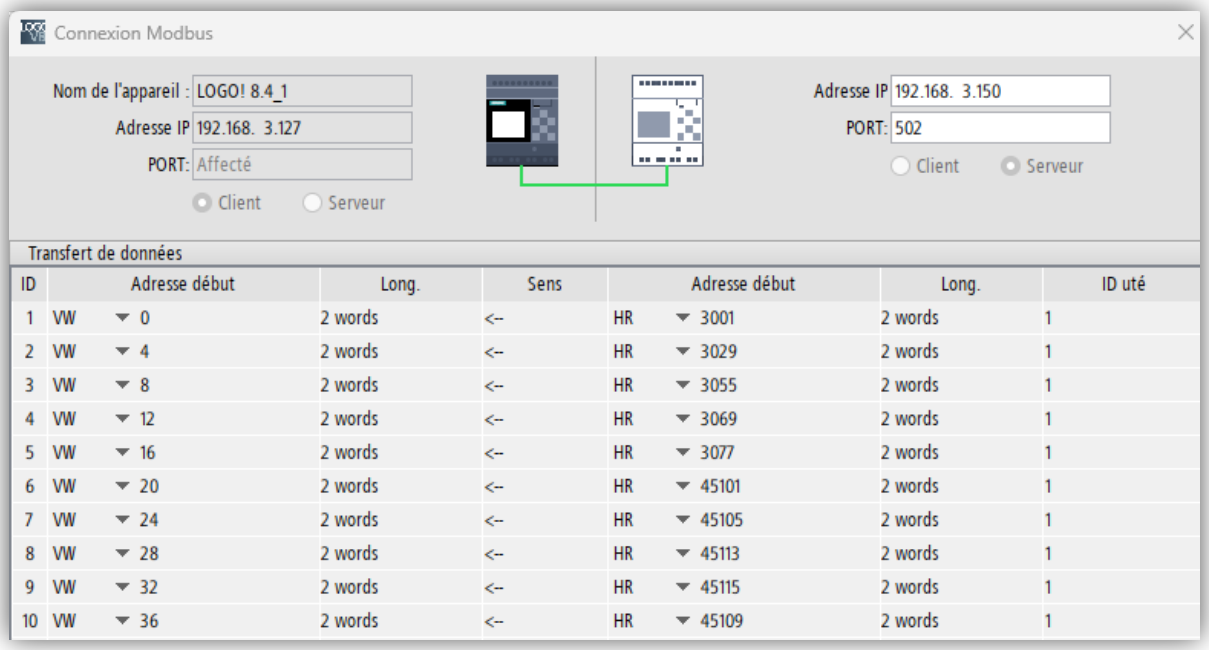
5.6.2 Résolution des problèmes de communication

Pour voir si ce n'était pas à cause du pc. J'ai testé de communiquer directement avec un Arduino uno. J'avais dans mon matériel un shield DMX (Digital Multiplex) dédié au contrôle des lumières en spectacle qui permet de communiquer en modbusRTU. A la mise en place, les tests sont restés aussi infructueux.

J'ai acheté un convertisseur USB/RTU et recommencé les tests qui se sont passés correctement avec les nouvelles configurations trouvées. J'ai réitéré avec l'Arduino et la communication a également réussi. Cependant, avec le Logo!8 il n'y en avait toujours pas. Après une lecture plus approfondie, il apparait que Le modbus RTU et TCP ont une différence d'adressage de 1 lors de la communication. Par exemple, l'adresse pour lire le courant sur le compteur en modbus RTU, l'adresse est 3001 et pour le TCP/IP l'adresse est 3000. C'est en écrivant l'adresse en hexadécimal et en modifiant la configuration RTU ou TCP/IP que j'ai compris et dès le moment où l'on comprend ça, on ne se trompe plus dans la communication.

5.6.3 Mises en place avec le LOGO

Dès que cela est fait, j'ai pu commencer à lire compteur IEM2055 avec le Logo!8. Étant sur un banc d'essai avec peu de puissance et le compteur n'étant pas directement relié au réseau, il fonctionne au « ralenti ». Pour comprendre le principe, lorsque de l'énergie passe par le compteur celui-ci envoie des impulsions. Si peu d'énergie passe les impulsions sont trop faible et la lecture inexistante. J'ai placé une source de 500w afin de pouvoir lancer les impulsions pour avoir une vraie lecture. Ce qui a permis de faire de la programmation en FBS dans le Logo!8 et à stabiliser la communication entre chaque élément.



ID	Adresse début	Long.	Sens	Adresse début	Long.	ID uté
1	VW 0	2 words	<--	HR 3001	2 words	1
2	VW 4	2 words	<--	HR 3029	2 words	1
3	VW 8	2 words	<--	HR 3055	2 words	1
4	VW 12	2 words	<--	HR 3069	2 words	1
5	VW 16	2 words	<--	HR 3077	2 words	1
6	VW 20	2 words	<--	HR 45101	2 words	1
7	VW 24	2 words	<--	HR 45105	2 words	1
8	VW 28	2 words	<--	HR 45113	2 words	1
9	VW 32	2 words	<--	HR 45115	2 words	1
10	VW 36	2 words	<--	HR 45109	2 words	1

Figure n°18 : adressage et ajout des variable Modbus dans le Logo!8

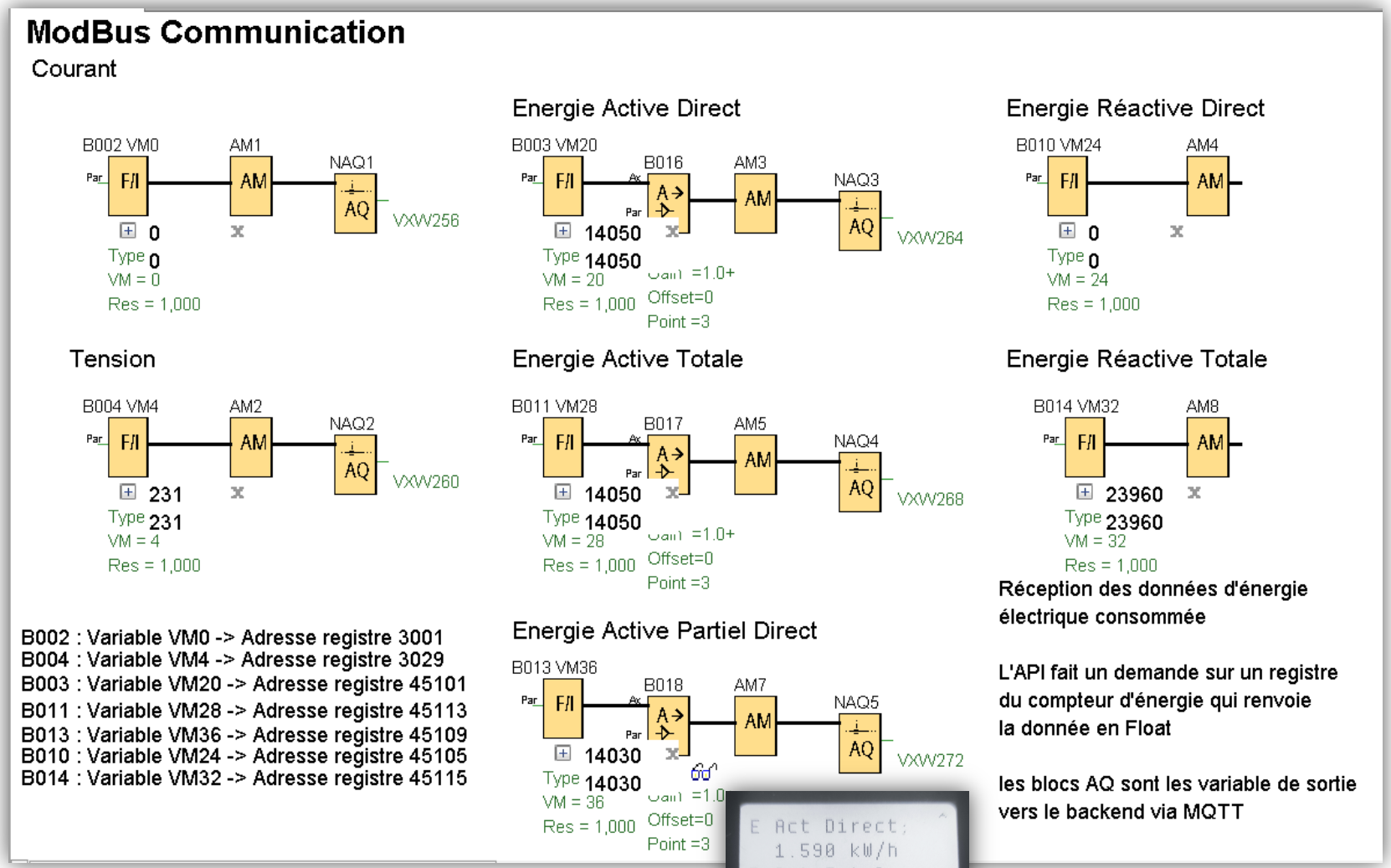


Figure n°19 : visualisation de la lecture modbus avec le Logo !8 avec affichage sur l'écran de ce dernier via le logiciel Logo SoftComfort

En parallèle, le backend est refait afin que celui-ci envoie la température minimale au Logo!8. Il faut évidemment penser au futur du projet qui pourra accepter plusieurs capteurs envoyant leurs données. Il ne faut pas que le chauffage s'allume et s'éteigne en continu, c'est pour cela que, lorsqu'il y a un changement, le Logo!8 reçoit un *setPoint* (température minimale pour la gestion du chauffage) et la température la plus basse parmi tous les capteurs. En programmant le Logo!8, l'intérêt s'est porté sur un bloc de fonction particulier, minuterie, qui permet de créer des settings avec la possibilité d'avoir 3 paramètres journaliers avec un *setPoint* différent comme avec un thermostat traditionnel. Cette partie sera à améliorer dans le futur.

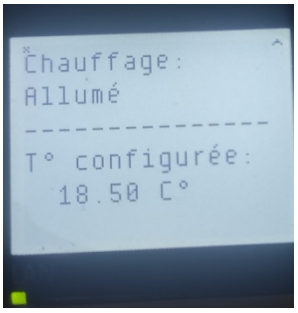


Figure n°20 : visuel sur l'écran du Logo!8

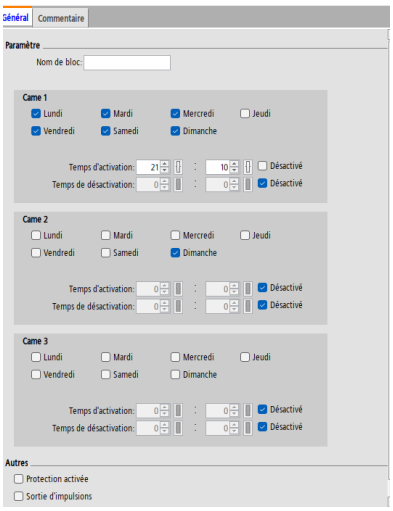


Figure n°21 : Configuration du bloc minuterie

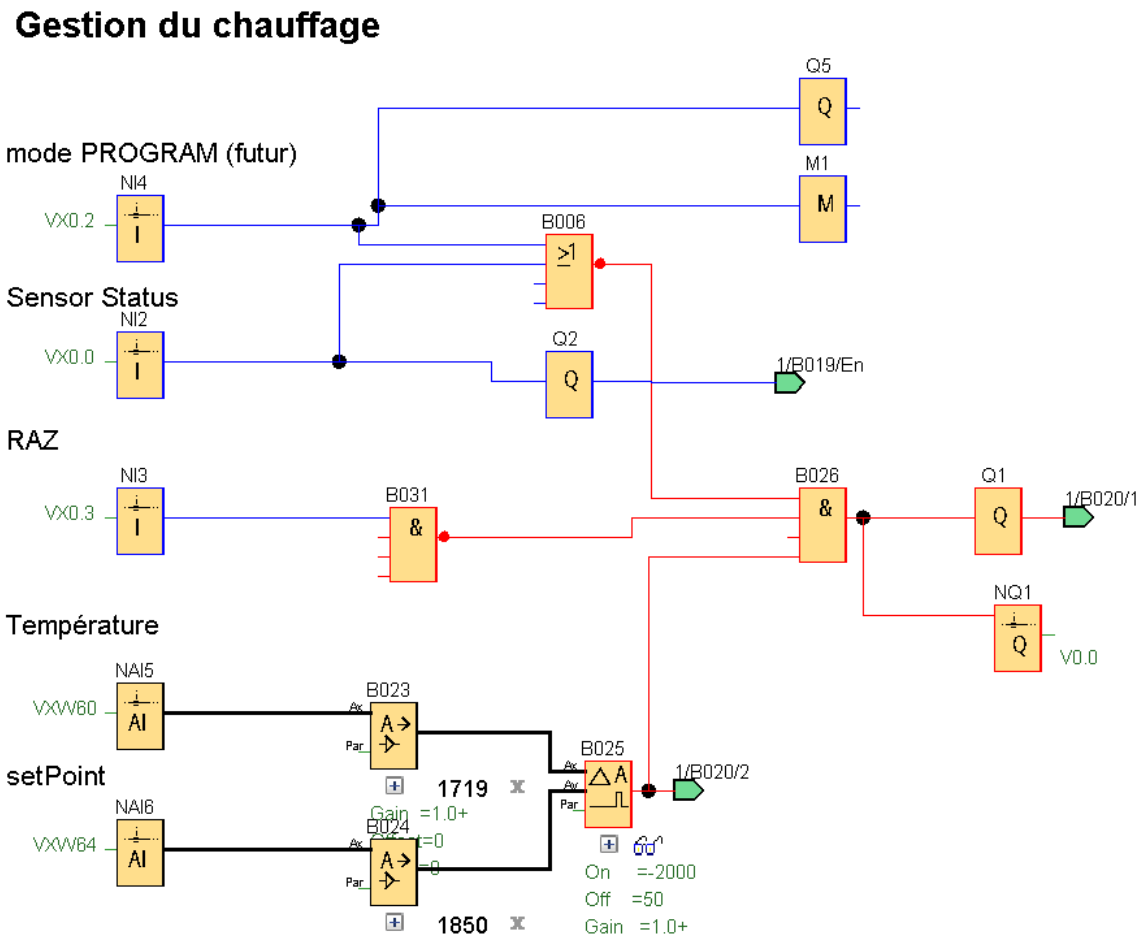


Figure n°22 : vue du chauffage lorsque celui-ci est actif dans le logiciel Logo SoftComfort

5.7 L'enregistreur et le lecteur de badges Mifare.

5.7.1 Le backend

Pour enregistrer un nouveau badge à la demande d'un administrateur, le backend devra faire les vérifications nécessaires. Par exemple : si l'utilisateur a déjà un badge, il ne sera pas autorisé à en avoir un second. Cependant, si l'utilisateur n'en possède pas et qu'il est inscrit dans l'entreprise alors un badge s'enregistrera avec les rôles associés à celui-ci.

Dans le Sens de la lecture, il va vérifier que la badge est bien un badge enregistré et sécurisé. Il communiquera directement avec l'API afin de lui donner la validation d'ouverture de la porte. S'il venait à avoir une déconnexion du lecteur, une alerte sera envoyée à l'API. Quand il y aura plusieurs lecteurs, ceux-ci pourront être associés à des rôles.

J'ai pris le temps d'adapter l'entité user en lui ajoutant un rôle. Deux nouvelles entités ont été créés , badges et accessLogs avec de fausses données pour simuler ces premiers. Afin de vérifier le fonctionnement du code, des tests unitaires ont été créés permettant de simuler la création d'un badge.

5.7.2 L'enregistreur de badge

Étant donné que le système est scalable, je pars du principe de plusieurs portes et donc plusieurs types d'utilisateurs avec des accès plus ou moins limité à certains secteurs d'un bâtiment. Celui-ci sera fait comme le lecteur mais sans écran. L'idée est d'avoir un code AES-128 crypté sur le badge et aussi de crypter les clés A et B afin de sécuriser un maximum et faire en sorte que le secteur puisse être écrit qu'avec la clé B.

La difficulté était de savoir si je construisais un enregistreur avec une communication indépendante ou lié à un pc. Si on opte pour la solution liée au pc, il fallait créer un petit logiciel à installer sur nos OS, ce qui aurait donné une énorme perte de temps. Afin de garantir le processus de démonstration, il est préférable d'utiliser un lecteur indépendant qui communique directement avec le backend par réseau sans passer l'intermédiaire de la machine sur lequel il est branché. Ça permet de l'utiliser quand nous le voulons dès qu'il est alimenté.

5.7.3 Processus d'enregistrement de badge

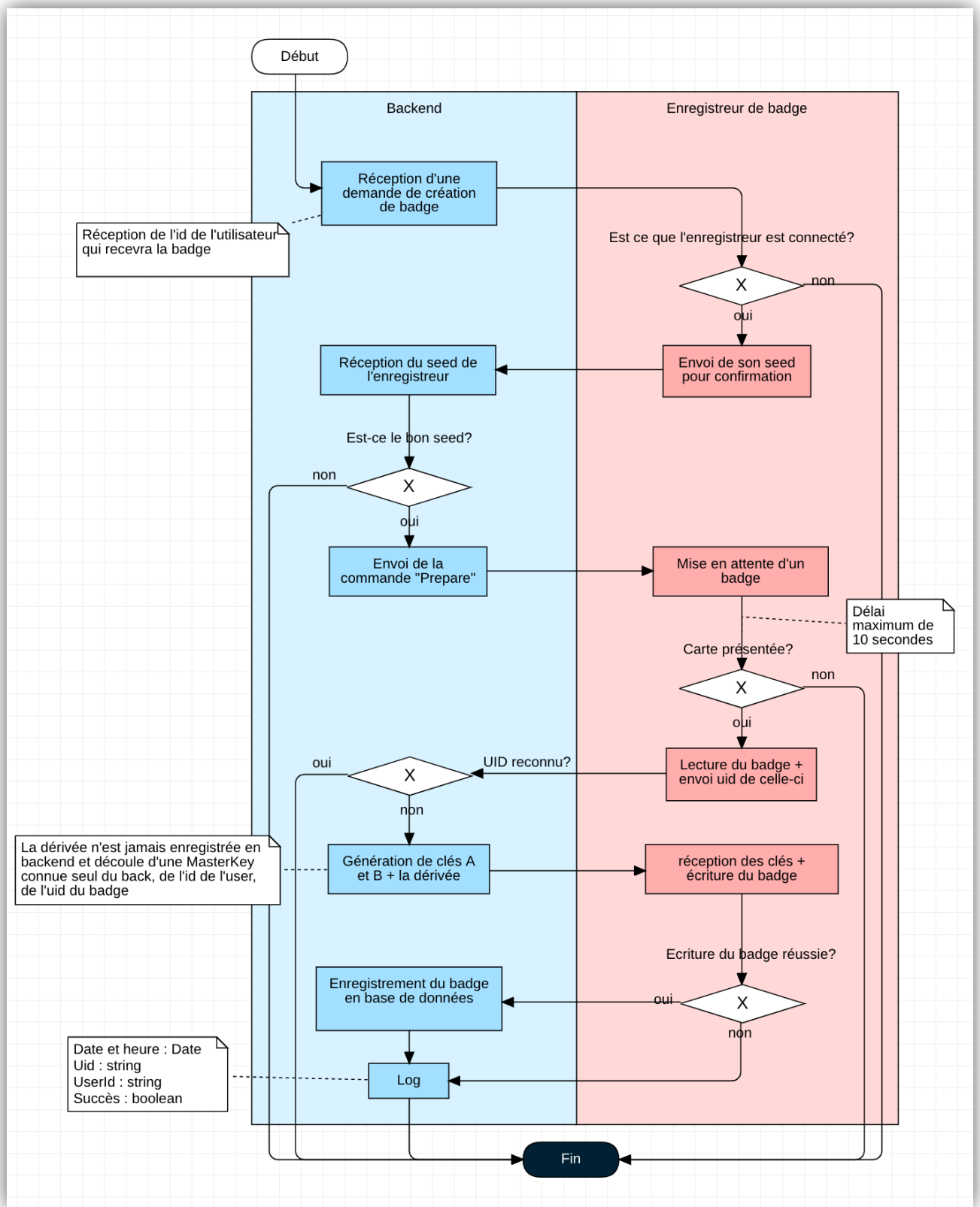


Figure n°23 : schéma de processus d'enregistrement d'un badge

5.7.4 Le cryptage d'un badge

Les badges Mifare Classic 1K sont très faciles à cloner. Le but est de modifier le badge au maximum et de lui fournir une clé cryptée la plus forte possible. Le mieux est un chiffrement AES-256 mais malheureusement, les badges n'ont pas cette capacité donc nous nous portons sur du chiffrement AES-128.

Le chiffrement AES (Advanced Encryption Standard) est un algorithme de chiffrement symétrique par blocs standardisés, ce qui signifie qu'il utilise la même clé pour chiffrer et déchiffrer les données. Tout d'abord, on va dériver les clés A et B et les stocker dans le backend, ensuite dériver une clé AES-128 basé sur le NUID du badge et l'id de l'utilisateur. Seule celui-ci contient la clé.

Avec le frontend, il faudra faire une demande pour créer le badge et donc créer une route badge/add (exemple : http://localhost:8081/badge/add) qui permet d'enregistrer un nouveau badge. Pour ce faire, il faut modifier le service mqtt permettant de publier ou de souscrire à l'enregistreur de badge.

5.7.5 Le Lecteur de badges

Son rôle est de fournir les informations au backend. L'écran et la led seront sur un arduino et le module de lecture de badge sur l'ESP32. Pour des questions de sécurité. Le lecteur doit se trouver du côté intérieur de la porte, et reste du côté extérieur. Ça permet de ne pas pouvoir le pirater directement. Le module RFID est protégé contre les déconnexions. La communication entre l'ESP et l'Arduino est unidirectionnelle afin de rendre impossible de communiquer à partir de l'Arduino vers l'ESP pour des raisons de sécurité en excluant l'injection d'instructions par ce canal.

Figure n°24 : états possibles lors de la lecture d'une badge

33…	2025-0…	2025-06…	f5d28…	BIN…	granted	216399…
85…	2025-0…	2025-06…	NULL	BIN…	denied	216399…

Figure n°25 : enregistrement dans la base de donnée

5.7.6 User story

Rôle / Acteur	User Story
Administrateur système	1. En tant qu'administrateur système, je veux créer, activer, mettre en liste noire ou supprimer des badges, afin de gérer les accès. 2. En tant qu'administrateur système, je veux recevoir une notification quand un badge n'a pas été utilisé depuis 3 mois, afin d'investiguer. 3. En tant qu'administrateur système, je veux consulter les journaux locaux, afin d'assurer la traçabilité.
Administrateur réseau	1. En tant qu'administrateur réseau, je veux que mon badge ouvre la salle serveur, afin de réaliser la maintenance IT. 2. En tant qu'administrateur réseau, je veux être averti si le lecteur ou l'antenne devient indisponible, afin d'intervenir rapidement.
Employé	En tant qu'employé, je veux que mon badge soit refusé à la salle serveur mais enregistré, afin de respecter la politique de sécurité.
Badge étranger	En tant que porteur d'un badge non enregistré, je veux que ma tentative soit refusée et tracée, afin que le système reste sécurisé.
Personne malveillante	En tant qu'attaquant, lorsque j'essaie de forcer l'ouverture ou de présenter un badge falsifié, le système doit rester verrouillé, déclencher une alarme et journaliser l'incident, afin de décourager toute intrusion.

5.7.7 Processus de lecture de badge

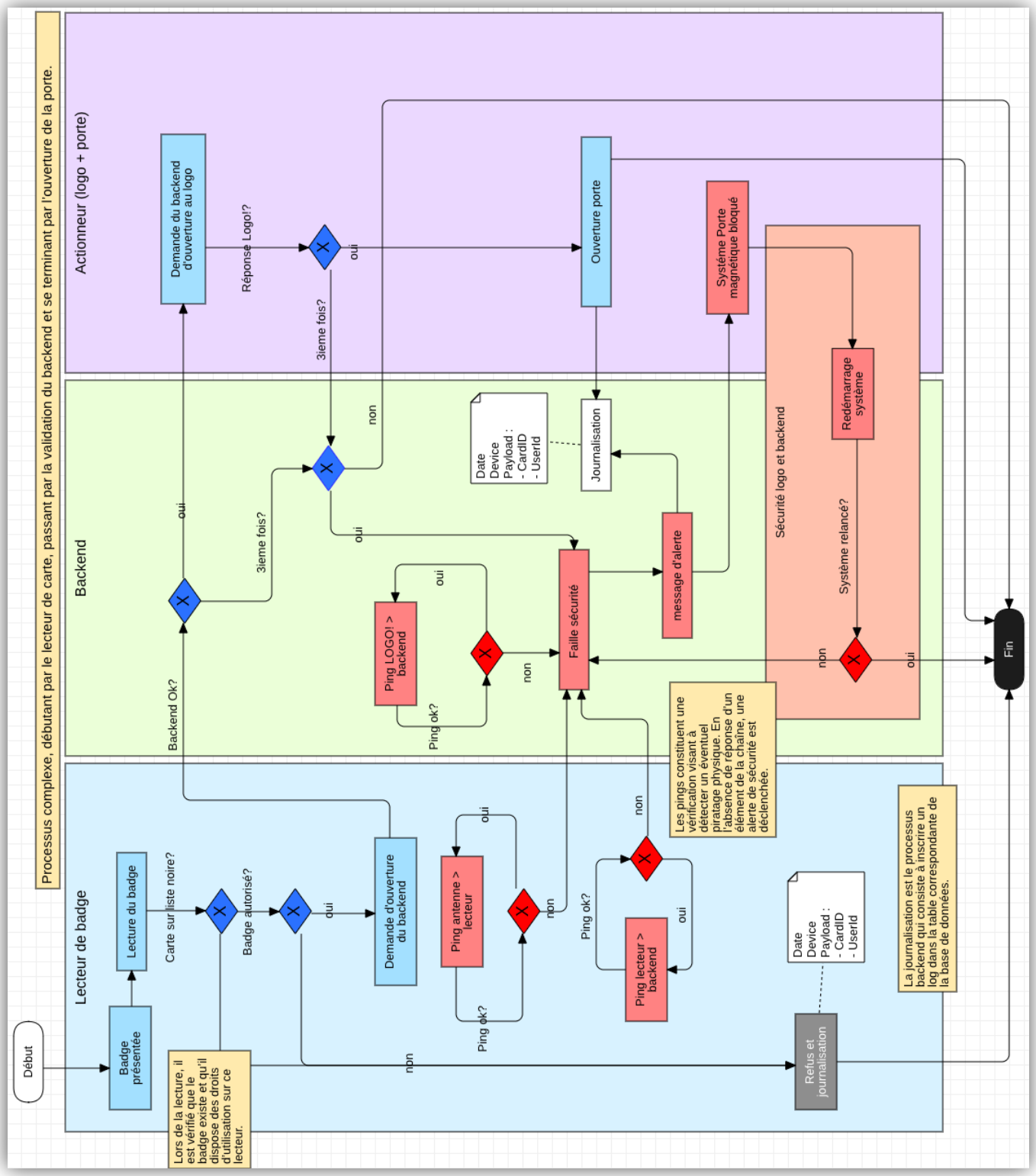


Figure n°26 : Schéma de processus de lecture d'un badge

5.8 *Configuration du broker Mosquitto avec un port sécurisé (SSL/TLS)*

Le port sécurisé TSL avec clé SSL permet de se connecter au réseau mqtt protégé et il ne sera possible de publier et souscrire que si l'on possède la clé du serveur et des clés esclaves lisibles par celui-ci. Dans le processus, il faut commencer par générer un certificat d'autorité (CA) qui est utilisé pour établir la confiance entre un client et un broker MQTT

```
include_dir /etc/mosquitto/conf.d

allow_anonymous false
password_file /etc/mosquitto/passwd

listener 1883
protocol mqtt

#listener secu
listener 8883
protocol mqtt
cafile /etc/mosquitto/ca_certificates/ca.crt
certfile /etc/mosquitto/ca_certificates/server.crt
keyfile /etc/mosquitto/ca_certificates/server.key
required_certificate true
use_identity_as_username true
password_file /ect/mosquitto/paswd

#logs

log_dest syslog
log_dest stout
log_type warning
connection_messages true
```

Figure n°27 : fichier de configuration MQTT sur le Raspberry Pi

Ensuite, il faut générer le certificat serveur (avec SAN) afin que les clients valident correctement le certificat. En général l'adresse IP fixe ou le nom DNS du broker figure dans le SAN (Subject Alternative Name). Le SAN est une extension d'un certificat numérique qui permet à un seul certificat de sécuriser plusieurs noms de domaine, sous-domaines ou adresses IP. Autrement dit, au lieu d'utiliser un certificat pour chaque nom de domaine, un certificat SAN peut en sécuriser plusieurs, simplifiant ainsi la gestion et réduisant les coûts.

Il ne reste plus qu'à générer les certificats clients pour chaque élément du réseau. Quand cela est fait, il faut fournir les certificats au backend et aux deux ESP32 (lecteur et enregistreur). Concernant le backend ce n'est pas très compliqué ; il suffit de créer un dossier et placer les fichiers générés dedans. Pour ce qui est des microcontrôleurs, il faut générer les certificats puis les convertir en fichier de type C++ qui pourront être téléversé dans la mémoire des microcontrôleurs.

L'écriture des commandes pour convertir les fichiers en C++ sur Windows est très complexe et, après plusieurs tentatives infructueuses, j'ai procédé avec le terminal linux du Raspberry Pi dont les commandes sont bien plus simples.

5.9 *Le refactor du backend*

5.9.1 Acceptation des deux ports MQTT (1883 et 8883)

En communication MQTT soit celle-ci est ouverte avec le port 1883 soit elle est sécurisée avec le port 8883. En général, les petits IoTs ne sont pas sécurisés et donc pour avoir contact, on passe par le port 1883. Pour la sécurité du système, on doit impérativement passer par le port sécurisé. Il fallait donc que les deux ports soient ouverts et lisibles en même temps par le serveur nodeJS. J'ai retravaillé sur le backend afin que celui-ci puisse faire cette double

communication. En faisant cela, j'ai remarqué une fuite de mémoire car la souscription se rechargeant toutes les secondes, ajoutant un élément identique à la pile mémoire existante. Ce qui lui faisait lire en boucle une donnée et ce jusqu'à saturation. Exemple avec un capteur de température, après 10 rechargements, à la réception de la donnée, celle-ci aurait été lue 10 fois. Nous avons pu régler cela mais une autre erreur est survenue avec une déconnexion au port 1883.

5.9.2 La petite erreur qui bloque le système

Après la relance du système complet avec l'API Logo, le simulateur de température, tous les topics MQTT sont bien reçus par le backend mais après quelques secondes, ceux-ci arrêtent d'être réceptionné. Après avoir regardé avec MQTT explorer, j'ai constaté que les messages étaient toujours reçus donc cela ne pouvait venir que de l'erreur décrite plus haut. Après deux bonnes heures de recherche, celle-ci a été déterminée. Il y a un cycle de lecture des topics qui prend le port 1883 et lui ajoute les certificats de validation. Comme ce port n'en a pas besoin, le broker MQTT rejette celui-ci et ferme la communication. Il a fallu créer un algorithme qui permet de différencier les ports afin que les certificats ne soient pris en compte que pour le 8883. Après modification, tout est redevenu nominal.

5.10 Rendre le code plus propre

Les codes pour les firmwares des ESP32 devenant de plus en plus lourds, plus de 400 lignes. Il a fallu créer des librairies arduino dit « helper » afin de faciliter la lecture du code mais aussi pour avoir une réutilisation plus facile si le besoin se fait ressentir. En somme, comme quand on appelle une librairie téléchargée sur la plateforme arduino, nous pouvons créer nos propres librairies. Ainsi, les librairies pour la gestion de la led, pour les appels MQTT, pour la communication avec le module et une pour la gestion de l'écran Nokia 5110 ont été créées.

5.11 Mise en place de la première sécurité

Ensuite, a été créée l'ouverture de porte sur le Siemens Logo. Celle-ci déclenche le relai de sortie Q3 du LOGO. La possibilité que l'on coupe la communication avec le module RFID a été pris en compte. Si celui-ci ne donne pas signe de vie, toutes les 5 secondes, une alerte est envoyée au backend qui le transmet ensuite au Logo!8 . Ce dernier a un décompte et après cinq alertes, il bloque la possibilité d'ouvrir la porte et envoie un mail d'alerte.

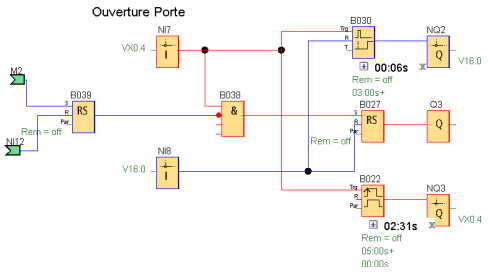


Figure n°28 : activation de la porte lorsque l'accès est autorisé

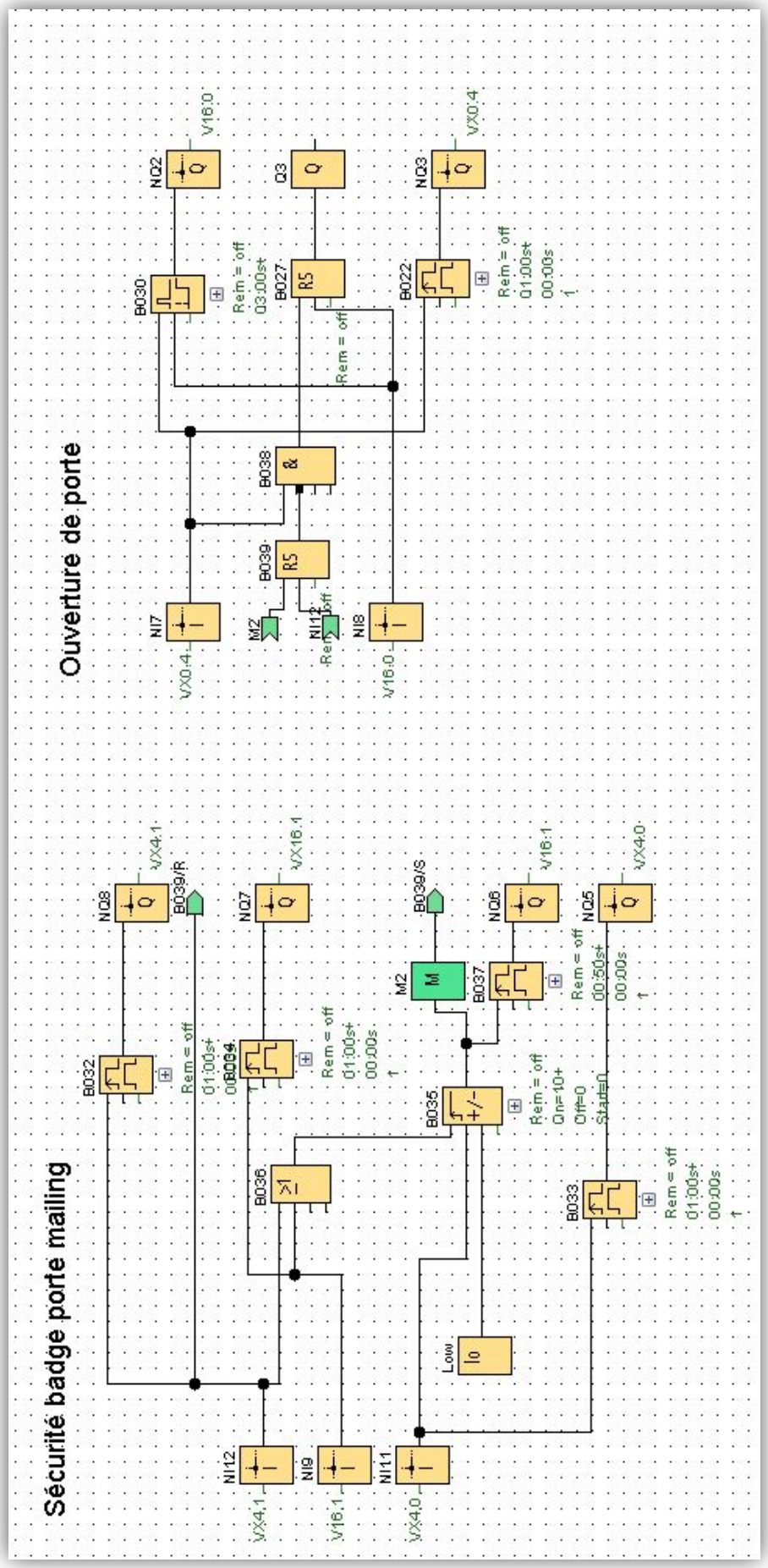


Figure n°29 : codage d'activation de l'ouverture et de la sécurité de la porte dans le logiciel logo SoftComfort

Afin de tester, le reset via le Logo!8, pour l'instant, lorsqu'on rebranche le module, après une minute l'alerte est stoppée. Dans le futur, ça devra se faire par une manipulation de l'administrateur.

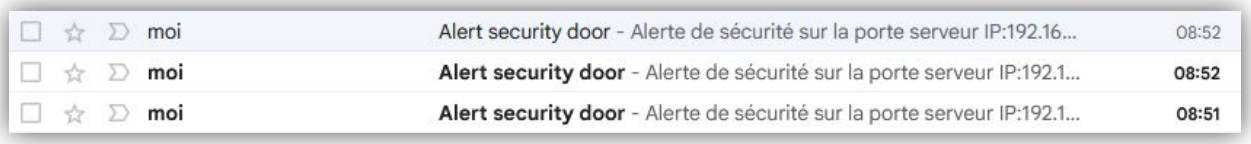


Figure n°30 : message d'alerte reçu dans la boite mail

5.12 L'interface utilisateur

Dernière partie du projet, c'est évidemment l'interface utilisateur. Au niveau des stage, cette partie n'était pas prévue. Mais je voulais absolument avoir une partie de présentation en condition de démonstration réelle.

5.12.1 La connexion

Les routes de login et reset password ont été créées au niveau du backend. Ensuite les formulaires de connexion ont été mis en place sur le frontend. Pour finir, la liaison avec le backend a été faite. Un utilisateur peut se connecter ou changer son mot de passe si celui-ci l'a oublié. Quand ce dernier se connecte, il est directement redirigé vers le dashboard. Nous verrons ce dernier plus tard dans ce chapitre. Pour l'instant six espaces sont prévus. Le tableau de bord, le journal des accès, les chauffages (pas développé pour l'instant), les éclairages (pas développé pour l'instant), les appareils (liste d'IoTs), les utilisateurs.

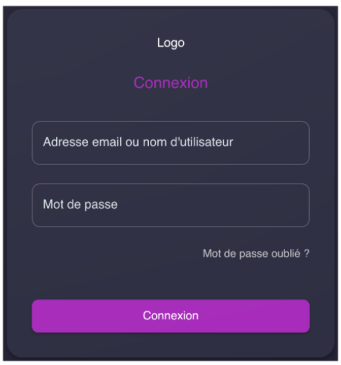


Figure n°31 : page de connexion à l'application

5.12.2 L'onglet Utilisateurs.

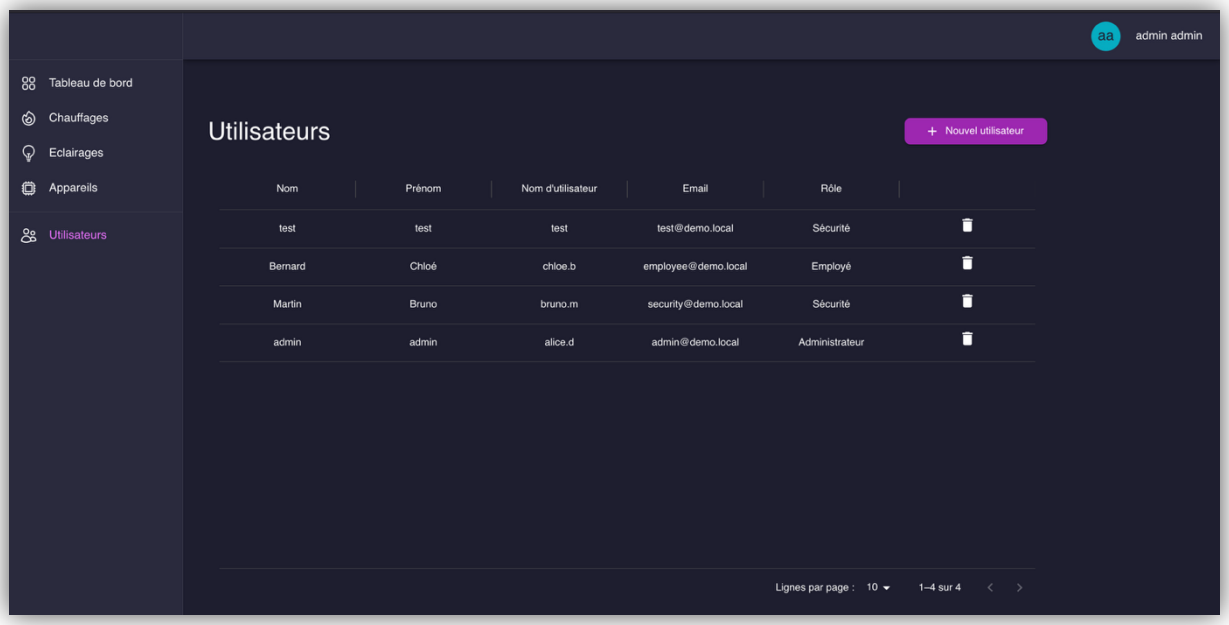


Figure n°32 : Liste des utilisateurs

Lorsqu'on clique sur cet onglet on est dirigé vers une liste des utilisateurs. Dans celle-ci, on peut créer un nouvel utilisateur en cliquant sur le bouton approprié, il suffit de renseigner les informations de celui-ci ; nom d'utilisateur, nom, prénom, email, rôle,… Lorsque celui-ci est enregistré, un email lui est envoyé et à partir de cet email, un lien est fourni qui le redirige vers la modification du mot de passe. Il est évident que personne ne connait le mot de passe du nouvel utilisateur. Celui-ci n'est même pas enregistré en clair dans la base de données et donc il doit absolument passer par cette étape pour pouvoir se connecter.

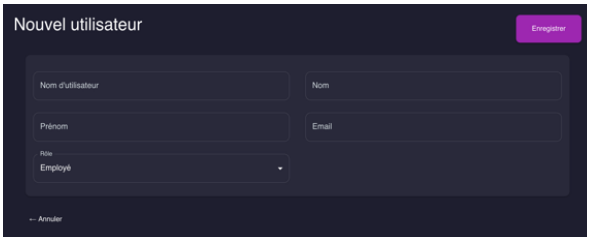


Figure n°33 : formulaire de création d'un utilisateur

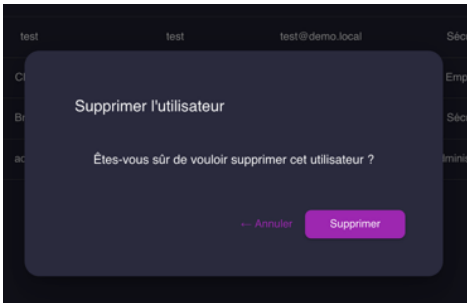


Figure n°34 : modale de confirmation de suppression d'un utilisateur

On peut aussi modifier un utilisateur en cliquant sur l'un de ceux-ci dans la liste et directement en supprimer un utilisateur à partir de celle-ci en cliquant sur la poubelle. Une modale de confirmation s'ouvre afin de valider cette suppression.

C’est à partir de la page d’édition d’un utilisateur que l’on peut enregistrer un nouveau badge. Si un badge est déjà enregistré, il ne sera pas possible d’en refaire un sans détruire le premier. Il est possible de bloquer temporairement celui-ci. Si l’on détruit un badge, il sera définitivement inutilisable par la perte des clés A et B permettant la lecture et l’écriture sur ce dernier.

Figure n°35 : formulaire de mise à jour d’un utilisateur avec l’enregistrement d’un badge

5.12.3 L’onglet appareils.

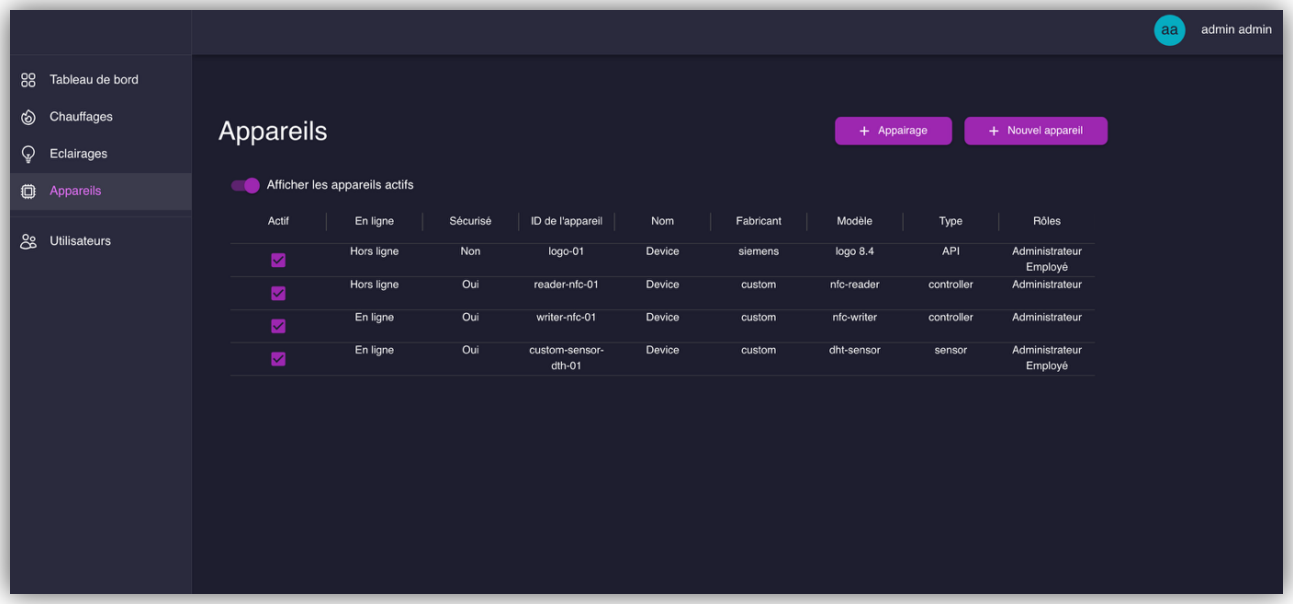


Figure n°36 : Liste des appareils

Lorsqu’on clique dessus, celui-ci nous redirige aussi vers une liste mais cette fois ci avec tous les IoTs reconnus par l’application. Dans cette liste, nous pouvons voir d’un coup d’œil si des IoTs sont hors ligne, ce qui veut dire qu’il n’y a pas de communication et qu’aucunes données ne transitent et s’il utilise le protocole sécurisé ou non. On peut d’un clic désactiver l’un d’eux.

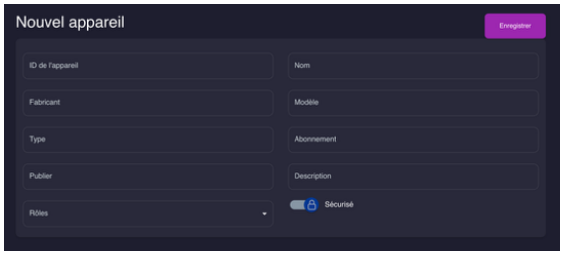


Figure n°37 : Formulaire d’ajout d’un nouvel appareil

Il est possible d’enregistrer un nouvel appareil en lui fournissant les informations nécessaires. Ce qui permet d’avoir des IoTs fait personnellement avec le protocole MQTT. Un bouton appairage existe mais pour l’instant, nous le laissons de côté car celui-ci est prévu pour le futur et la possibilité d’enregistrer des IOTS en Zigbee ou thread.

Dans la liste, nous pouvons afficher les appareils inactifs. De base, seuls les actifs apparaissent. Il est évident que nous pouvons modifier des appareils en cliquant sur l’un d’eux.

5.12.4 L'onglet tableau de bord

Cet onglet permet de visualiser en temps réel les données transmises par l'automate et le capteur. Pour cela, nous ne passons plus par une requête HTTP, car ce protocole est unidirectionnel : on envoie une demande et l'on reçoit une réponse.

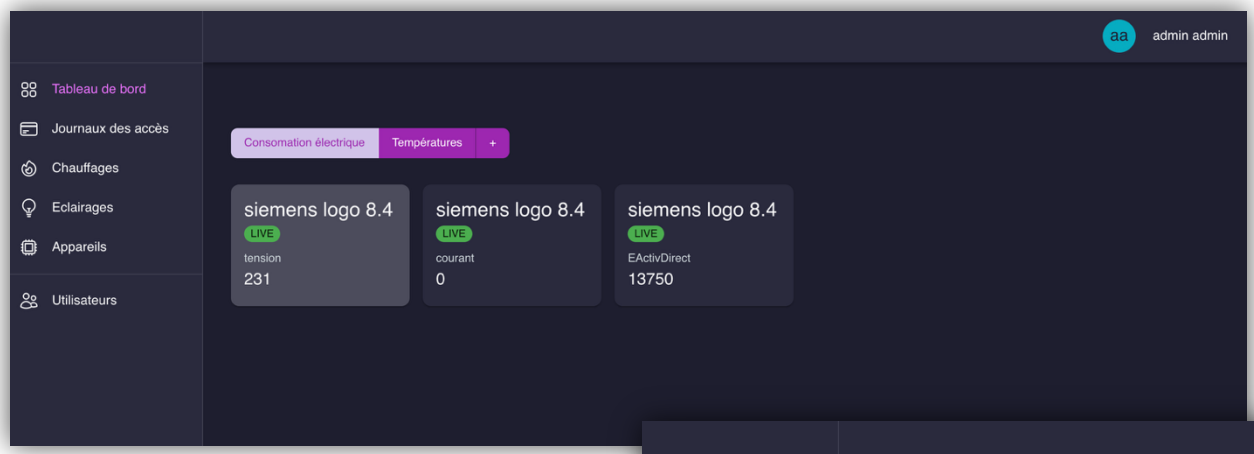


Figure n°38 : tableau de bord de l'API

Il est normalement possible d'automatiser des requêtes HTTP à intervalles réguliers, mais cela entraînerait une perte de données ou exigerait une fréquence élevée qui risque de saturer le système. De manière générale, on évite donc de multiplier les requêtes HTTP en chaine.

Figure n°39 : tableau de bord du simulateur de température

À la place, la communication passera par un canal WebSocket. Ce protocole est bidirectionnel, c'est-à-dire qu'une fois la communication ouverte, chaque changement détecté par le backend est immédiatement transmis. En parallèle, le frontend peut aussi envoyer ses propres requêtes et recevoir les données en continu.

5.12.4.1 Le groupe de boutons

Pour naviguer entre les pages, un groupe de boutons est disponible. Dans cette démonstration, deux boutons sont fonctionnels : l'un mène à la page des données fournies par l'automate et l'autre à celles fournies par le capteur.

À l'avenir, ce groupe de boutons sera dynamique. Seul celui de l'automate restera générique ; pour les autres, les utilisateurs pourront créer leurs propres pages en fonction de leurs objets connectés et de leurs préférences.

Lorsqu'un bouton est actionné, une requête WebSocket est envoyée afin d'ouvrir une communication avec les éléments de la page correspondante affiché avec le flux continu de données.

5.12.5 L'onglet Journal d'accès

Prévisualisation des accès au lecteur de badges servant à donner l'information des passages dans la salle sécurisée par le lecteur de badges. Chaque passage est enregistré dans la base de données, pour avoir une traçabilité des allées et venues.

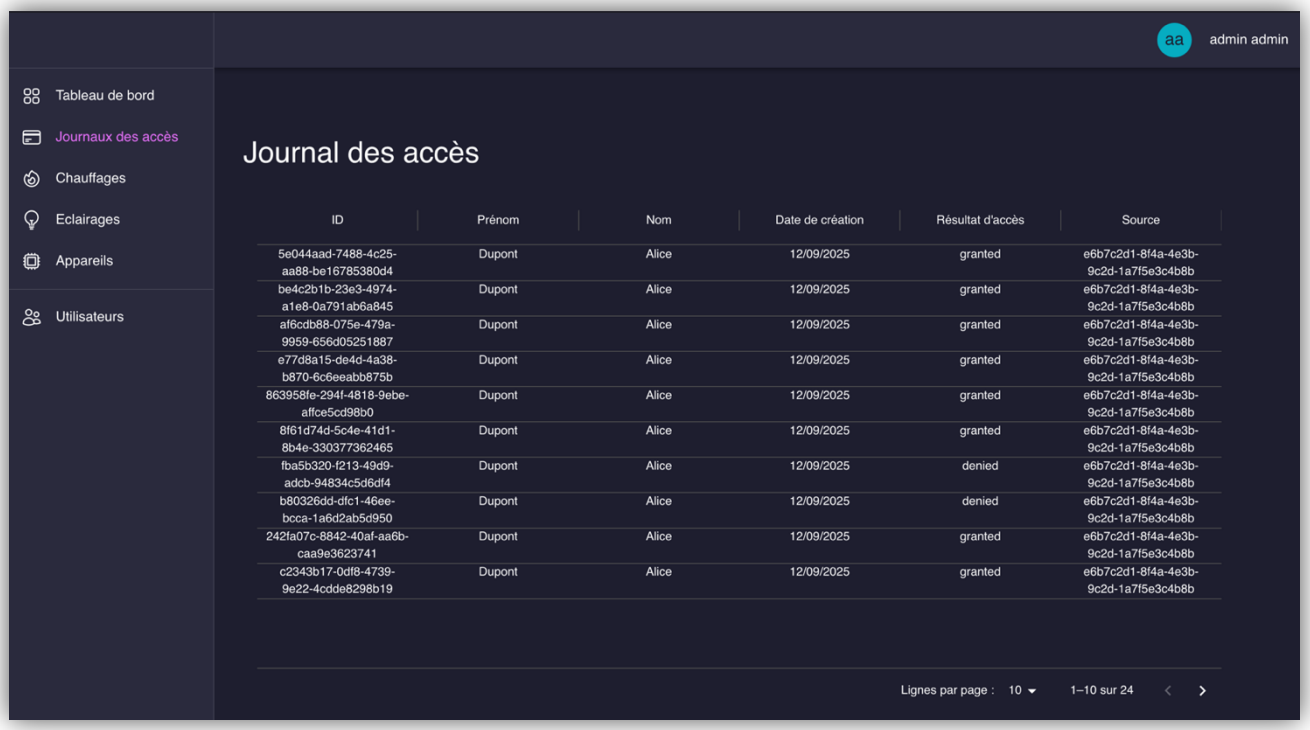


Figure n°40 : liste des journalisations des accès enregistrés à partir d'une lecture d'un badge

6 Le Déploiement de l'application

Ceci est la dernière étape : rendre l'application totalement autonome. Une fois le projet déployé en production, il se lance automatiquement dès le démarrage du Raspberry Pi. En accédant à son adresse, on arrive directement sur la page de connexion.

La version en production pourra être mise à jour à tout moment, que ce soit pour ajouter de nouvelles fonctionnalités ou pour corriger des bugs.

À partir de là, il n'y aura plus un conteneur Docker contenant la base de données mais chaque environnement de l'application disposera de son propre conteneur (frontend, backend, base de données).

6.1 La préparation

En premier lieu, il faut préparer la connexion SSH[24] entre la machine locale et le Raspberry Pi. Le mieux est de créer une paire de clés SSH composée de deux parties :

- **La clé privée** : elle ne doit absolument pas être partagée et reste soigneusement stockée sur la machine locale.
- **La clé publique** : elle se termine par l'extension .pub et permet l'établissement de la connexion sécurisée. C'est cette clé que l'on fournit au serveur afin de pouvoir s'y connecter en toute sécurité.

6.1.1 Création une clé SSH

Sur la machine local, si ce n'est déjà fait créer un dossier caché, vous y rendre dans le terminal et ensuite écrire la commande :

- ssh-keygen -t raspberry-tfe

6.1.2 Création du fichier config

Toujours dans ce dossier, il faut créer un fichier config et y inscrire les lignes ci-dessous

```
Host tfe
   Hostname 192.168.***.***
   User ******
   IdentityFile ~/.<mon_dossie>r/raspberry-tfe
```

les données de host permettant de se connecter rapidement avec ssh user@ip

6.1.3 Authorized_keys

Sur le Raspberry Pi, dans le fichier authorized_keys. Ajouter la clé publique générée sur la machine locale

6.2 Préparation de docker sur le Rapsberry Pi

6.2.4 Ajouter la clé Docker officielle

Afin de pouvoir installer docker, il faut ajouter la clé officielle. Cette partie est décrite par la documentation de docker.

[24] SSH [secure shell] : protocole réseau cryptographique permettant créer une connexion sécurisée à une machine distante via une clé de protection pour exécuter des commandes en mode terminal.

- sudo apt-get update
- sudo apt-get install ca-certificates curl
- sudo install -m 0755 -d /etc/apt/keyrings
- sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o /etc/apt/keyrings/docker.asc
- sudo chmod a+r /etc/apt/keyrings/docker.asc

6.2.5 Ajouter le dépôt aux sources Apt

- echo \
- "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/debian \
- $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
- sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
- sudo apt-get update

6.2.5.1 Installation

- sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

6.2.5.2 Test de l'installation

- sudo docker run hello-world

6.2.5.3 Création un registre

Le registre est un service qui stocke, gère et distribue des images Docker.

- docker run -d -p 5000:5000[25] --restart always --name registry registry:3

6.2.5.4 Création d'un context docker

Permet de se connecter directement à docker sur le serveur à partir de la machine locale

- docker context create <nom_du_context> \
  --docker "host=ssh://utilisateur@hote"

[25] Port d'exposition part défaut de docker pour le registre (port exposé sur la machine hôte / port utilisé à l'intérieur du conteneur)

6.3 Préparation de l'application

6.3.6 L'environnement

Lors du démarrage d'un projet, il est possible de créer différents environnements : des environnements de travail, des environnements intermédiaires pour les tests, et enfin un environnement de production.

Un environnement correspond à un fichier contenant des variables. Cela permet d'éviter d'écrire en dur des mots de passe ou des tokens, et de définir une configuration adaptée à chaque contexte.

Ici l'environnement production a été créé

Exemple de variables pour la production :

MQTT_BASE_URL=192.168.3.100

Exemple de variables pour le développement :

MQTT_BASE_URL= http://localhost:3000

6.3.7 Les DockerFile (en Annexe)

Pour créer les images, il y a besoin d'un fichier DockerFile dans chaque environnement (backend et frontend). Dans celui-ci on y écrit toutes les configurations des images qui vont être générées, du système d'exploitation aux variables d'environnement utilisées.

6.3.8 Le docker compose

Le fichier **docker-compose.yml** décrit plusieurs services (containers) à lancer ensemble. Il définit images, réseaux, volumes et configurations pour orchestrer facilement l'application. C'est avec lui que je vais travailler principalement.

6.3.9 Les scripts

Afin de faciliter le travail, deux scripts sont disponibles :

- **tag.sh** : il met à jour les numéros de version dans tous les fichiers nécessaires, comme le *docker-compose* et les *package.json* de chaque environnement. Un petit fichier texte l'accompagne pour vérifier qu'une version n'a pas déjà été créée.
- **build.sh** : il automatise la création des images qui seront ensuite poussées sur le serveur. À la fin, trois fichiers distincts sont générés.
    - frontend_ « version ».tar
    - Backend_ « version ».tar

- postgres_15-alpine_ « version ».tar

6.4 Procédure de mise en production

Comme la production du démonstrateur n'a pas d'accès à internet, il faut passer par l'étape de création sur le réseau local avec internet car il pourrait y avoir des mises à jour de l'image ou des packages utilisés par le projet, ensuite il reste à basculer sur le réseau du démonstrateur afin de pousser les images avec les nouvelles modifications sur les environnements.

1. Versionning

   - Bash tag.sh v1.0.2

2. Créer les images

   - Bash build.sh

3. Changer de réseau et de context

C'est à cette étape qu'il faut basculer sur le réseau du démonstrateur avec la commande ci-dessous, la machine locale se connecte directement au serveur, sur le context Docker pour charger et pousser les nouvelles images Docker.

   - Docker context use tfe

4. Charger les images

- docker --context tfe load -i production/artifacts/backend_${VERSION}.tar
- docker --context tfe load -i production/artifacts/frontend_${VERSION}.tar
- docker --context tfe load -i production/artifacts/postgres_15-alpine.tar

5. Pousser les images dans le registre privé du Pi

- docker --context tfe push ${REG}/backend:${VERSION}
- docker --context tfe push ${REG}/frontend:${VERSION}
- docker --context tfe push ${REG}/postgres:15-alpine

6. Déployer avec la commande compose sur le Pi

- docker --context tfe compose -f docker-compose.production.yml up -d

Compose va démarrer les nouvelles images et si tout se passe bien, l'accès à l'application s'exécutera via l'adresse http://192.168.3.100

Pour vérifier que conteneurs avec leurs images sont bien lancés.

- docker --context tfe ps

Il est possible de voir les logs avec les lignes

- docker --context tfe logs backend --tail=50 #50 dernier logs
- docker --context tfe logs frontend --tail=20 #20 dernier logs

6.5 In Fine

Cette partie critique a été longue à mettre en place car beaucoup de problèmes surviennent toujours à ce moment-là quel que soit le projet. Lors de celle-ci, il a fallu plusieurs fois redéployer celui-ci car soit les images ne démarraient pas à cause d'une mauvaise variable ou configuration et il a fallu synchroniser correctement le backend avec les IoTs et le web Socket.

7 Sécurité du système

Étant un démonstrateur, la sécurité du système a été mise de côté, tout en gardant à l'esprit que la suite devra intégrer la mise en place de garde-fous concernant aussi bien l'aspect physique que logiciel de la solution proposée.

7.1 Le matériel physique

Dans ce dossier, certains problèmes liés au démonstrateur ont déjà été soulevés, comme le lecteur de badges fonctionnant via le réseau Wi-Fi 802.11n. Il est aussi à noter que j'utilise un modem Wi-Fi 4 qui commence doucement à dater. Il reste bien évident que la connexion filaire reste plus sécurisée, avec des conditions supplémentaires mises en place afin de garantir un niveau élevé de protection, même dans une démarche de traçabilité. Nous savons également que la sécurité des badges eux-mêmes reste limitée pour l'instant et qu'un renforcement devra être envisagé à ce niveau.

Si la solution venait à être adoptée et que l'entreprise décidait d'aller plus loin, il serait certainement nécessaire de réaliser un audit de sécurité et de procéder à une phase de Pentest[26] afin d'identifier les améliorations supplémentaires à mettre en œuvre pour renforcer le système.

[26] PenTest : test d'intrusion, méthode de test de vulnérabilité d'un système réseau avec des outils de cyberattaques

7.2 Le réseau

Celui-ci intègrera plusieurs protocoles, dont certains comme WebSocket ou HTTP, qui ne sont pas parmi les plus sécurisés. Pour atteindre un niveau de sécurité élevé, il sera probablement nécessaire de mettre en place un périmètre défini par logiciel à l'aide d'une solution ZTNA, permettant d'accéder au système interne depuis l'extérieur. Ce dispositif renforcerait la protection de la zone interne contre toute intrusion simple, sans recourir à des solutions trop complexes.

Une alternative consisterait à utiliser un VPN, mais celui-ci repose sur un modèle de confiance, ce qui est à l'opposé du principe de la ZTNA : « ne jamais faire confiance, toujours vérifier ».

7.3 Le code de l'application

Souvent oubliés dans la sécurité, et particulièrement au regard du RGPD, ce sont nos propres codes. Il est donc nécessaire de réaliser des tests afin de vérifier qu'ils ne soient pas faciles à contourner. Une simple injection SQL pourrait, par exemple, donner un accès complet à l'application et permettre de contrôler les appareils qui y sont reliés. Les conséquences pourraient aller de la simple récupération des données des utilisateurs jusqu'à l'envoi d'une confirmation à l'API pour l'ouverture d'une porte.

Pour ce faire, il faut passer par l'utilisation d'applications qui testent la solidité du code, telles que SonarQube[27] ou Snyk[28], afin de vérifier à la fois la qualité et la sécurité. Du point de vue du RGPD, un travail technique et juridique pourrait également être nécessaire concernant la gestion des cookies, la protection des données personnelles et la documentation des traitements associés.

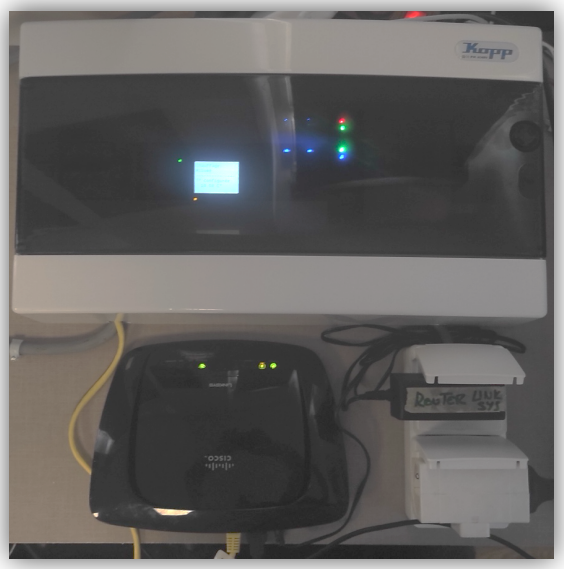


Figure n°41 : Platine de démonstration finalisée

[27] https://www.sonarsource.com/products/sonarqube/
[28] https://snyk.io/fr/

8 Résultats

Sachant que je devais faire une proposition à la fin des stages dont la durée cumulée équivalait à six semaines, l'objectif était de parvenir à mettre en place un dialogue entre chaque élément jusqu'au développement du frontend sans visuel créé.

Pendant les stages, les réalisations mises en place ont été les suivantes :

- Créer un dialogue entre un IoT et l'application via MQTT.
- Lire et enregistrer les données de l'IoT dans la base de données via l'application.
- Créer un dialogue entre un automate et le compteur d'énergie via Modbus.
- Récupérer, lire et afficher sur l'écran de l'API les données fournies par le compteur d'énergie.
- Créer un dialogue entre l'automate et l'application via MQTT.
- Renvoyer les données d'énergie directement à l'application afin qu'elles soient lues et enregistrées dans une base de données.
- Communiquer en MQTT sécurisé avec un lecteur et un enregistreur de badges.
- Écrire sur un badge RFID une clé cryptée avec l'enregistreur.
- Lire un badge RFID et vérifier qu'elle existe et qu'elle soit liée à un utilisateur.
- Lors de la lecture, afficher sur l'écran du lecteur si l'accès est autorisé ou refusé, avec un retour visuel par LED.
- Fournir à l'API la confirmation d'ouverture d'une porte.
- Fermer automatiquement la porte après quelques secondes.
- Fournir une alerte à l'API et, après cinq alertes, bloquer l'ouverture de la porte et envoyer un e-mail d'alerte.
- Depuis l'application, fournir un setpoint ainsi que des valeurs de température permettant de commander l'allumage ou l'extinction d'un chauffage.
- Afficher le setpoint et la température sur l'écran de l'API.

Hors des stages, les réalisations mises en place ont été les suivantes :

- Connexion à l'application via le frontend.
- Possibilité de changer le mot de passe.
- Liste des utilisateurs et liste du matériel regroupant les IoT et l'API.
- Création et modification d'un utilisateur.
- Création d'un badge associée à un utilisateur, avec vérification que le badge n'existe pas déjà et qu'un utilisateur ne possède pas déjà un badge actif.
- Blocage temporaire d'un badge.
- Création et modification d'un IoT.
- Activation ou désactivation d'un IoT.
- Visualisation directe de l'état d'un IoT (en ligne ou hors ligne).
- Visualisation des lectures de badges.

9 Conclusion et perspectives

Ce projet a été une belle expérience. J'ai pu m'y investir pleinement et j'y ai beaucoup appris. Le fait d'avoir travaillé seul dessus m'a permis de m'affirmer en programmation. Comme beaucoup de développeurs juniors, je ressens parfois le syndrome de l'imposteur, et ce genre de projet, avec ses réussites et ses frustrations, aide à gagner en confiance et en logique de code. J'ai pu renforcer des compétences déjà acquises, comme la programmation en C++, mais aussi en apprendre davantage sur des technologies que je n'avais jamais implémentées dans un projet JavaScript.

J'ai aussi pas mal appris sur l'automate programmable et j'ai vraiment apprécié de travailler avec le petit Logo!8 de Siemens, sur la transaction de données avec l'application et surtout avec le compteur d'énergie. J'ai parfois eu du mal avec le logiciel dédié, mais après avoir vu plusieurs logiciels pendant ma scolarité, ça m'a permis de me sentir plus sûr de moi quant à devoir les reprendre en main. Je pense avoir gagné en maturité avec la prise en main de ce logiciel.

Grâce à cela, j'ai aussi acquis des compétences en gestion de projet dans le domaine de l'informatique et de l'automatique, me dépasser dans la prise en main d'un projet et m'ouvrir à de nouvelles possibilités dans le domaine de l'automatique.

Dans l'ensemble, je pense avoir atteint l'objectif pour présenter le projet. Les tâches demandées ont été réalisées, parfois avec des difficultés, parfois plus facilement.

Concernant le potentiel pour une installation futur au sein de nos bureaux, nous sommes satisfaits du résultat obtenu et nous pensons qu'il serait intéressant de continuer ce développement. Cependant, le projet reste encore fragile et il faudra corriger et renforcer certains points. La suite passera sûrement par un cahier des charges un peu plus détaillé, ainsi qu'une meilleure analyse des besoins, aussi bien pour l'entreprise que pour d'autres situations possibles.

Le projet ne sera certainement pas développé juste après ce TFE, mais il est certain qu'il y aura une continuité tôt ou tard. On a pu constater un potentiel avec le développement de celui-ci et il serait dommage de ne pas lui donner toutes ses chances d'exister pleinement.

Je vous remercie de votre attention et j'espère que ce dossier vous aura apporté un éclairage technique tout en vous permettant de partager notre vision de ses potentialités.

ANNEXES

Annexe A. Retro planning

Semaine	Période	Objectifs	Description
30	25/08/2025 - 31/08/2025	Finalisation et remise du dossier TFE	
29	18/08/2025 - 24/08/2025	Peaufinement du déploiement démonstratif pour le TFE Écriture Dossier TFE !!!	
28	11/08/2025 - 17/08/2025	Écriture Dossier TFE !!! test + gestion des bugs	
27	04/08/2025 - 10/08/2025	Écriture Dossier TFE !!! test + gestion des bug Crud IOT	Pouvoir gérer les règles de température afin de modifier dynamiquement la température d'une pièce / créé des horaires de température Pouvoir ajouter un badge d'identification
26	28/07/2025 - 03/08/2025	Écriture Dossier TFE !!! Crud IOT	Pouvoir récupérer les élément IOT connu modifier leur nom
25	21/07/2025 - 27/07/2025	Écriture Dossier TFE !!! fin Crud user (back/front)	
24	14/07/2025 - 20/07/2025	Écriture Dossier TFE !!! Crud User + rôle (back/front)	
23	07/07/2025 - 13/07/2025	Écriture Dossier TFE !!! suite statistique dashboard (back/front)	
22	30/06/2025 - 06/07/2025	Écriture Dossier TFE !!! création statistique compteur énergie (back/front)	
21	23/06/2025 - 29/06/2025	Écriture Dossier TFE !!! création du dashboard avec statistique température (back/front)	
20	16/06/2025 - 22/06/2025	Date butoir second Stage - rendre rapport final second stage Rapport de stage !!!	
19	09/06/2025 - 15/06/2025	Rapport de stage !!! création du login / reset password frontend-backend	
18	02/06/2025 - 08/06/2025	Rapport de stage !!! Mise en place frontend	Mise en place du frontend en react
17	26/05/2025 - 01/06/2025	Rendre seconde partie second rapport de stage Rapport de stage !!!	Vide pour la gestion du retard sur le travail
16	19/05/2025 - 25/05/2025	Rapport de stage !!! communication Logo!8 --> Backend erreur porte / erreur chauffage	S'il y avait une coupure de communication entre la porte ou le chauffage le LOGO doit pouvoir envoyer un message d'alerte au backend + voyant rouge d'erreur
15	12/05/2025 - 18/05/2025	Rapport de stage !!! communication backend --> Logo!8 ouvertures de porte / ou blocage	Régle de porte : Si le badge est autorisé : la porte s'ouvre / un voyant vert s'active Si le badge est refusé : un message est envoyé sur un écran (3310) / un voyant rouge s'active
14	05/05/2025 - 11/05/2025	Rendre première partie second rapport de stage Rapport de stage !!! mise en place d'un écran (3310) et de voyant sur l'Arduino mise en place d'un voyant rouge et un vert pour la porte sur le LOGO	
13	28/04/2025 - 04/05/2025	Rapport de stage !!! communication entre backend <--> arduino (MQTT RJ45) pour récupérer les autorisations de badge	Règle : toutes les 30sec le backend et l'Arduino communique afin de voir s'il n'y a pas de coupure (sinon envoi d'une alerte) le LOGO mettra en sécurité la porte (désactivation de l'ouverture) qui ne pourra être réactivée que par un code sur le LOGO lui-même
12	21/04/2025 - 27/04/2025	Début second stage Rapport de stage !!! lecteur de badges avec un arduino (physique) lire les données du badge -->arduino	

11	14/04/2025 - 20/04/2025	Date butoir premier stage - rendre Rapport de stage final Rapport de stage !!	Vide pour la gestion du retard sur le travail
10	07/04/2025 - 13/04/2025	Rapport de stage !! Stabilisation de toute les communication (si nécessaire)	
9	31/03/2025 - 06/04/2025	Rapport de stage !! Mise en place des règles des températures Logo!8 communication MQTT ESP32 --> Logo!8	Règles de ° : si ° en dessous de la t° demandée démarrage du chauffage si ° au-dessus de la température demandée arrêt du chauffage Nous placerons un indicateur lumineux qui fera référence de chauffage
8	24/03/2025 - 30/03/2025	Rendre deuxième partie rapport de stage Rapport de stage !! Mise en place des règles Logo!8 si coupure avec le réseau teste de coupure réseau	
7	17/03/2025 - 23/03/2025	Rapport de stage !! communication Logo!8 <--> backend - TCP enregistrement DB backend des données reçues du Logo!8	
6	10/03/2025 - 16/03/2025	Rapport de stage !! fin communication modbus (si non fini) mise en place DB Logo!8 + enregistrement des données compteur	
5	03/03/2025 - 09/03/2025	Rapport de stage !! Mise en place du compteur d'énergie (physique) communication compteur --> Logo!8 (Modbus) configuration de la passerelle modbus <--> RJ45	
4	24/02/2025 - 02/03/2025	Début du premier stage Rapport de stage !! enregistrement des données reçu de l'ESP32 dans la db backend test écoute ESP32 et sauvegarde des données	
3	17/02/2025 - 23/02/2025	Rapport de stage !! écoute capteur température - ESP32 envois des donnée température en (MQTT WIFI) ESP32 --> backend	Problème avec l'esp32 et le capteur de température (essaye avec s capteur) on va voir avec un autre ESP (achat en mars d'une version adafruit)
2	10/02/2025 - 16/02/2025	Rapport de stage !! Fin de la mise en place backend communication ESP32 --> Backend MVP	
1	03/02/2025 - 09/02/2025	Préparation documents pour les stages MoSCoW Rétroplanning Mise en place backend	

Annexe B. Protocoles de communication.

- ***MQTT :***

  - **Présentation** : initialement Message Queuing Telemetry Transport

Le MQTT est un protocole basé sur le TCP/IP qui fonctionne en publication/abonnement axé sur l'internet des objets connectés. Le protocole est né en 1999 pour la surveillance d'un oléoduc. Celui-ci devait être léger et efficace avec une petite bande passante. En 2013, le protocole devient un standard ouvert de L'Organization for the Advancement of Structured Information Standards (OASIS) qui est un consortium mondial travaillant pour la standardisation de formats de fichiers ouverts fondés notamment sur XML.

  - **Caractéristiques principales** :
    - *Publication/abonnement* :
      - Utilise un broker (serveur spécifique au MQTT) central de découplage éditeur-abonné
    - *Légèreté et faible empreinte* :
      - En-tête minimale (2 octets + taille du topic)
      - Spécialement conçu pour des environnements contraints (IoT embarqué, réseaux à faible bande passante)
    - *Qualité de service* :
      - QoS 0 "Au plus une fois" : envoi "fire-and-forget" sans accusé
      - QoS 1 "Au moins une fois" : retransmission jusqu'à réception d'un ACK
      - QoS 2 "Exactement une fois" : échange à quatre phases pour éviter les doublons
    - *Sécurité* :
      - Authentification via username/password, certificats X.509
      - Chiffrement TLS pour protéger la confidentialité et l'intégrité
    - *Cas d'usage typiques* :
      - IoT et capteurs (« smart home », industrie 4.0)
      - Applications mobiles nécessitant une synchronisation légère
      - Messagerie machine-à-machine (M2M)

- ***Modbus :***

  - **Présentation** :

Créé en 1979 par Modicon et racheté par Schneider, c'est un protocole de communication non-propriétaire qui permet à des composants industriels à communiquer entre eux, soit via communication série, soit via TCP/IP. Il s'agit d'une norme véritablement ouverte et le protocole de réseau le plus utilisé dans l'environnement de fabrication industrielle. Il a été mis en œuvre pour transférer des entrées/sorties/sorties analogiques et enregistrer des données entre des dispositifs de contrôle. C'est un lingua franca ou un dénominateur commun entre différents fabricants. Un rapport l'a qualifié de "norme de fait en matière d'intégration multifournisseurs". Les analystes de l'industrie ont signalé plus de 7 millions de nœuds Modbus rien qu'en Amérique du Nord et en Europe.

  - **Caractéristique principale** :
    - *Communication multiple* :
      - ModbusRTU (Remote Time Unit) : Communication en série
        - Format binaire, plus compact et efficace en bande passante
        - Utilise un codage CRC16 pour la détection d'erreurs
        - Trame : adresse esclave (1 octet) + code fonction (1 octet) + données (N octets) + CRC (2 octets)
      - Modbus TCP/IP : utilisant des liaisons Ethernet avec le port 502
        - Format binaire, plus compact et efficace en bande passante
        - Utilise un codage CRC16 pour la détection d'erreurs
        - Trame : adresse esclave (1 octet) + code fonction (1 octet) + données (N octets) + CRC (2 octets)
    - *Architecture serveur-client*
      - Le client interroge périodiquement un ou plusieurs serveurs
      - Chaque client possède une adresse unique (1–247)
      - Un seul serveur par segment de bus (sauf en Modbus TCP où plusieurs clients peuvent se connecter)
    - *Débit et performances*
      - Sur RS-485, 1 200 à 115 200 bauds typiques
      - Jusqu'à 32 nœuds (slaves) par segment RS-485 sans répéteur (extensible avec répéteurs)

- ***Http :***

  - **Présentation** : *HyperText Transfert Protocol*

Créé en 1990, HTTP (HyperText Transfer Protocol) est un protocole de communication client-serveur conçu pour le World Wide Web. Les clients HTTP les plus connus sont les navigateurs Web, mais HTTP est également utilisé hors du Web pour implémenter des API distribuées reposant sur SOAP ou REST.

  - **Caractéristiques principales** :
    - *Basé sur TCP/IP* :
      HTTP fonctionne au-dessus de la pile TCP/IP (port par défaut : 80 pour HTTP, 443 pour HTTPS).
    - *Format textuel* :
      Les en-têtes et le début des messages (ligne de requête et ligne d'état) sont exprimés en texte lisible, ce qui facilite le débogage.
    - *Modèle requête-réponse* :
      Le client envoie une requête (méthode, URI, en-têtes, éventuellement corps) et le serveur renvoie une réponse (code d'état, en-têtes, corps).
    - *Méthodes standard*
      - **GET** : récupérer une ressource
      - **POST** : envoyer des données au serveur
      - **PUT** : remplacer ou créer une ressource
      - **DELETE** : supprimer une ressource
      - **Autres : HEAD**, **OPTIONS**, **PATCH**, etc.

- ***Websocket :***

  - **Présentation** :

C'est aussi un standard du web comme le http, à la différence du http celui-ci est full duplex c'est-à-dire bidirectionnel et persistant entre un client (navigateur, application) et un serveur et donc celui-ci n'a pas besoin d'une requête du client pour recevoir une réponse. Le websocket permet l'échange de données instantanément et sans réouverture de la connexion.

  - **Caractéristiques Principales** :

    - **Connexion persistante** : une fois ouverte, la connexion reste active jusqu'à sa fermeture explicite.
    - **Bidirectionnelle** : client ↔ serveur peuvent s'envoyer des messages à tout moment.

- **Basé sur TCP** : il utilise une seule connexion TCP, ce qui réduit la surcharge.
- **Faible latence** : idéal pour des échanges rapides et fréquents.

- ***UART :***

- **Présentation** : *Universal Asynchronous Receiver Transmitter*

Émetteur-récepteur asynchrone est un protocole courant dans le monde de l'informatique et de l'électronique. Comme avec le port série sur les ordinateurs. La norme est née début des année 60 avec la norme rs-232 sur transmission en paire torsadée qui posera les bases physiques de la communication UART

- **Caractéristiques Principales** :
  - *Transmission asynchrone*
    Pas de signal d'horloge partagé : l'émetteur et le récepteur s'accordent sur un débit en bauds (ex. 9600, 115200 bauds).
  - *Format de trame configurable*
    - **Bits de données** : généralement 5 à 9 bits (courant : 8 bits)
    - **Bit de parité** : aucun, pair ou impair (optionnel pour la détection d'erreurs simples)
    - **Bits de stop** : 1, 1.5 ou 2 bits (délimitent la fin de chaque mot)
  - *Full-duplex*
    Deux lignes séparées pour Tx (transmission) et Rx (réception), permettant d'envoyer et de recevoir simultanément.
  - *Contrôle de flux (optionnel)*
    - **Matériel** : RTS/CTS (Request To Send / Clear To Send)
    - **Logiciel** : XON/XOFF
  - *FIFO et buffer :*
    Tampons internes (généralement 16 à 64 octets) pour réduire les pertes de données à haut débit.
  - **Applications courantes**
    Communication série entre microcontrôleurs, modules GPS, modems, capteurs, consoles de debug, etc.

- ***TCP/IP :***

  - **Présentation** : *Transmission control Protocol / Internet Protocol*

Les protocoles TCP et IP sont deux protocoles associés, ils ont été conçus dans les années 70 pour unifier les échanges sur ARPANET : TCP gère l'établissement de connexions fiables et ordonne les données, tandis qu'IP achemine chaque paquet de façon indépendante. Début des année 80 ceux-ci sont officiellement déployés, ce qui posera les bases d'Internet tel que nous le connaissons. Aujourd'hui, TCP reste le protocole de choix pour les applications nécessitant fiabilité et contrôle de flux (web, email, fichiers), et IP assure le routage de tous les paquets entre réseaux.

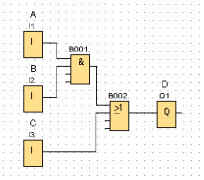
  - **Caractéristiques de TCP** :
    - *Orienté connexion* : Avant d'envoyer des données, TCP établit une « liaison » entre émetteur et récepteur (handshake en 3 étapes).
    - *Fiable* : Chaque segment envoyé doit être acquitté (ACK). En cas de perte, les données sont renvoyées automatiquement.
    - *Contrôle de flux* : Le récepteur indique à l'émetteur combien de données il peut traiter à la fois (fenêtre glissante).
    - *Contrôle de congestion* : TCP adapte dynamiquement son débit pour éviter de saturer le réseau (algorithmes comme Reno, CUBIC…).
    - *Ordonnancement* : Les segments arrivent et sont remis dans le bon ordre avant d'être transmis à l'application.

  - **Caractéristiques d'IP** :
    - *Sans connexion* : Chaque paquet (datagramme) est acheminé indépendamment, sans établir de session préalable.
    - *Best effort* :IP ne garantit pas la livraison, l'ordre ou l'intégrité ; c'est à la couche transport (TCP) ou à l'application de gérer la fiabilité.
    - *Adressage logique* : Utilise des adresses IPv4 (32 bits) ou IPv6 (128 bits) pour identifier source et destination.
    - *Routage* : Les routeurs déterminent le chemin optimal d'un réseau à l'autre, en fonction de tables et protocoles (OSPF, BGP…).
    - *Fragmentation / Réassemblage* : Si un paquet est trop grand pour un lien, IP le découpe en fragments puis le réassemble chez le destinataire.

Annexe C. Les langages de programmation

- ***FBD :***

  - **Présentation** : Function Block Diagram



Le FBD est un langage particulier des automates programmables qui a la particularité d'être un langage graphique constitué de blocs de fonction reliant les entrées aux sorties faisant passer des variables prenant ses origines dans les années 50.

  - **Caractéristiques principales** :
    - *Modularité et réutilisation* : Chaque bloc réalise une fonction précise (arithmétique, logique, temporisation, PID, etc.) et peut être stocké dans une bibliothèque pour être réutilisé dans plusieurs programmes ou projets, ce qui améliore la maintenabilité.
    - *Flux de données explicite* : Les liaisons entre blocs montrent clairement la provenance et la destination de chaque signal : on suit le chemin des données de l'entrée jusqu'à la sortie, ce qui simplifie la lecture et le débogage.
    - *Typage fort et vérification à la conception* : Les blocs et leurs connexions sont typés (BOOL, INT, REAL, etc.), et l'outil de développement vérifie dès la phase d'édition que les types sont compatibles, réduisant ainsi les erreurs de configuration.
    - *Blocs standard et blocs utilisateurs*
      - *Blocs standards* : fournis par l'éditeur (AND, OR, timers, compteurs, fonctions mathématiques, PID, etc.)
      - *Blocs utilisateurs* : définis par l'utilisateur (ou créés en ST/IL), permettant d'encapsuler des séquences personnalisées et de respecter les principes de la programmation structurée.
    - *Intégration avec les autres langages IEC 61131-3* :On peut imbriquer des blocs écrits en Structured Text (ST) ou Instruction List (IL) dans un FBD, et inversement, ce qui facilite l'utilisation conjointe du meilleur de chaque paradigme.
    - *Adapté aux traitements temps réel et asynchrones* :FBD gère naturellement les opérations parallèles (blocs indépendants évalués simultanément dans un même cycle), ce qui est adapté aux architectures d'E/S distribuées et aux processus industriels dynamiques.

- ***JavaScript / TypeScript***

  - **Présentation :**

Le javascript est un langage haut niveau né en 1995. Haut niveau, car il ne dialogue pas avec le processeur directement mais a besoin d'un interpréteur pour fonctionner. C'est un langage propre aux applications web qui a su évoluer avec le temps depuis sa normalisation en 1997 avec ECMAscript. Le javascript a le plus large écosystème grâce au gestionnaire npm. A la base il servait à rendre le frontend réactif mais à la naissance de node.js celui-ci a permis de créer aussi des serveurs backends extrêmement souple permettant le dialogue avec des bases de données et le frontend. Le langage est orienté objet.

Le typescript est un langage libre créé en 2012. Grosso modo, il est une surcouche du javascript permettant de typer le langage afin de le rendre plus sûr. Celui-ci n'est pas utilisé en finalité mais sera transcompilé en javascript interprétable.

  - **Caractéristiques principales** :

    - ***Javascript*** :
      - *Prototype-based et orienté objet* : L'héritage se fait par prototypes plutôt que par classes (même si `class` est désormais disponible en ES6).
      - *Écosystème riche et universel* : Fonctionne côté client (navigateur) et côté serveur (Node.js), avec un vaste registre de paquets (npm).
      - *Asynchrone par nature* : Callbacks, Promises et async/await gèrent la concurrence non bloquante, essentielle pour les I/O et les animations.
      - *Évolution rapide (ECMAScript)* : Des mises à jour régulières (ES6, ES7, …) apportent constamment de nouvelles fonctionnalités (modules, déstructuration, spread, etc.).

    - ***Typescript*** :
      - *Sur-ensemble typé de JavaScript* : Ajoute un système de types statiques optionnels (primitifs, interfaces, génériques…) tout en restant compatible JS.
      - *Vérification et sécurité à la compilation* : Détecte un grand nombre d'erreurs (types incompatibles, propriétés manquantes, etc.) avant l'exécution du code.
      - *Support avancé de l'IDE* : Grâce aux types, l'édition bénéficie d'autocomplétions, de refactoring fiable et de navigation dans le code.
      - *Transpilation en JavaScript standard* : Le code TS est compilé en JS ciblant différentes versions d'ECMAScript et environnements (navigateurs, Node).

- *Orienté projet et maintenabilité* : Conçu pour les bases de code de grande taille : architecture modulaire, code auto-documenté par les types, meilleures pratiques encouragées.

- ***C++ :***

  - **Présentation :**

Le C++ est un langage de programmation de bas niveau. Compilé celui-ci communique directement avec le processeur voir ici le microcontrôleur. Le langage est beaucoup utilisé dans l'IoTs car étant extrêmement léger il permet de créer des programmes pour les microcontrôleurs tel que l'on trouve sur des arduino, esp, …

  - **Caractéristiques principales :**
    - **Performance et contrôle bas-niveau**
      Accès direct à la mémoire et aux ressources système (pointeurs, allocation manuelle), permettant d'optimiser chaque cycle processeur pour des applications critiques.
    - **Paradigme multi-style**
      Supporte la programmation impérative, orientée objet, générique (templates) et même fonctionnelle, offrant une grande flexibilité pour choisir le meilleur modèle selon le besoin.
    - **Généricité et métaprogrammation par templates**
      Les templates permettent d'écrire du code hautement réutilisable et typé, avec la possibilité de réaliser des optimisations à la compilation (compile-time).
    - **Écosystème mature et compatibilité ascendante**
      Bibliothèque standard (STL) et nombreux frameworks industriels, plus compatibilité avec le C, ce qui est un gage de pérennité et d'interopérabilité.

Annexe D. Frameworks et librairies importantes

- ***Frontend :***

  - **React :** est une bibliothèque open source javascript d'interface utilisateur créé en 2013. Le but est de créer des interfaces monopages simples et rapides au chargement. Sa force réside par sa flexibilité et ses performances. Il a la particularité de bien s'intégrer à d'autres framework ou bibliothèque.
  - **Tanstack :** est un ensemble de libraires pour le routing (correspondance entre une URL et une requête http) les query, le store que l'on utilisera ici notamment. Il permet de faciliter et de sécuriser au sein de react
  - **Material UI :** libraire react pour créer rapidement des composants d'interface. Il permet d'avoir des éléments réactifs simples et efficaces (bouton, input, checkbox, formulaire…).
  - **Zod :** libraire typescript permettant la validation lors de formulaire. Il permet de définir des schémas à base d'objets et d'objet nesté (imbriqué)
  - **I18next :** framework permettant de facilité la traduction des mots d'une application. Su base sur un format json. En choisissant la langue celui-ci fait automatiquement le lien entre le dossier de traduction et l'application. Ce qui permet d'avoir une application multilingue.
  - **Argon2 :** celui-ci est une fonction de dérivation permettant de créer des clés hautement sécurisées. Ici il est décliné en une petite libraire facile d'utilisation sans devoir réécrire toute la fonction.

- ***Backend :***

  - **Fastify :** est un framework pour node.js, c'est un serveur avec une infrastructure légère, rapide et extrêmement efficace sous forte charge. Permet de traite de grosse quantité de requêtes avec une bonne gestion des ressources.

  - **TypeOrm :** framework permettant la gestion et la communication direct avec la base de données. Couplé avec Fastify, il permet une communication fine, rapide et puissance lors des requêtes de forte charge. Il a la possibilité de la gestion de la base de données en créant celle-ci mais aussi en la modifiant via des migrations.

  - **Mqtt.js :** est une librairie open source communautaire liée au projet avec des objets connectés (IoTs) la libraire permet la communication entre le backend et ces derniers via le protocole du même nom.

- ***Base de données :***

    - **PostgreSQL:** système de gestion de base de données relationnel orienté objet open source (SGBDRO). Il prend en charge les transactions avec des propriétés ACID (Atomicité, Cohérence, isolation, durabilité)

- ***La Conteneurisation (Docker) :***

Technologie d'empaquetage et de déploiement d'applications qui regroupe une application avec ses dépendances (bibliothèques, binaires, fichiers de configuration) dans une unité légère et autonome appelée conteneur. Cette méthode assure que l'application s'exécute de manière cohérente sur n'importe quelle infrastructure, offrant ainsi portabilité, évolutivité et efficacité. Les conteneurs partagent le noyau du système d'exploitation hôte et s'exécutent dans leur propre espace utilisateur isolé, ce qui permet d'exécuter plusieurs conteneurs sur un seul hôte sans conflits. Des plateformes comme Docker et des outils d'orchestration comme Kubernetes sont utilisés pour créer et gérer ces conteneurs.

Annexe E. Logiciels utilisés lors de la conception

Automate :
Siemens softcomfort
https://www.siemens.com/global/en/products/automation/systems/industrial/plc/logo/logo-software.html

ESP et Arduino :
Arduino Ide
https://www.arduino.cc/en/software/
MqttExplorer
https://mqtt-explorer.com/

Application nodeJs et frontend

Visual studio code
https://code.visualstudio.com/

Docker
https://www.docker.com/

Schneider :
Modbus Poll
https://www.modbustools.com/mbpoll-user-manual.html

WaveShare
virCom
https://www.waveshare.com/wiki/RS232_RS485_TO_POE_ETH_(B)#Vircom_Test

Annexe F. Étape de création du backend

1. Initialisation du projet
- Créer le dossier du projet et l'initialiser avec `pnpm init -y`.
- Installer Fastify et les dépendances de base.
- Lien : https://www.fastify.io/

2. Configuration de TypeScript
- Créer un fichier `tsconfig.json`.
- Définir les options de compilation.
- Lien : https://www.typescriptlang.org/

3. Structure du projet
- Organiser le projet en dossiers : `src/`, `routes/`, `services/`, `entities/`, `database/`.
- Créer un point d'entrée `server.ts`.

4. Configuration des variables d'environnement
- Utiliser `dotenv` pour gérer `.env` avec des paramètres comme `PORT`, `DATABASE_URL`, `MQTT_BROKER_URL`.
- Lien : https://www.npmjs.com/package/dotenv

5. Installation des packages essentiels
- Installer `fastify`, `@fastify/jwt`, `@fastify/auth`, `casl`, `dotenv`, `cron-schedule`, `date-fns`, `yup`, `reflect-metadata`.
- Lien : https://www.npmjs.com/package/fastify

6. Gestion de la base de données
- Configurer PostgreSQL avec `docker-compose.yml`.
- Utiliser TypeORM pour gérer les entités et migrations.
- Lien : https://typeorm.io/

7. Mise en place de l'authentification et des permissions
- Ajouter `@fastify/jwt` pour la gestion des tokens JWT.
- Implémenter `casl` pour la gestion des rôles et permissions.
- Lien : https://casl.js.org/

8. Intégration MQTT
- Installer `mqtt` pour la communication avec Mosquitto.
- Configurer un client MQTT dans un fichier dédié.
- Lien : https://www.npmjs.com/package/mqtt

9. Organisation des routes
- Créer des fichiers de routes dans `routes/` et les enregistrer dans `server.ts`.
- Lien : https://www.npmjs.com/package/fastify-plugin

10. Gestion des tâches planifiées
- Utiliser `cron-schedule` pour exécuter des tâches automatiques.
- Lien : https://www.npmjs.com/package/cron-schedule

11. Sécurisation du backend
- Ajouter `@fastify/cors` et `@fastify/helmet` pour la sécurité.
- Lien : https://www.npmjs.com/package/@fastify/helmet

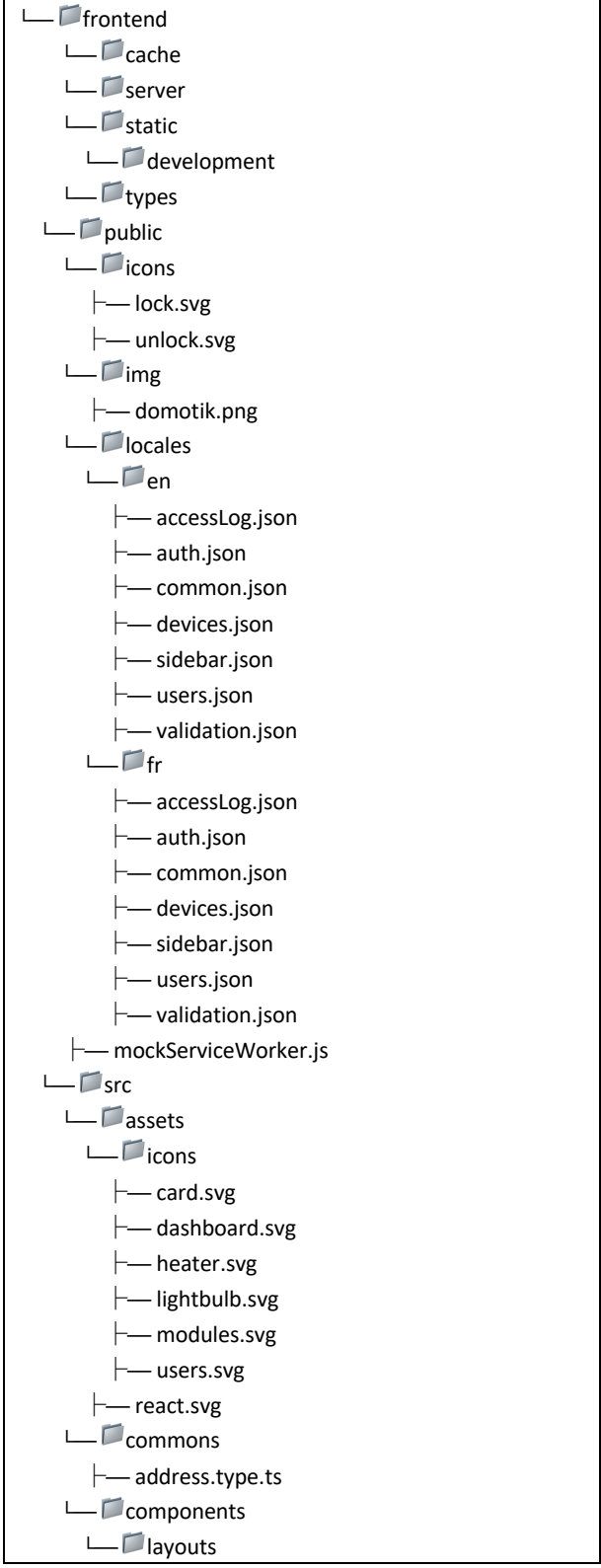
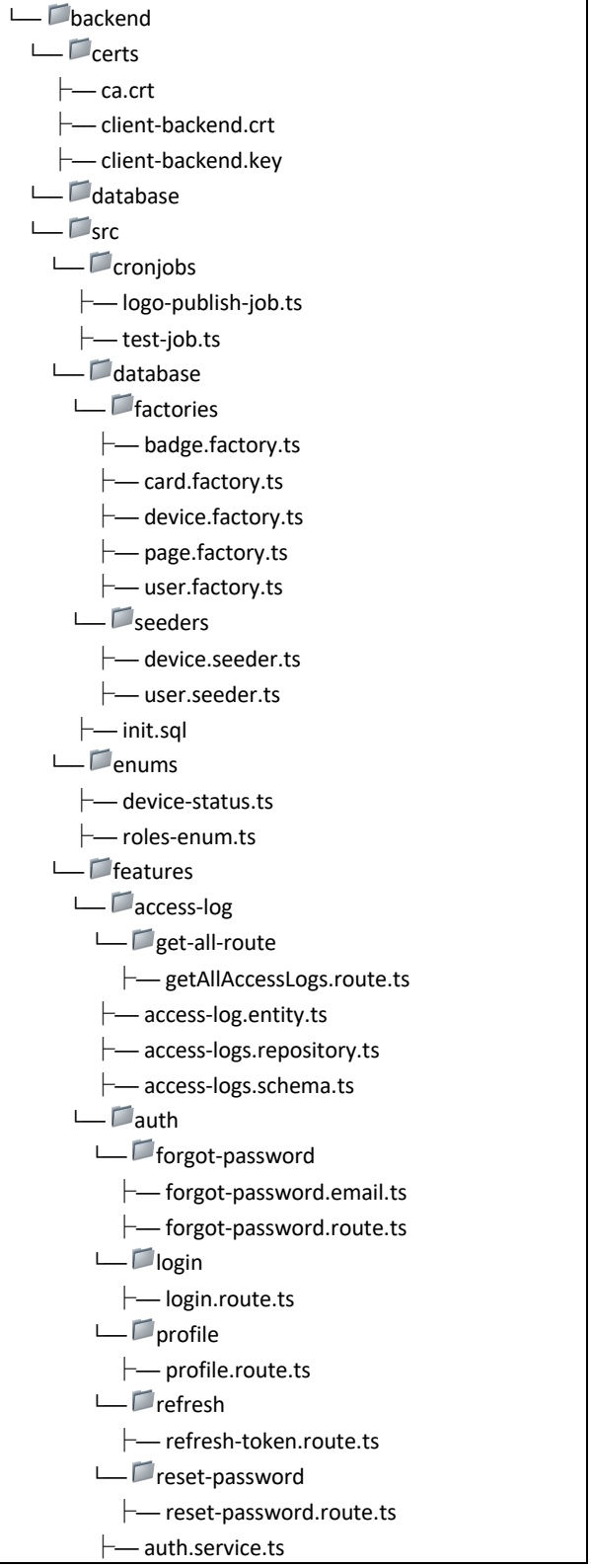
12. Gestion des erreurs et standardisation des réponses
- Ajouter un gestionnaire global d'erreurs.
- Standardiser les réponses API avec des utilitaires.

13. Création des entités et des migrations
- Définir les entités avec TypeORM.
- Générer et appliquer les migrations avec TypeORM CLI.

14. Mise en place des seeders
- Utiliser `@jorgebodega/typeorm-seeding` pour insérer des données de test.
- Lien : https://www.npmjs.com/package/@jorgebodega/typeorm-seeding

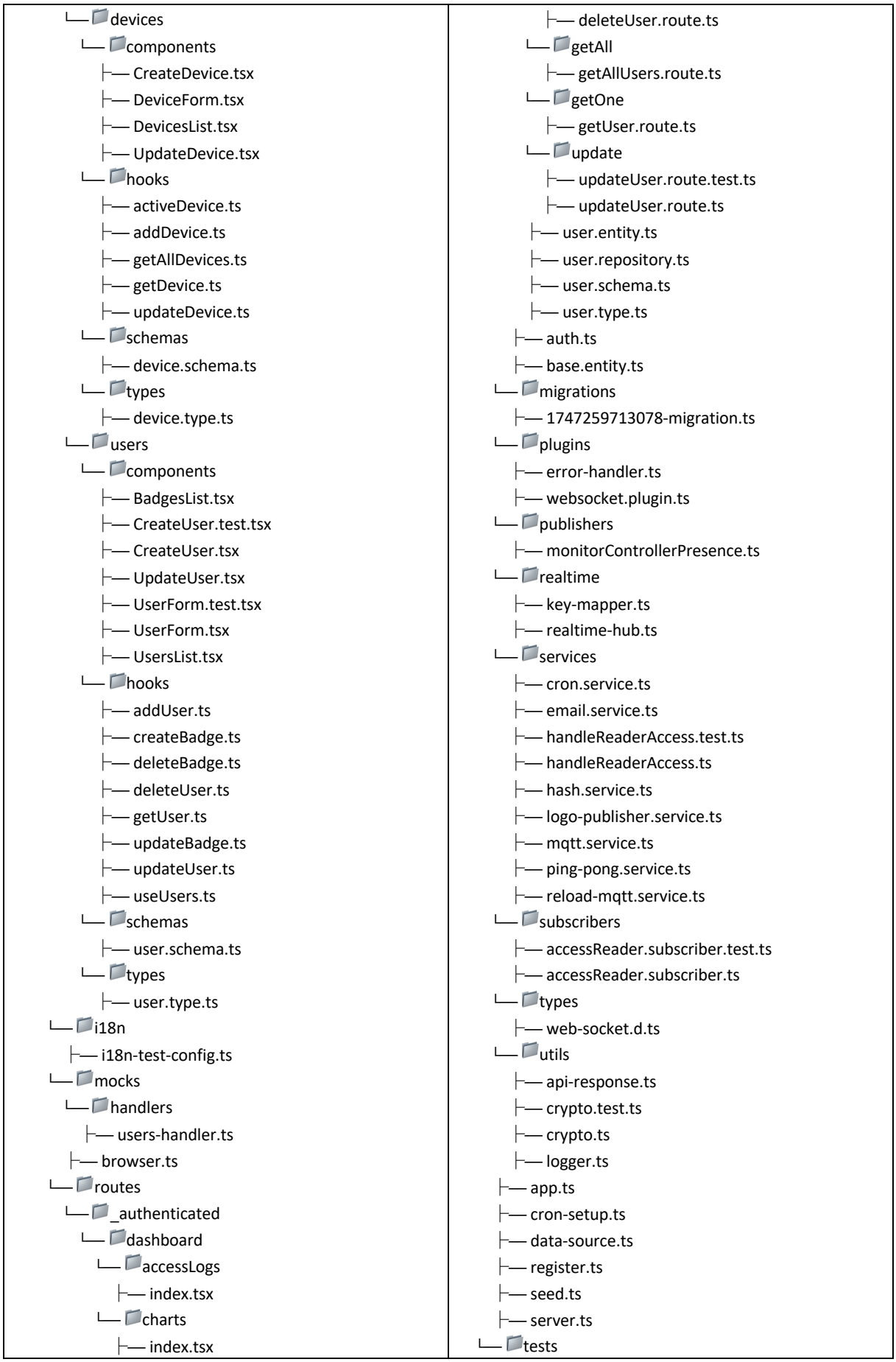
15. Ajout des tests unitaires et d'intégration
- Utiliser `vitest` et `supertest` pour tester les services et les routes.
- Lien : https://vitest.dev/

Annexe G. Schémas Backend/Frontend

Frontend	Backend
	

```
        ├── DashboardLayout.tsx
    ├── HeaderActions.tsx
    ├── InputSearch.tsx
    ├── LanguageSwitcher.tsx
    ├── Meta.tsx
    ├── SideBar.tsx
└── enums
    ├── device-status.ts
    ├── roles-enum.ts
└── features
    └── accessLogs
        └── components
            ├── AccessLogsList.tsx
        └── hooks
            ├── getAllAccessLogs.ts
        └── types
            ├── access-logs.types.ts
    └── auth
        └── components
            ├── AuthLayout.tsx
            ├── ForgotPasswordForm.test.tsx
            ├── ForgotPasswordForm.tsx
            ├── LoginForm.test.tsx
            ├── LoginForm.tsx
            ├── Logout.tsx
            ├── ResetPasswordForm.test.tsx
            ├── ResetPasswordForm.tsx
        └── hooks
            ├── index.ts
            ├── useAuth.ts
            ├── useAuthRefresh.ts
            ├── useAuthStore.ts
            ├── useAuthToken.ts
            ├── useWs.ts
        └── mocks
            ├── handlers.ts
            ├── index.ts
        └── types
            ├── auth.types.ts
            ├── index.ts
    └── charts
        └── components
            ├── Charts.tsx
            ├── DynamicNavbar.tsx
            ├── DynamicPage.tsx
        └── hooks
            ├── getAllPagesName.ts
            ├── getPage.ts
            ├── useLiveKeyValues.ts
        └── schemas
            ├── page.schema.ts
```

```
    └── badges
        └── add-route
            ├── badge.add.route.test.ts
            ├── badges.add.route.ts
        └── delete-route
            ├── badges.delete.route.ts
        └── update-route
            ├── badges.update.route.ts
        ├── badge.service.ts
        ├── badges.entity.ts
        ├── badges.repository.ts
        ├── deriveBadgeKey.utils.test.ts
        ├── deriveBadgeKey.utils.ts
        ├── deriveKeysAB.utils.test.ts
        ├── deriveKeysAB.utils.ts
        ├── validateBadge.ts
    └── charts
        └── getAllPages
            ├── getAllPages.route.ts
        └── getOnePage
            ├── getOnePage.route.ts
        └── ws
            ├── charts.ws.ts
        ├── cards.entity.ts
        ├── page.entity.ts
        ├── page.repository.ts
        ├── page.schema.ts
    └── data-histories
        ├── data-histories.repository.ts
        ├── data-histories.route.ts
        ├── data-history.entity.ts
    └── devices
        └── active
            ├── ActiveDevice.route.test.ts
            ├── ActiveDevice.route.ts
        └── add
            ├── addDevice.route.ts
        └── getAll
            ├── getAllDevices.route.ts
        └── getOne
            ├── getDevice.route.ts
        └── update
            ├── updateDevice.route.ts
        ├── device.entity.ts
        ├── device.repository.ts
        ├── device.schema.ts
    └── users
        └── add
            ├── addUser.route.test.ts
            ├── addUser.route.ts
        └── delete
```


devices
components
CreateDevice.tsx
DeviceForm.tsx
DevicesList.tsx
UpdateDevice.tsx
hooks
activeDevice.ts
addDevice.ts
getAllDevices.ts
getDevice.ts
updateDevice.ts
schemas
device.schema.ts
types
device.type.ts
users
components
BadgesList.tsx
CreateUser.test.tsx
CreateUser.tsx
UpdateUser.tsx
UserForm.test.tsx
UserForm.tsx
UsersList.tsx
hooks
addUser.ts
createBadge.ts
deleteBadge.ts
deleteUser.ts
getUser.ts
updateBadge.ts
updateUser.ts
useUsers.ts
schemas
user.schema.ts
types
user.type.ts
i18n
i18n-test-config.ts
mocks
handlers
users-handler.ts
browser.ts
routes
_authenticated
dashboard
accessLogs
index.tsx
charts
index.tsx
deleteUser.route.ts
getAll
getAllUsers.route.ts
getOne
getUser.route.ts
update
updateUser.route.test.ts
updateUser.route.ts
user.entity.ts
user.repository.ts
user.schema.ts
user.type.ts
auth.ts
base.entity.ts
migrations
1747259713078-migration.ts
plugins
error-handler.ts
websocket.plugin.ts
publishers
monitorControllerPresence.ts
realtime
key-mapper.ts
realtime-hub.ts
services
cron.service.ts
email.service.ts
handleReaderAccess.test.ts
handleReaderAccess.ts
hash.service.ts
logo-publisher.service.ts
mqtt.service.ts
ping-pong.service.ts
reload-mqtt.service.ts
subscribers
accessReader.subscriber.test.ts
accessReader.subscriber.ts
types
web-socket.d.ts
utils
api-response.ts
crypto.test.ts
crypto.ts
logger.ts
app.ts
cron-setup.ts
data-source.ts
register.ts
seed.ts
server.ts
tests

```
        └── devices
            ├── $id.tsx
            ├── create.tsx
            └── index.tsx
        └── users
            ├── $id.tsx
            ├── create.tsx
            └── index.tsx
        ├── accessLogs.tsx
        ├── charts.tsx
        ├── devices.tsx
        ├── index.tsx
        ├── lightings.tsx
        └── users.tsx
    ├── dashboard.tsx
  └── auth
    ├── __auth.tsx
    ├── forgot-password.tsx
    ├── login.tsx
    ├── logout.tsx
    └── reset-password.tsx
  ├── __root.tsx
  ├── _authenticated.tsx
  ├── index.tsx
└── styles
  ├── App.css
  └── button.css
└── themes
  └── blackDashboardTheme.ts
└── utils
  ├── fetchUtil.ts
  ├── useDebounce.ts
  ├── wrapQueryResult.tsx
  └── ws.ts
 ├── App.tsx
 ├── config.ts
 ├── i18n.ts
 ├── main.tsx
 ├── routeTree.gen.ts
 ├── sw.ts
 ├── TanstackRouterAppProvider.tsx
 └── vite-env.d.ts
└── tests
 ├── dummy.test.tsx
 ├── sleep.ts
 ├── test-extend.ts
 ├── test-worker.ts
 └── vitest.setup.ts
├── .dockerignore
├── .env
└── .gitignore
```

```
    └── integrations
      └── __snapshots__
        ├── devices-integration.test.ts.snap
        └── user-integrations.test.ts.snap
      ├── devices-integration.test.ts
      └── user-integrations.test.ts
    └── units
      └── user.test.ts
  ├── .dockerignore
  ├── .eslintignore
  ├── .eslintrc.json
  ├── .gitignore
  ├── development.env
  ├── docker-compose.yml
  ├── Dockerfile
  ├── package.json
  ├── pnpm-lock.yaml
  ├── pnpm-workspace.yaml
  ├── rollup.config.js
  ├── test.env
  ├── tsconfig.json
  ├── vite.config.ts
  └── vitest.config.ts
```

├── .prettierignore ├── .prettierrc.json ├── Dockerfile ├── eslint.config.js ├── index.html ├── nginx.conf ├── package.json ├── playwright.config.ts ├── pnpm-lock.yaml ├── pnpm-workspace.yaml ├── pwa-assets.config.ts ├── README.md ├── tailwind.config.js ├── tsconfig.app.json ├── tsconfig.json ├── tsconfig.node.json ├── vite.config.ts └── vitest.config.mts	

Annexe H. Littérature

- ***MQTT :***

https://fr.wikipedia.org/wiki/MQTT
https://fr.wikipedia.org/wiki/Organization_for_the_Advancement_of_Structured_Information_Standards
https://mqtt.org/

- *Modbus :*

https://modbus.org

- ***http :***

https://fr.wikipedia.org/wiki/Hypertext_Transfer_Protocol
https://www.w3.org/Protocols/

- **REST**

https://fr.wikipedia.org/wiki/Representational_state_transfer

- **uart :**

https://fr.wikipedia.org/wiki/UART

- ***tcp/ip***

https://datatracker.ietf.org/doc/html/rfc791
https://datatracker.ietf.org/doc/html/rfc793
https://fr.wikipedia.org/wiki/Transmission_Control_Protocol

- ***FSB***

https://en.wikipedia.org/wiki/Function_block_diagram

- ***Javascript***

https://fr.wikipedia.org/wiki/JavaScript
https://fr.wikipedia.org/wiki/Node.js
https://nodejs.org/en
https://fr.wikipedia.org/wiki/TypeScript
https://www.typescriptlang.org/
https://ecma-international.org/

- ***CPP***

https://isocpp.org/

- ***HTML***

https://html.spec.whatwg.org/

- ***Frontend***

https://react.dev/
https://fr.wikipedia.org/wiki/React
https://tanstack.com/
https://mui.com/material-ui/
https://zod.dev/
https://www.i18next.com/

- ***Backend***

https://fastify.dev/
https://fr.wikipedia.org/wiki/Argon2
https://github.com/ranisalt/node-argon2
https://github.com/mqttjs
https://typeorm.io/

- ***DB***

https://www.postgresql.org/

- ***Git et github***

https://fr.wikipedia.org/wiki/Git
https://github.com/

Annexe I. Coût total du matériel après développement

Appareil	Qtt	Prix	Fournisseur
Arduino nano	3	15,11 €	Amazon
Raspberry Pi 4go	1	75,00 €	McHobby
AZ-Delivery ESP32	3	30,24 €	Amazon
Elechouse module PN532	2	9,29 €	Amazon
Waveshare Industrial USB to RS485 Bidirectional Converter	1	16,02 €	Amazon
Waveshare 2-Ch RS485 to RJ45 Ethernet Serial Serveur, Dual channels	1	49,57 €	Amazon
Licence Siemens soft comfort	1	53,52 €	Rexel
Siemens Logo! 8.4.1 + alimentation	1	192,54 €	Rexel
Schneider acti9 IEM2055	1	152,36 €	Rexel
MeanWel Alimentation rail DIN HDR-15-5 5 V/DC 2.4 A 12 W	1	14,11 €	Amazon
MeanWell Alimentation rail DIN HDR-15-12 V/DC 1.25A 5W	1	11,70 €	Amazon
Câble USB C Male	5	8,05 €	Amazon
Capteur de température DHT22	5	16,12 €	Amazon
Disjoncteur Schneider 2 pôles 6A	1	13,59 €	Brico plan-it
Coffert Kopp 1*18 modules	1	31,99 €	Brico plan-it
Prise 2X 16A	1	44,99 €	Brico plan-it
	Total	734,20 €	

Annexe J. Sketch arduino

A – Simulation température.

```
/**
 * @file DS3132-DHT22-V2.ino
 * @version 2.0
 * @author Patrick Pagnoulle
 * @date 2023-10-14
 * @brief Programme permettant de lire la température et l'humidité relative à
 * l'aide d'une sonde DHT22 envoyer en MQTT
 */
#include <WiFi.h>
#include <PubSubClient.h>
#include <Adafruit_Sensor.h>
#include <DHT.h>
#include <Arduino_JSON.h>

// Structure de données
struct SensorData {
  float temperature;
  float humidity;
  int errorCode; // 0 pour succès, autre valeur pour erreur
};
struct ingoingData {
  float temperature;
  float humidity;
};

// Paramètres WiFi
const char* ssid = "**********";
const char* password = "**********";

// Définition des pins
#define DHT_PIN 13 // Pin de la sonde DHT22
#define DHT_TYPE DHT22

// Définition des variables
DHT dht(DHT_PIN, DHT_TYPE);

const int arraySize = 36;
int i = 0;
ingoingData* dataArray;

// Paramètres MQTT
const char* mqtt_server = "*********";
const int mqtt_port = 1883;
const char* mqtt_user = "**********";
const char* mqtt_password = "*******";
const char* topic = "unifylots/sensor/custom-sensor-dth-01/get";
```

```
// Création des objets WiFi et MQTT
WiFiClient espClient;
PubSubClient client(espClient);

void setup() {
  Serial.begin(115200);
  // Génération des données et stockage dans dataArray
  dataArray = generateData(arraySize);
  Serial.println("---------- Données générées : ----------");
  displayData(dataArray);
  Serial.println("------------------------------------------------------------");
  setup_wifi();
  client.setServer(mqtt_server, mqtt_port);
  delay(300);
  Serial.println("\n\n Lecture de l'humidité et de la température \n\n ");
  delay(2700);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();

  // Publier un message toutes les 5 secondes
  static unsigned long lastMsg = 0;
  if (millis() - lastMsg > 5000) {
    if(i==36){
     i=0;
    }
    lastMsg = millis();
    SensorData sensorData;
    sensorData = CheckSensor(dataArray[i]);
    String jsonString;
    JSONVar payload;
    JSONVar doc;
    doc["brand"] = "custom";
    doc["model"] = "sensor";
    doc["type"] = "dth";
    if (sensorData.errorCode == 0) {
      payload["temperature"] = String(sensorData.temperature, 2);
      payload["humidity"] = String(sensorData.humidity, 2);
      // Ajouter le sous-objet à l'objet principal
      doc["payload"] = payload;
      // Convertir l'objet JSON en chaîne de caractères
      jsonString = JSON.stringify(doc);

      Serial.print("Envoi du message: ");
      Serial.println(jsonString);
    } else if (sensorData.errorCode == 1) {
      doc["Error"] = String(sensorData.errorCode);
```

```
        doc["payload"] = payload;
        jsonString = JSON.stringify(doc);
        Serial.print(jsonString);
      } else {
        Serial.println("Erreur inconnue");
      }
      client.publish(topic, jsonString.c_str());
      i++;
    }
}

// Fonction de connexion au WiFi
void setup_wifi() {
  delay(10);
  Serial.println("Connexion au WiFi...");
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi connecté");
  Serial.print("Adresse IP: ");
  Serial.println(WiFi.localIP());
}

// Fonction de connexion au broker MQTT
void reconnect() {
  while (!client.connected()) {
    Serial.println("Connexion au broker MQTT...");
    if (client.connect("ESP32Client", mqtt_user, mqtt_password)) {
      Serial.println("Connecté au broker MQTT");
    } else {
      Serial.print("Échec, rc=");
      Serial.print(client.state());
      Serial.println(" Nouvelle tentative dans 5 secondes...");
      delay(2000);
    }
  }
}
```

```
/**
 * @brief
 * Fonction permettant de vérifier le bon fonctionnement de la sonde et d'afficher
 * les données si tout est ok sinon renvoie un message d'erreur
 * @return SensorData Structure contenant la température, l'humidité et un code d'erreur
 */
SensorData CheckSensor(ingoingData d) {
   SensorData data;
   data.temperature = d.temperature;
   data.humidity = d.humidity;

   if (isnan(data.humidity) || isnan(data.temperature)) {
      Serial.println("Erreur lors de la lecture de la sonde DHT22");
      data.errorCode = 1;
   } else {
      Serial.println("Température: ");
      Serial.println(String(data.temperature, 2));
      Serial.println("Humidité: ");
      Serial.println(String(data.humidity, 2));
      data.errorCode = 0;
   }
   return data;
}

ingoingData* generateData(int arraySize) {
 static ingoingData data[36]; // Utilisation d'un tableau statique
 int index = 0;

 int ascendElements = (arraySize - 12) / 2;  // Exclure les cycles fixes et les éléments vides
 int descendElements = ascendElements;
 float tempIncrement = (22.99 - 15.01) / (ascendElements - 1);  // Pas entre chaque valeur

 // 2 occurrences de 15.00°C au début
 for (int i = 0; i < 2 && index < arraySize; i++) {
  data[index].temperature = 15.00;
  data[index].humidity = random(10, 51);
  index++;
 }

 // Températures montantes
 float temp = 15.01;
 for (int i = 0; i < ascendElements && index < arraySize; i++) {
  data[index].temperature = temp;
  data[index].humidity = random(10, 51);
  temp += tempIncrement;
  index++;
 }

 // 4 occurrences de 23.00°C au sommet
 for (int i = 0; i < 4 && index < arraySize; i++) {
  data[index].temperature = 23.00;
```

```
  data[index].humidity = random(10, 51);
  index++;
 }

 // 4 cycles vides
 for (int i = 0; i < 4 && index < arraySize; i++) {
  data[index].temperature = NAN;  // Valeur par défaut
  data[index].humidity = NAN;
  index++;
 }

 // Températures descendantes
 temp = 22.99;
 for (int i = 0; i < descendElements && index < arraySize; i++) {
  data[index].temperature = temp;
  data[index].humidity = random(10, 51);
  temp -= tempIncrement;
  index++;
 }

 // 2 occurrences de 15.00°C à la fin
 for (int i = 0; i < 2 && index < arraySize; i++) {
  data[index].temperature = 15.00;
  data[index].humidity = random(10, 51);
  index++;
 }

 // Vérification de la taille du tableau
 if (index != arraySize) {
  Serial.println("Erreur : Le tableau généré n'a pas la bonne taille.");
 }

 return data;
}


void displayData(ingoingData* data) {
  for (int i = 0; i < arraySize; i++) {
    if (data[i].temperature == NAN && data[i].humidity == NAN) {
      Serial.print("Élément ");
      Serial.print(i);
      Serial.println(": Cycle vide - Données non disponibles");
    } else {
      Serial.print("Élément ");
      Serial.print(i);
      Serial.print(": Température = ");
      Serial.print(data[i].temperature);
      Serial.print("°C, Humidité = ");
      Serial.print(data[i].humidity);
      Serial.println("%");
    }
```

```
  }
}
```

B – Lecteur de badges

```
/**
 * @file CARDCODER07ISACTIVE.ino
 * @version 5.0
 * @author Patrick Pagnoulle
 * @date 2023-10-14
 *
 * @brief
 * Programme permettant de lire un badge Mifare 1k via un module NFC (PN532),
 * de communiquer avec un serveur distant via MQTT sur WiFi, et de gérer l'accès
 * selon la réponse du backend. Le système fonctionne sur une badge compatible Arduino.
 *
 * Schéma de fonctionnement :
 * 1. Initialisation des modules (NFC, WiFi, MQTT, série).
 * 2. Attente de présentation d'une badge NFC.
 * 3. Lecture de l'UID et envoi au backend pour obtenir la clé A.
 * 4. Vérification de la clé A, lecture de données, envoi au backend.
 * 5. Attente de la réponse d'accès (granted/denied) et retour d'état via série.
 * 6. Gestion des reconnexions WiFi/MQTT et surveillance du module NFC.
 *
 */

#include <Arduino.h>
#include <SPI.h>
#include <Adafruit_PN532.h>
#include <PN532Helper.h>
#include <WiFi.h>
#include <ArduinoJson.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <HardwareSerial.h>
#include "ca.h"
#include "client_reader_crt.h"
#include "client_reader_key.h"


/**
 * @brief Hardware serial port used for debugging or communication with an external device.
 */
HardwareSerial SerialCom(1);


/**
 * @brief Secure WiFi client (TLS) for network communication.
 */
WiFiClientSecure secureClient;
```

```
/**
 * @brief MQTT client for communication with the broker.
 */
PubSubClient client(secureClient);


/**
 * @brief WiFi network credentials.
 */
const char* ssid = "**********";
const char* password = "**********";

/**
 * @brief MQTT connection parameters (server, port, credentials, topics).
 */
const char* mqtt_server   = "**********";
const int   mqtt_port     = 8883;
const char* mqtt_user     = "**********";
const char* mqtt_password = "**********";
const char* topic_pub     = "unifylots/controller/reader-nfc-01/get";
const char* topic_sub     = "unifylots/controller/reader-nfc-01/set";


/**
 * @brief PN532 (NFC) module pinout.
 */
#define SS_PIN   2
#define SCK_PIN  18
#define MISO_PIN 19
#define MOSI_PIN 23


/**
 * @brief Unique reader seed for identification.
 */
const char* reader_seed = "1dea7ca1d7c819bb12cce6dc7c51d56c";


/**
 * @brief Main state machine states for the reader.
 */
enum State {
  WAIT_CARD = 0,   ///< Waiting for an NFC card
  WAIT_KEYA = 1,   ///< Waiting for Key A from backend
  WAIT_ACCESS = 2  ///< Waiting for access decision
};
/**
 * @brief Current state of the state machine.
 */
State step = WAIT_CARD;

ul removalTime = 0;
```

```
typedef unsigned long ul; ///< Alias for unsigned long
/**
 * @brief Indicates if a card was present during the last cycle.
 */
bool cardWasPresent = false;
/**
 * @brief Indicates if the system is waiting for card removal.
 */
bool waitingRemoval = false;
/**
 * @brief Timer for delayed serial communication.
 */
unsigned long serialComTimer = 0;
/**
 * @brief Indicates if a delayed serial communication is pending.
 */
bool serialComPending = false;
/**
 * @brief Timestamp for the start of card removal waiting period.
 */
ul removalTime = 0;


/**
 * @brief UID of the card read (string format).
 */
String uidStr;
/**
 * @brief UID of the card read (raw bytes).
 */
uint8_t uid[7];
/**
 * @brief Length of the UID read.
 */
uint8_t uidLength;


/**
 * @brief Helper object for managing the PN532 (NFC) module.
 */
PN532Helper nfcHelper(SS_PIN, SCK_PIN, MISO_PIN, MOSI_PIN);


// === Forward declarations ===

/**
 * @brief Initial connection to the WiFi network (blocking).
 */
void connectToWiFi();

/**
```

```
 * @brief Initial connection to the MQTT server (blocking).
 */
void connectToMQTT();

/**
 * @brief Non-blocking monitoring and reconnection for WiFi/MQTT.
 */
void spyConnections();


/**
 * @brief Publishes a JSON payload to the MQTT publish topic.
 * @param payload JSON string to send.
 */
void sendPayload(const String& payload) {
  client.publish(topic_pub, payload.c_str());
}


/**
 * @brief Requests Key A from the backend for the detected card.
 */
void requestKeyA() {
  String payload = String("{\"step\":\"") + String(WAIT_CARD) + "\","
                   "\"seed\":\"" + reader_seed + "\","
                   "\"uid\":\"" + uidStr + "\"}";
  sendPayload(payload);
  step = WAIT_KEYA;
}


/**
 * @brief Triggers a delayed serial communication (2s).
 */
void triggerSerialComDelayed() {
  serialComTimer = millis();
  serialComPending = true;
}


/**
 * @brief MQTT callback: handles reception of messages (KeyA, access).
 * @param topic Received MQTT topic
 * @param payloadBytes Message data
 * @param length Message length
 */
void callback(char* topic, byte* payloadBytes, unsigned int length) {
  String msg;
  msg.reserve(length);
  for (unsigned int i = 0; i < length; i++) msg += (char)payloadBytes[i];

  StaticJsonDocument<256> doc;
```

```
 if (deserializeJson(doc, msg)) return;

 String access = doc["access"];
 if (doc.containsKey("access") && access == "denied"){
    SerialCom.write(7);
    step = WAIT_CARD;
    waitingRemoval = false;
    triggerSerialComDelayed();
    return;
  }
 if (step == WAIT_KEYA && doc.containsKey("keyA")) {
  const char* keyA = doc["keyA"];

  if (nfcHelper.checkKeyA(uidStr, keyA, 4)) {
    String derived = nfcHelper.readData(uidStr, keyA, 4);
    String out = String("{\"step\":\"") + String(WAIT_KEYA) + "\","
                 "\"seed\":\"" + reader_seed + "\","
                 "\"derivedKey\":\"" + derived + "\","
                 "\"uid\":\"" + uidStr + "\"}";
    sendPayload(out);
    step = WAIT_ACCESS;
  } else {
    SerialCom.write(7);
    step = WAIT_CARD;
    waitingRemoval = false;
    triggerSerialComDelayed();
    waitingRemoval = false;
  }
  return;
 }
 if (step == WAIT_ACCESS && doc.containsKey("access")) {
  String access = doc["access"];
  if (access == "granted") {
    SerialCom.write(6);
  } else {
    SerialCom.write(7);
  }
  // reset state for next presentation
  step = WAIT_CARD;
  waitingRemoval = false;
  triggerSerialComDelayed();
  return;
 }
}


/**
 * @brief Initial WiFi connection (blocking, 10s timeout).
 */
void connectToWiFi() {
  WiFi.begin(ssid, password);
```

```
 ul start = millis();
 while (WiFi.status() != WL_CONNECTED && millis() - start < 10000) {
  delay(200);
 }
 SerialCom.write(WiFi.status() == WL_CONNECTED ? 0 : 2);
}


/**
 * @brief Initial connection to the MQTT server (blocking).
 */
void connectToMQTT() {
 secureClient.setCACert(ca_pem);
 secureClient.setCertificate(client_reader_crt_pem);
 secureClient.setPrivateKey(client_reader_key_pem);

 client.setServer(mqtt_server, mqtt_port);
 client.setCallback(callback);

 while (!client.connected()) {
  client.connect("arduinoClient", mqtt_user, mqtt_password);
  delay(200);
 }
 client.subscribe(topic_sub);
 SerialCom.write(8);
}


/**
 * @brief Non-blocking monitoring and reconnection for WiFi/MQTT.
 */
void spyConnections() {
 if (WiFi.status() != WL_CONNECTED) {
  WiFi.begin(ssid, password);
  SerialCom.write(2);
 }
 if (!client.connected()) {
  client.connect("arduinoClient", mqtt_user, mqtt_password);
  if (client.connected()) {
   client.subscribe(topic_sub);
   SerialCom.write(8);
   triggerSerialComDelayed();
  } else {
   SerialCom.write(10);
  }
 }

}


/**
 * @brief Checks the presence of the NFC module and notifies the backend if absent.
```

```
 */
void spyNFCModule() {
  static unsigned long lastCheck = 0;
  unsigned long now = millis();
  // Vérifie toutes les 5 secondes (modifiable)
  if (now - lastCheck > 5000) {
    lastCheck = now;
    if (!nfcHelper.isPresent()) {
      // Envoie un message d'erreur au backend
      sendPayload("{\"error\":\"nfc_module_absent\"}");
    }else{
      nfcHelper.begin();
    }
  }
}

/**
 * @brief Arduino initialization function (setup).
 */
void setup() {
  Serial.begin(115200);
  SerialCom.begin(115200, SERIAL_8N1, 16, 17);

  if (!nfcHelper.begin()) {
    SerialCom.write(4);
    while (true);
  }

  // Initial network setup
  connectToWiFi();
  connectToMQTT();
  delay(500);
  SerialCom.write(5);
}

/**
 * @brief Main Arduino loop function.
 */
void loop() {
  spyConnections();
  spyNFCModule();
  client.loop();

  if (serialComPending && millis() - serialComTimer >= 2000) {
    SerialCom.write(5);
    serialComPending = false;
  }

  bool present = nfcHelper.readCardUID(uidStr, uid, &uidLength, 100);
  Serial.println("----------");
  Serial.println("read present : " + String(present));
```

```
 // Si badge posée
 if (present && !cardWasPresent && !waitingRemoval && step == WAIT_CARD) {
   cardWasPresent = true;
   requestKeyA();
 }

 if (!present && cardWasPresent) {
   cardWasPresent = false;
   waitingRemoval = true;
   removalTime = millis();
 }

 if (waitingRemoval && (millis() - removalTime >= 2000)) {
   waitingRemoval = false;
 }
 delay(100);
}
```

C – Exemple de libraire pour le led

Fichier helper

```
#ifndef LEDHELPER_H
#define LEDHELPER_H

#include <Arduino.h>
#include <FastLED.h>
#include <avr/pgmspace.h>

#define LED_PIN    2
#define NUM_LEDS   1

typedef enum {
  COLOR_RED,
  COLOR_GREEN,
  COLOR_YELLOW,
  COLOR_BLUE,
  COLOR_ORANGE,
  COLOR_OFF
} LedColor;

typedef enum {
  BREATH_FAST,
  BREATH_SLOW
} BreathingMode;

class LedHelper {
public:
```

```
  LedHelper();

  void begin();
  void setPresetColor(LedColor color, uint8_t brightness = 64);
  void startBreathing(LedColor color, BreathingMode mode = BREATH_FAST);
  void updateLEDBreathing();
  inline void stopBreathing();

private:
  static const uint8_t _breatheTableFast[];
  static const uint8_t _breatheTableFastLength;
  static const uint8_t _breatheTableSlow[];
  static const uint8_t _breatheTableSlowLength;

  static const CRGB _presetColors[6];

  CRGB      _leds[NUM_LEDS];
  uint8_t   _fadeInterval;
  uint8_t   _breatheIndex;
  bool      _isBreathing;
  CRGB      _breathingBaseColor;
  const uint8_t* _currentBreatheTable;
  uint8_t       _currentBreatheTableLength;
  unsigned long _lastBreathUpdate;


};

#endif // LEDHELPER_H
```

Fichier cpp

```
#include "LedHelper.h"

// ------------------------------------------------------------------------
// 1) TABLE "RESPIRATION" (breatheTable) et sa longueur.
//    Valeurs 0→32→64→…→255→224→…→32 (16 étapes), pour un effet fade in/out.
//    Stockée en PROGMEM pour économiser la RAM.
// ------------------------------------------------------------------------

// Table de respiration rapide (16 valeurs)
const uint8_t LedHelper::_breatheTableFast[] PROGMEM = {
  0,  32,  64,  96, 128, 160, 192, 224,
 255, 224, 192, 160, 128,  96,  64,  32
};
const uint8_t LedHelper::_breatheTableFastLength =
   sizeof(LedHelper::_breatheTableFast) / sizeof(LedHelper::_breatheTableFast[0]);

// Table de respiration lente (32 valeurs)
const uint8_t LedHelper::_breatheTableSlow[] PROGMEM = {
 0, 16, 32, 48, 64, 80, 96, 112,
 128, 144, 160, 176, 192, 208, 224, 240,
 255, 240, 224, 208, 192, 176, 160, 144,
 128, 112, 96, 80, 64, 48, 32, 16
};
const uint8_t LedHelper::_breatheTableSlowLength =
   sizeof(LedHelper::_breatheTableSlow) / sizeof(LedHelper::_breatheTableSlow[0]);

const CRGB LedHelper::_presetColors[6] = {
 CRGB(0, 255, 0),   // Rouge
 CRGB(255, 0, 0),   // Vert
 CRGB(255, 255, 0),  // Jaune
 CRGB(0, 0, 255),   // Bleu
 CRGB(145, 255, 0),  // Orange
 CRGB(0, 0, 0)      // OFF
};

// ------------------------------------------------------------------------
// 2) Constructeur : initialise les membres par défaut.
// ------------------------------------------------------------------------
LedHelper::LedHelper()
 : _fadeInterval(20),           // 20 ms entre chaque étape de breathing
   _breatheIndex(0),
   _isBreathing(false),
   _breathingBaseColor(CRGB(0, 0, 0))
```

```
{
  // Rien de plus ici ; FastLED est configuré dans begin()
}

// ---------------------------------------------------------------------------
// 4) begin() : configure FastLED et éteint la LED.
// ---------------------------------------------------------------------------
void LedHelper::begin() {
  FastLED.addLeds<WS2812B, LED_PIN, GRB>(_leds, NUM_LEDS);
  _leds[0] = CRGB(0, 0, 0);
  FastLED.show();
}

// ---------------------------------------------------------------------------
// 5) setPresetColor() : allume une couleur fixe.
//    Si un breathing est en cours, on l'arrête.
// ---------------------------------------------------------------------------
void LedHelper::setPresetColor(LedColor color, uint8_t brightness) {
  stopBreathing();
  _leds[0] = _presetColors[color];
  FastLED.setBrightness((color == COLOR_OFF) ? 0 : brightness);
  FastLED.show();
}

// ---------------------------------------------------------------------------
// 6) startBreathing() : lance l'effet "breathing" sur la couleur indiquée.
//    - Si on est déjà en breathing, on se contente de changer la couleur de base.
//    - Sinon, on attache le Ticker pour appeler updateLEDBreathing() périodiquement.
// ---------------------------------------------------------------------------
void LedHelper::startBreathing(LedColor color, BreathingMode mode) {
  if (color == COLOR_OFF) {
    stopBreathing();
    return;
  }
  _breathingBaseColor = _presetColors[color];
  if (mode == BREATH_FAST) {
    _currentBreatheTable = _breatheTableFast;
    _currentBreatheTableLength = _breatheTableFastLength;
    _fadeInterval = 20;
  } else {
    _currentBreatheTable = _breatheTableSlow;
    _currentBreatheTableLength = _breatheTableSlowLength;
    _fadeInterval = 40;
  }
  if (_isBreathing) return;
  _breatheIndex = 0;
  _isBreathing = true;
  _leds[0] = CRGB(0, 0, 0);
  FastLED.setBrightness(255);
  FastLED.show();
  _lastBreathUpdate = millis();
```

```
}

// ---------------------------------------------------------------------------
// 7) stopBreathing() : arrête l'effet "breathing" et éteint la LED.
// ---------------------------------------------------------------------------
inline void LedHelper::stopBreathing() {
  _isBreathing = false;
  _leds[0] = CRGB(0, 0, 0);
  FastLED.setBrightness(0);
  FastLED.show();
}

// ---------------------------------------------------------------------------
// 8) updateLEDBreathing() : callback périodique (via Ticker) pour un pas de
//    breathing.
//    - Lit "brightness" dans _breatheTable (PROGMEM).
//    - Scale linéairement chaque composante de _breathingBaseColor par brightness/255.
//    - Incrémente _breatheIndex et boucle au besoin.
// ---------------------------------------------------------------------------
void LedHelper::updateLEDBreathing() {
  if (!_isBreathing) return;
  unsigned long now = millis();
  if (now - _lastBreathUpdate < _fadeInterval) return;
  _lastBreathUpdate = now;
  uint8_t brightness = pgm_read_byte(&_currentBreatheTable[_breatheIndex]);
  uint8_t r = (uint16_t)_breathingBaseColor.r * brightness / 255;
  uint8_t g = (uint16_t)_breathingBaseColor.g * brightness / 255;
  uint8_t b = (uint16_t)_breathingBaseColor.b * brightness / 255;
  _leds[0] = CRGB(r, g, b);
  FastLED.setBrightness(255);
  FastLED.show();
  _breatheIndex = (_breatheIndex + 1) % _currentBreatheTableLength;
}
```